尚硅谷嵌入式技术之 STM32 网关

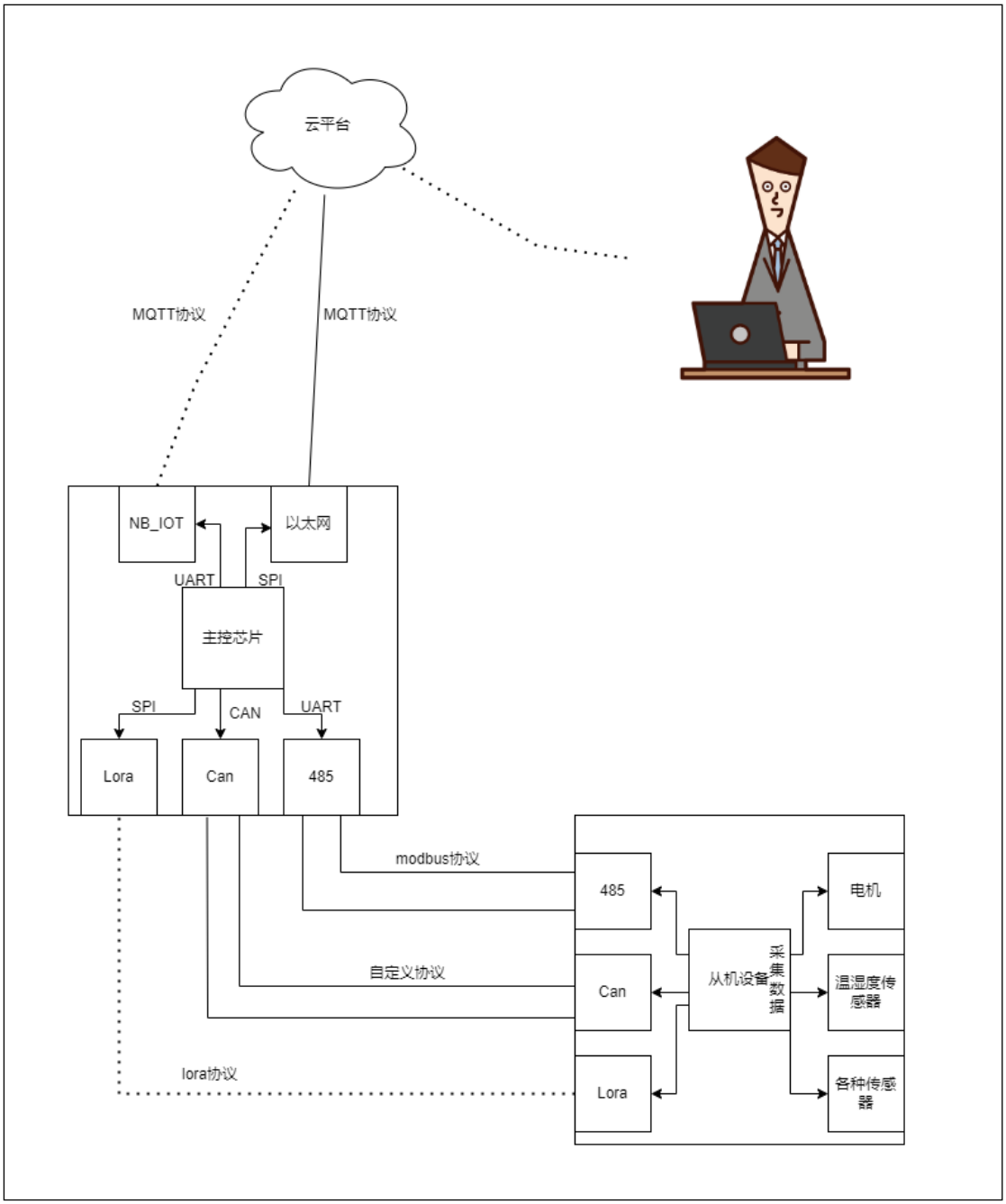
（作者：尚硅谷研究院）

版本：V1.0.0

第 1 章 项目需求

1.1 项目概述

网关主要功能就是让不能联网的物体可以进行联网，就是我们所说的物联网。



第 2 章 硬件架构

2.1 硬件选型

- 主控芯片：STM32F103C8T6
- Nb-Iot 模块：**QS-100**
- 以太网模块：W5500
- LoRa 模块：LoRa 芯片：LLCC68
- 485 模块：MAX13487EESA+T
- CAN 模块：PD1050S

第 3 章 软件架构

在项目中我们使用到了 FreeRTOS，通过两个任务对数据进行收发。

第 4 章 项目使用协议

4.1 MQTT(以太网，NB-IOT)

MQTT（Message Queuing Telemetry Transport）是一种轻量级的、基于发布/订阅模式的消息传输协议，广泛应用于嵌入式系统和物联网（IoT）设备中。由于其协议设计简单、开销小、易于实现，MQTT 在带宽受限、网络不稳定、低功耗设备中尤其受到青睐。

（1）轻量级和高效：

MQTT 消息头小，只有 2 字节的开销，因此非常适合资源有限的设备。

（2）基于发布/订阅模式：

设备可以作为发布者或订阅者，发布者发送消息到主题（Topic），订阅者接收订阅的主题的消息。

（3）可靠性：

提供三种消息传递质量（QoS）等级：

- **QoS 0**：消息最多发送一次，且不确认。
- **QoS 1**：消息至少发送一次，且需要确认。
- **QoS 2**：确保消息仅发送一次，使用四步握手进行确认。

（4）保持连接（Keep Alive）机制：

设备与 MQTT 代理之间通过心跳机制保持连接，减少了网络中断的影响。

（5）持久化会话：

可以保存客户端的订阅信息和消息状态，当客户端重新连接时，仍能接收到未接收到的消息。

4.2 Modbus(RS485)

Modbus 是一种通信协议，广泛应用于工业自动化、控制系统以及嵌入式设备中的数据传输。它是一种基于主/从（Master/Slave）架构的协议，通常用于设备之间的串行通信，如 PLC（可编程逻辑控制器）、传感器、变频器、温控器等工业设备的连接。

Modbus 协议最初用于其 PLC 系统，现在已经成为工业通信的标准之一，尤其在 SCADA（监控和数据采集）系统中广泛使用。

（1）Modbus 的主要特点：

1. **简单**：Modbus 协议相对简单，易于实现。它不依赖于任何特定的硬件或操作系统，通常使用串口通信（RS-232/RS-485）或者以太网（Modbus TCP）。
2. **可靠性**：作为工业协议，Modbus 在实时性、稳定性和数据传输方面具有很好的表现。它有较强的容错性，适合工业现场环境。
3. **开源**：Modbus 是一个开放协议，没有专利和版权，所有设备制造商都可以自由实现和支持该协议。
4. **广泛支持**：由于其标准化和开放性，Modbus 协议被大量设备支持，尤其是工业设备。

（2）Modbus 的工作原理：

Modbus 是一个基于 请求-响应（Request-Response）模型的协议，通信过程一般如下：

- **主站（Master）**：是通信的发起方，发送请求到从站。
- **从站（Slave）**：是通信的响应方，接收主站的请求并返回数据。

（3）常见的 Modbus 协议变种：

1. **Modbus RTU（Remote Terminal Unit）**：

- 这是 Modbus 的传统版本，通常通过串行通信方式（如 RS-232 或 RS-485）实现。
- 数据以二进制格式传输，传输效率较高。
- 校验位：RTU 采用 CRC 校验，确保数据的完整性。
- 适合于低带宽、低延迟的环境。

2. **Modbus ASCII**：

- 类似于 Modbus RTU，但数据以 ASCII 字符形式传输，而不是二进制格式。
- 每个字节以两个 ASCII 字符表示，传输效率较低，但便于调试和查看。
- 使用 LRC 校验来检查数据传输的正确性。

3. **Modbus TCP/IP**：

- 基于以太网（Ethernet）通信，通过 TCP/IP 协议进行数据传输，适合于局域网（LAN）或广域网（WAN）环境。
- 以太网通信通常具有较高的传输速率和更长的传输距离。
- 采用了 Modbus RTU 的协议格式，但封装在 TCP 数据包中。


Modbus通信协议4.0.xlsx


Modbus RTU通讯协议详解

4.3 LoRa(433 频段)

LoRa（Long Range）是一种低功耗广域网（LPWAN）技术，主要用于远程物联网（IoT）设备的无线通信。它由 **Semtech Corporation** 开发，具备长距离、低功耗、低数据速率、较强的抗干扰能力，非常适合用于需要长距离通信并且对功耗有严格要求的场合。

4.4 自定义协议(CAN)

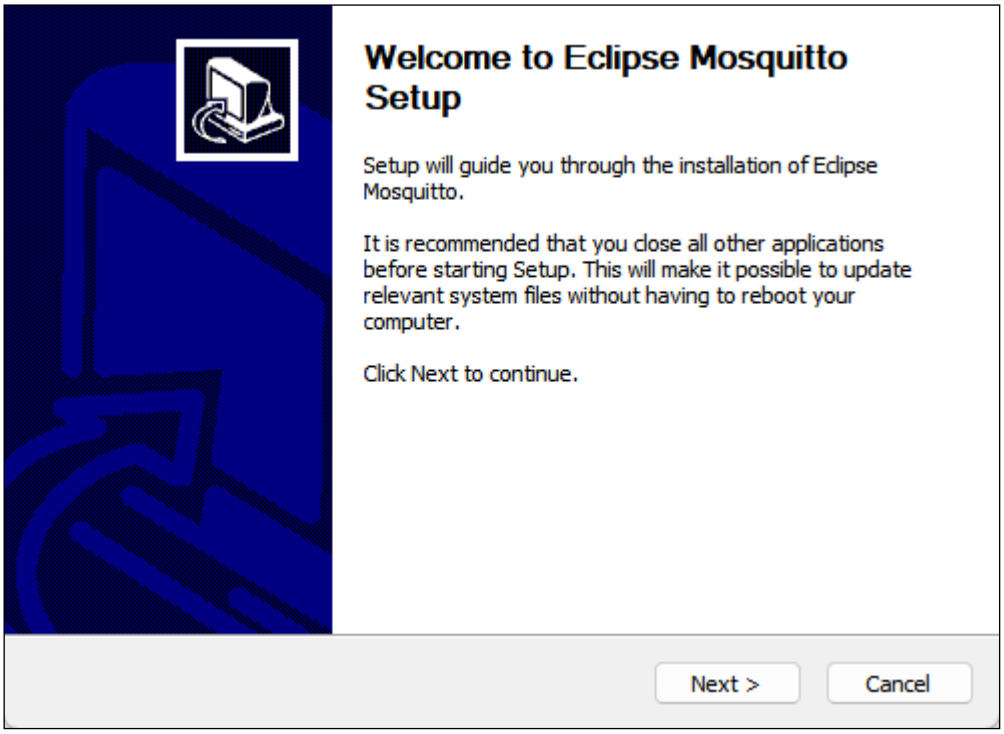
进入企业中难免会碰到要根据自己实际的项目需求而定义出一款适合自家产品的通信协议，在本项目中，我们根据项目需求，基于 can 协议的基础上自定义了自己的协议。

第 5 章 MQTT 使用

5.1 服务端部署

5.1.1 下载安装 mosquitto

Mosquitto 是轻量级命令行工具，方便大家在学习的过程中，轻松的在本机部署 MQTT 的服务端。建议安装在系统默认位置，便于查找和启动。



5.1.2 修改配置参数

Mosquitto 软件默认创建的服务端不能使用其他机器连接，所以需要对其进行配置，找到安装路径：C:\Program Files\mosquitto 。使用文本编辑器打开配置文件（注意需要管理员权限）。在文件的末尾添加两行配置：

```
listener 1883 0.0.0.0
allow_anonymous true
```

5.1.3 启动本地服务端

使用 windows 的 cmd 命令也行:

```
C:\Program Files\mosquitto>mosquitto -c ./mosquitto.conf -v
```

使用电脑进入到 POWER SHELL 中，输入以下命令：

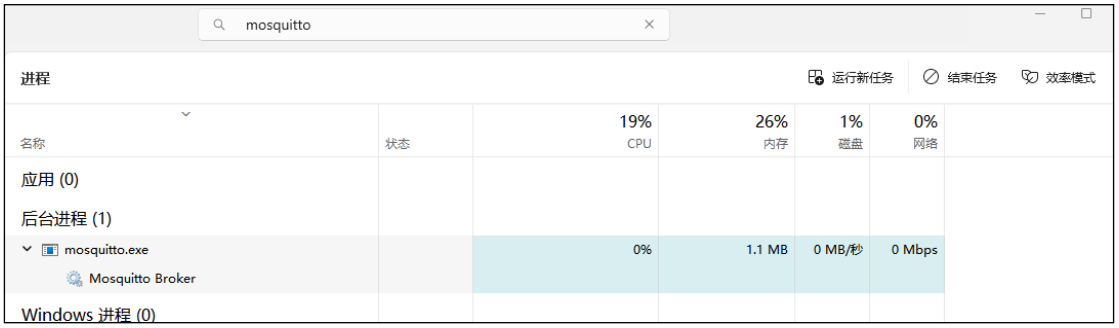
```
mosquitto -v -c 'C:\Program Files\mosquitto\mosquitto.conf'
```

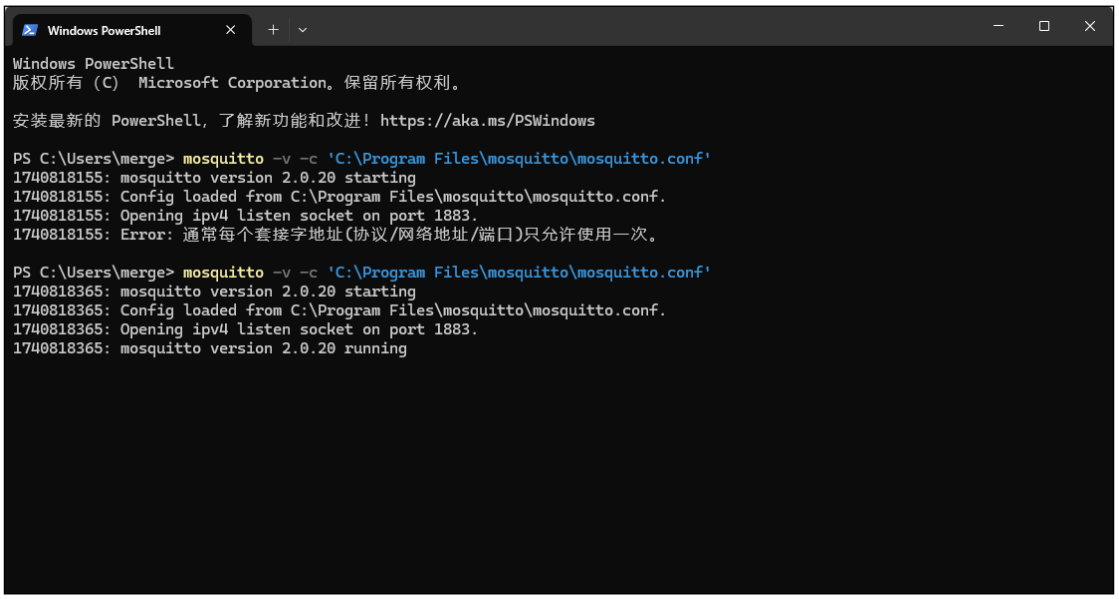
如果出现以下报错信息：

```
PS C:\Users\merge> mosquitto -v -c 'C:\Program
Files\mosquitto\mosquitto.conf'
```

```
1740818155: mosquitto version 2.0.20 starting
1740818155: Config loaded from C:\Program
Files\mosquitto\mosquitto.conf.
1740818155: Opening ipv4 listen socket on port 1883.
1740818155: Error: 通常每个套接字地址(协议/网络地址/端口)只允许使用一次。
```

是因为本机会默认启动一个无法连接的服务端，需要在任务管理器中关闭之后再重新执行：

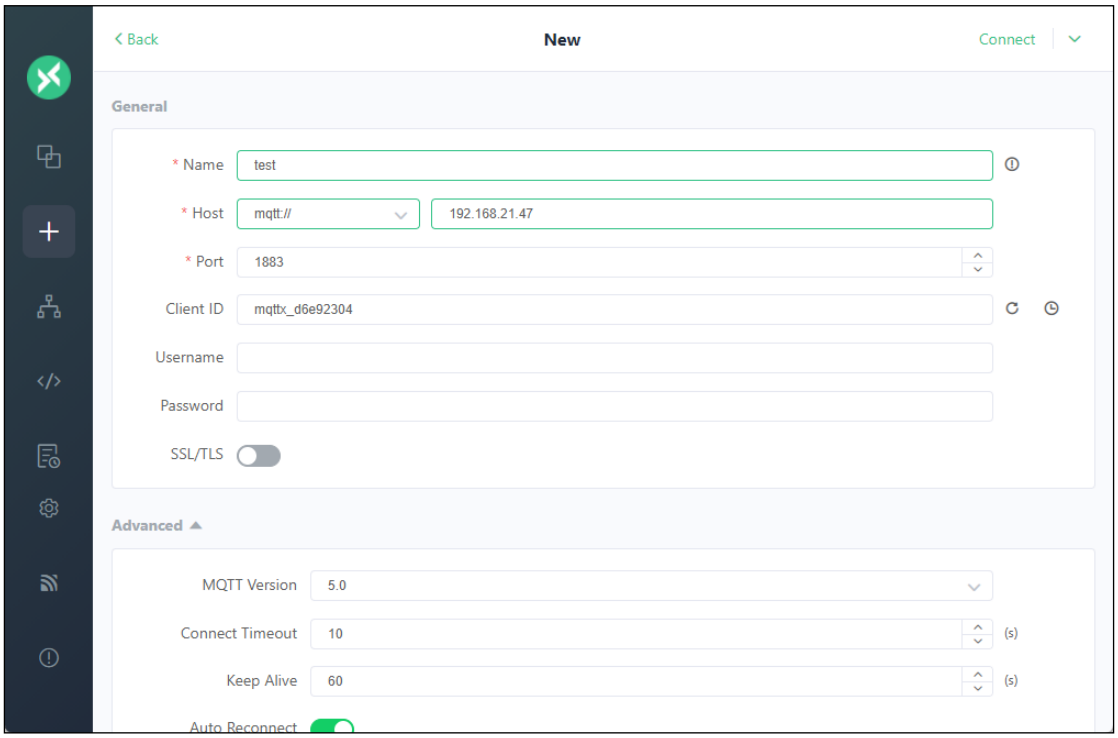




5.1.4 下载安装 MQTTX

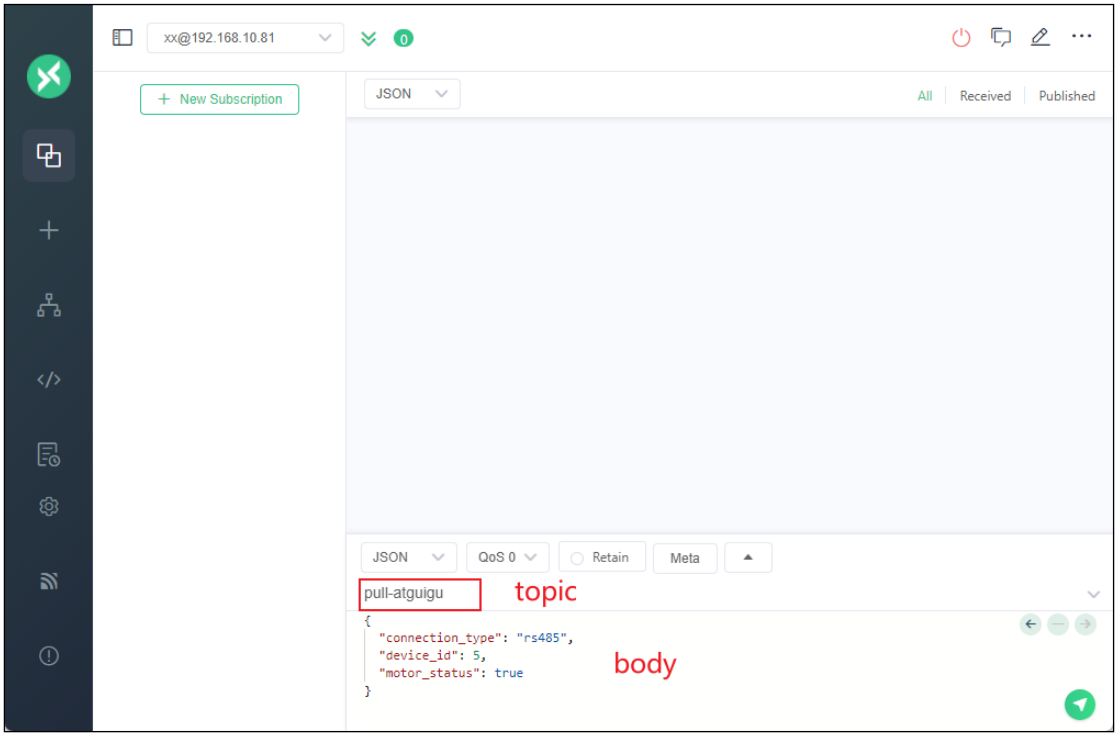
MQTTX 是一款便捷的图形化 MQTT 软件，方便直接使用 windows 窗口进行 MQTT 的调试。

安装完成之后添加新的 MQTT 连接，填写如下参数

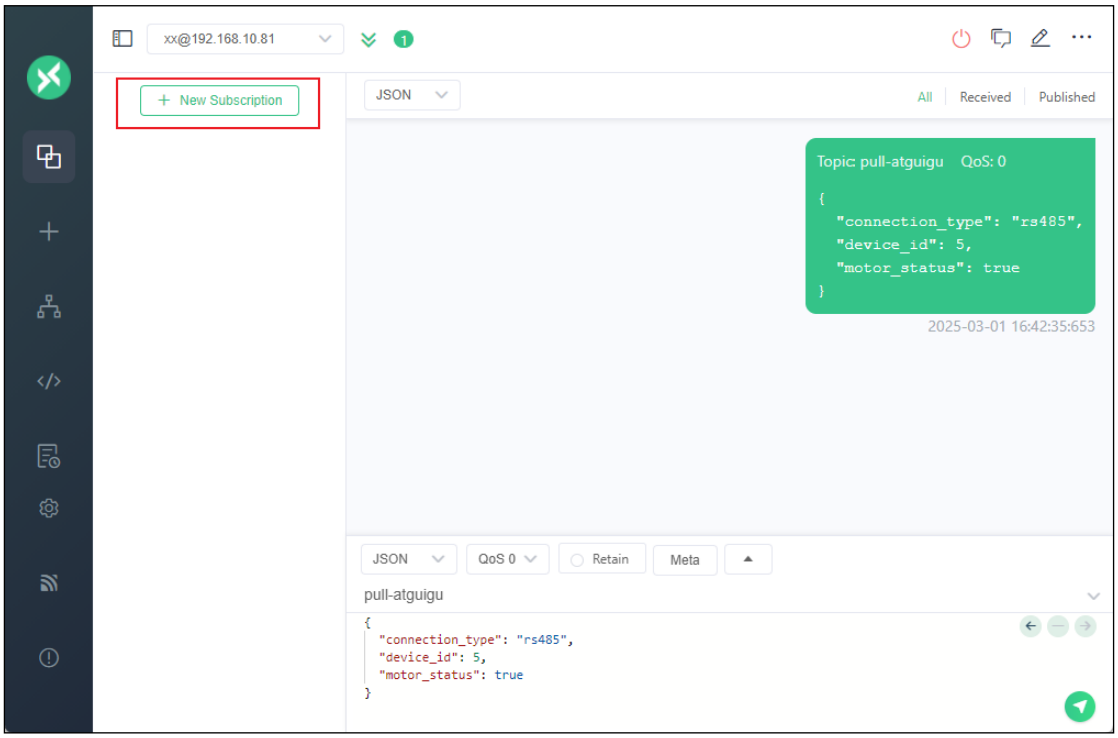


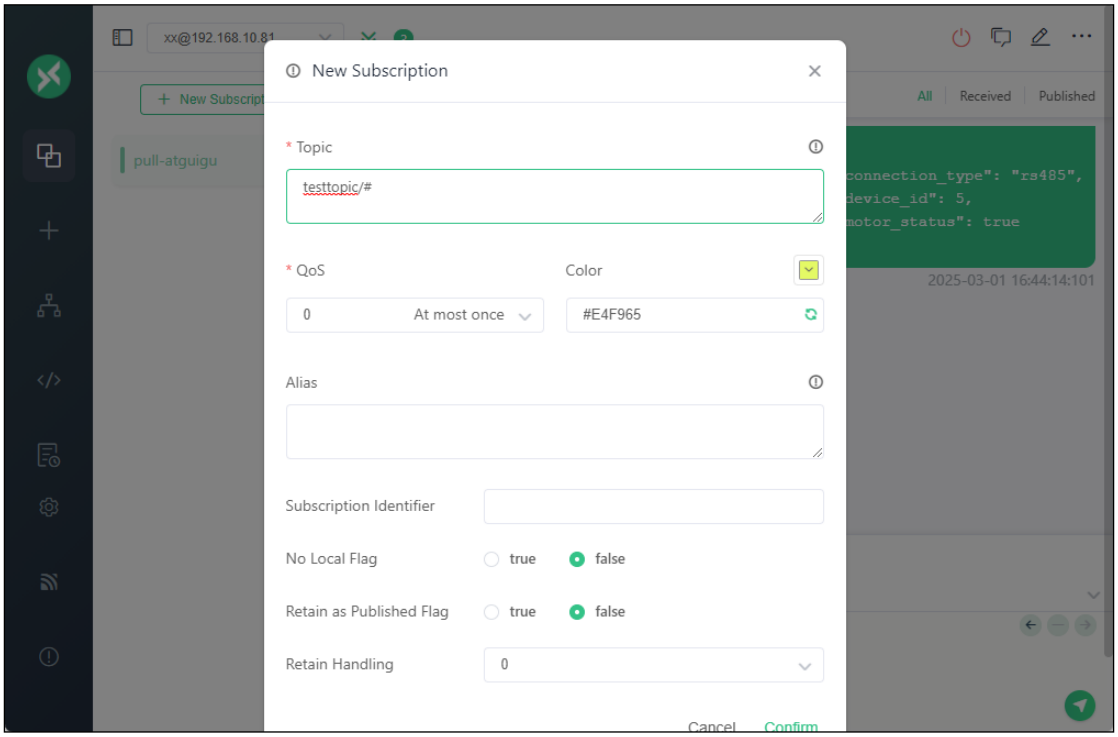
5.1.5 发布和订阅

直接使用图形界面下面的窗口即可发送数据：

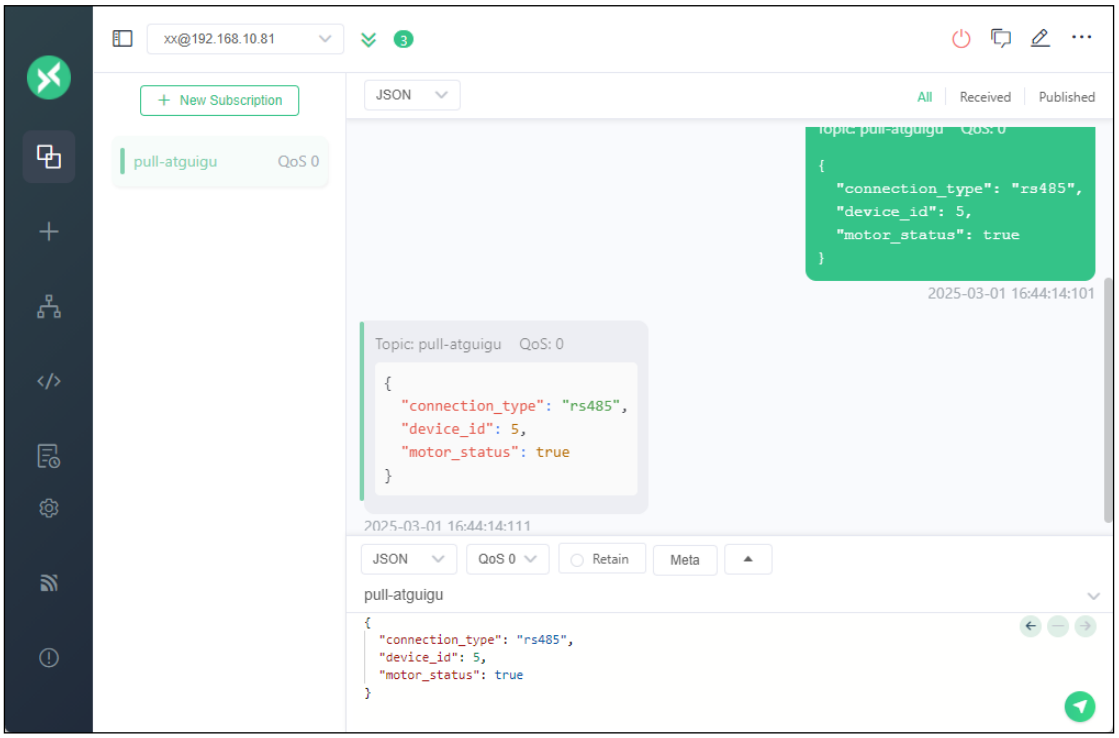


想要订阅主题需要点击左边的订阅窗口：之后填写对应的参数即可。



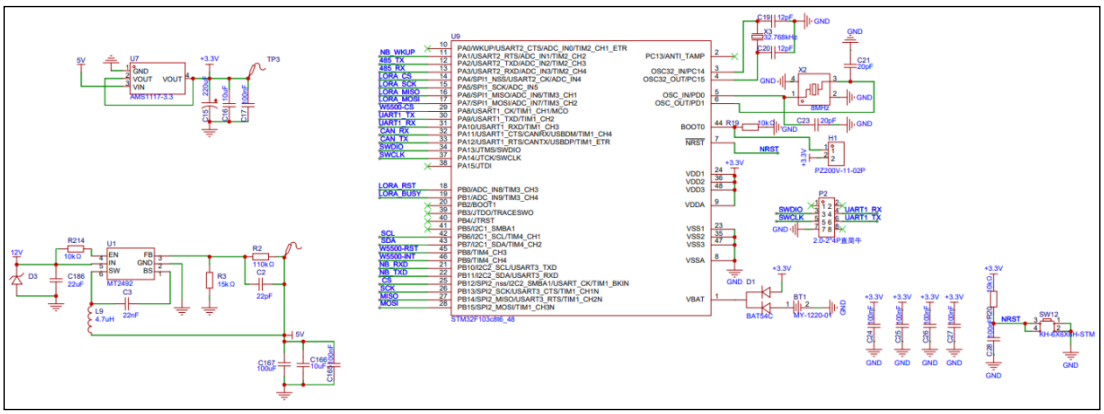


订阅之后再发送即可收到主题数据：

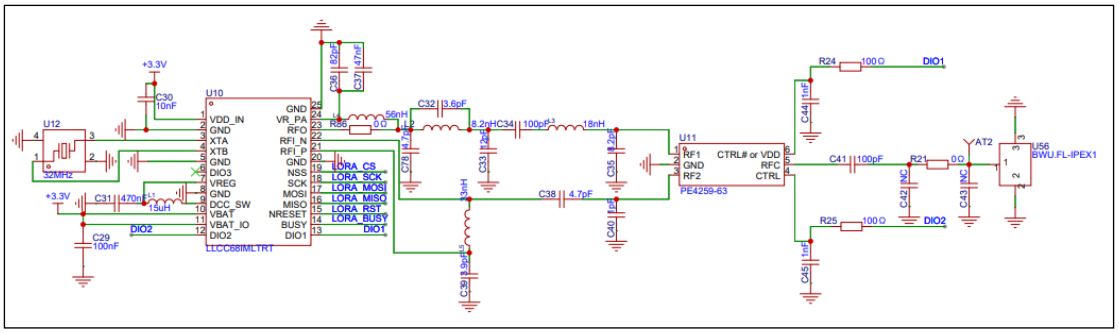


第 6 章 硬件原理图

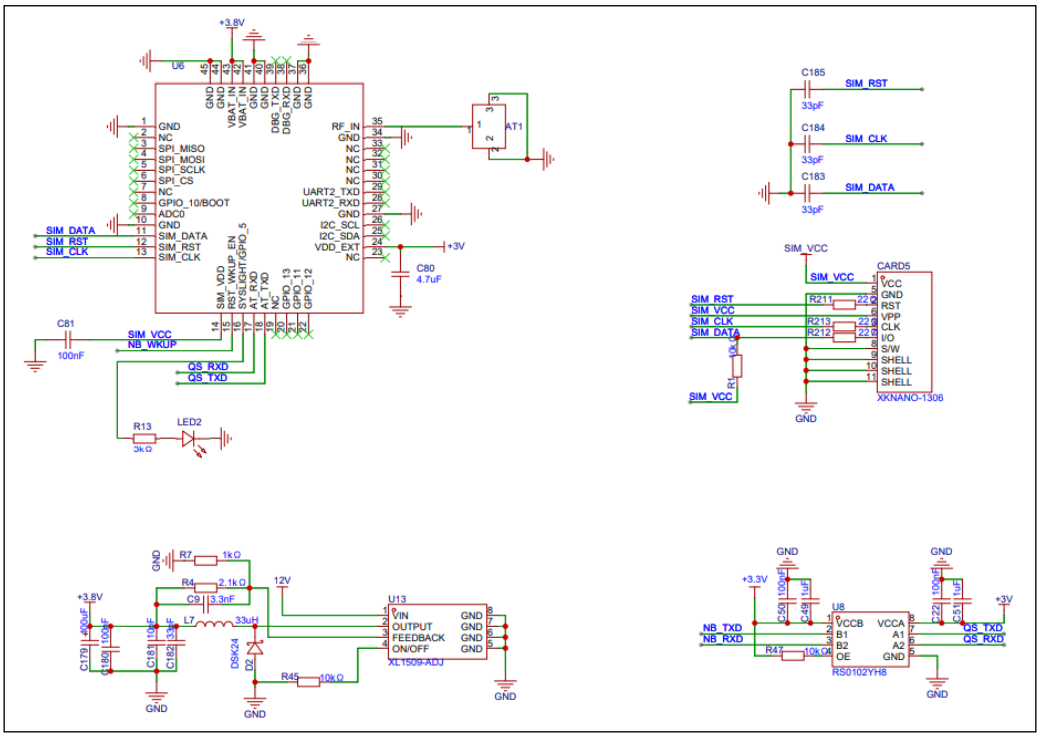
6.1 STM32F103C8T6



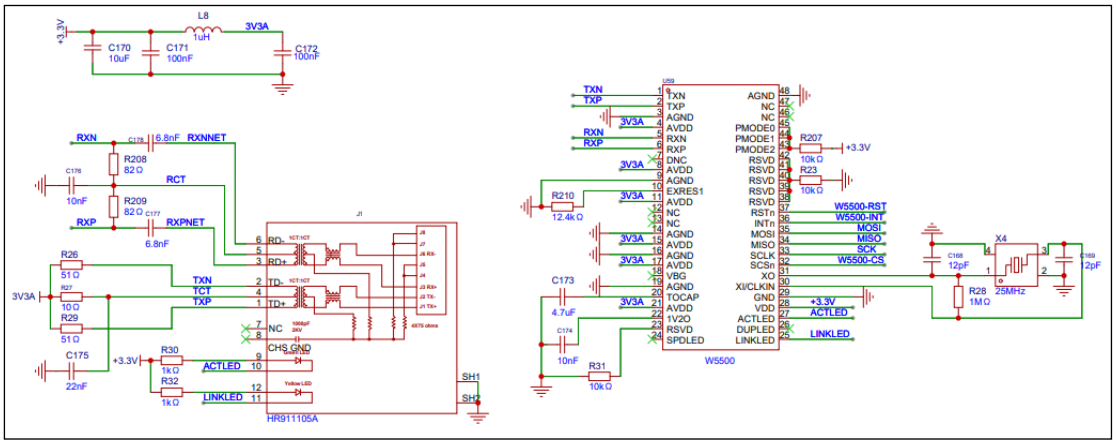
6.2 Lora



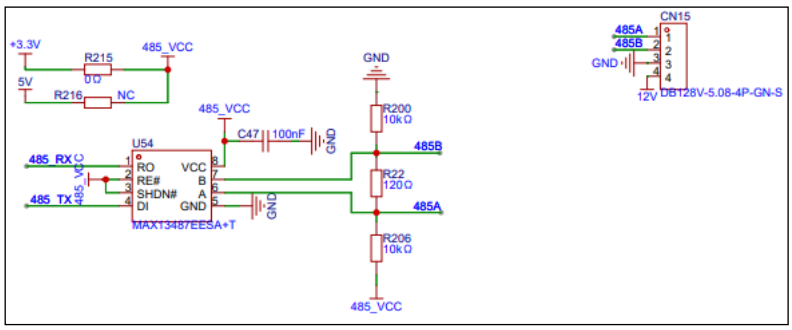
6.3 NB_IOT



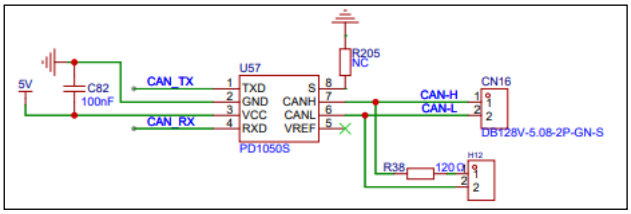
6.4 W5500



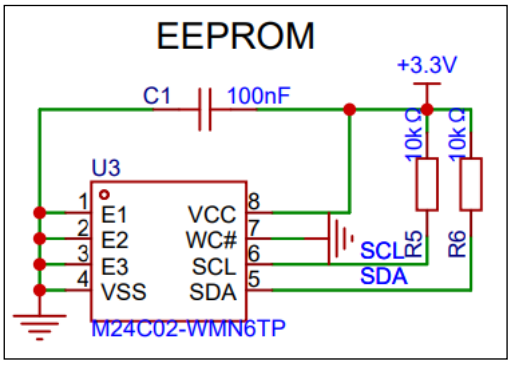
6.5 RS485



6.6 CAN



6.7 EEPROM



第 7 章 RS485 网关实现

7.1 FreeRTOS 移植

首先新建工程，移植我们的 FreeRTOS

7.1.1 FreeRTOS 库

FreeRTOS.zip

7.1.2 FreeRTOSConfig.h

```
/*
 * FreeRTOS V202212.01
 * Copyright (C) 2020 Amazon.com, Inc. or its affiliates.  All Rights
Reserved.
 *
 * Permission is hereby granted, free of charge, to any person
obtaining a copy of
 * this software and associated documentation files (the "Software"),
to deal in
 * the Software without restriction, including without limitation the
rights to
 * use, copy, modify, merge, publish, distribute, sublicense, and/or
sell copies of
```

```
 * the Software, and to permit persons to whom the Software is
furnished to do so,
 * subject to the following conditions:
 *
 * The above copyright notice and this permission notice shall be
included in all
 * copies or substantial portions of the Software.
 *
 * THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
EXPRESS OR
 * IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
MERCHANTABILITY, FITNESS
 * FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR
 * COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER
 * IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR
IN
 * CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
 *
 * https://www.FreeRTOS.org
 * https://github.com/FreeRTOS
 *
 */

#ifndef FREERTOS_CONFIG_H
#define FREERTOS_CONFIG_H

/*-----------------------------------------------------------
 * Application specific definitions.
 *
 * These definitions should be adjusted for your particular hardware
and
 * application requirements.
 *
 * THESE PARAMETERS ARE DESCRIBED WITHIN THE 'CONFIGURATION' SECTION OF
THE
 * FreeRTOS API DOCUMENTATION AVAILABLE ON THE FreeRTOS.org WEB SITE.
 *
 * See http://www.freertos.org/a00110.html
 *----------------------------------------------------------*/

// #define configUSE_PREEMPTION 1
```

```
#define configUSE_IDLE_HOOK 0
#define configUSE_TICK_HOOK 0
#define configCPU_CLOCK_HZ ((unsigned long)72000000)
#define configTICK_RATE_HZ ((TickType_t)1000) /**/
#define configMAX_PRIORITIES (32)
#define configMINIMAL_STACK_SIZE ((unsigned short)128)
#define configTOTAL_HEAP_SIZE ((size_t)(10 * 1024))
#define configMAX_TASK_NAME_LEN (16)
// #define configUSE_TRACE_FACILITY 0
#define configUSE_16_BIT_TICKS 0 /* 时间组最多有 24 个事件 */
#define configIDLE_SHOULD_YIELD 1

/* Set the following definitions to 1 to include the API function, or
zero
to exclude the API function. */

#define INCLUDE_vTaskPrioritySet 1
#define INCLUDE_uxTaskPriorityGet 1
#define INCLUDE_vTaskDelete 1 /* 删除任务 */
#define INCLUDE_vTaskCleanUpResources 0
#define INCLUDE_vTaskSuspend 1 /* 挂起任务(暂停) */
#define INCLUDE_vTaskDelayUntil 1
#define INCLUDE_vTaskDelay 1 /* 延时 */

/* This is the raw value as per the Cortex-M3 NVIC.  Values can be 255
(lowest) to 0 (1?) (highest). */
// #define configKERNEL_INTERRUPT_PRIORITY 255
/* !!!! configMAX_SYSCALL_INTERRUPT_PRIORITY must not be set to
zero !!!!
See http://www.FreeRTOS.org/RTOS-Cortex-M3-M4.html. */
// #define configMAX_SYSCALL_INTERRUPT_PRIORITY 191 /* equivalent to
0xb0, or priority 11. */

/* This is the value being used as per the ST library which permits 16
priority values, 0 to 15.  This must correspond to the
configKERNEL_INTERRUPT_PRIORITY setting.  Here 15 corresponds to the
lowest
NVIC value of 255. */
#define configLIBRARY_KERNEL_INTERRUPT_PRIORITY 15

/* 1. 基础的必须的配置 */
#define xPortPendSVHandler PendSV_Handler /* 用来处理 PendSV 中断:系统的任
务的切换 */
```

```
#define vPortSVCHandler SVC_Handler        /* svc: 实现操作系统的系统调用
*/

#define INCLUDE_xTaskGetSchedulerState 1 /* 用来获取调度状态  */

/* 2. 动态创建任务 */
/* 动态内存分配 */
#define configSUPPORT_DYNAMIC_ALLOCATION 1
#define configUSE_COUNTING_SEMAPHORES 1
#define configFRTOS_MEMORY_SCHEME 4  // 或者选择其他方案，根据你的需求

/* 3. 显示任务的状态 */
#define configUSE_TRACE_FACILITY 1              /* 使用追功能 */
#define configUSE_STATS_FORMATTING_FUNCTIONS 1 /* 字符格式化 */

/* 4. 中断相关 */
/* rtos 内核的中断优先级: 配置为最低优先级, 主要是为了不影响其他中断 */
#define configKERNEL_INTERRUPT_PRIORITY (15 << 4)
/* rtos 管理的中断的阈值 */
#define configMAX_SYSCALL_INTERRUPT_PRIORITY (5 << 4)
/* 是新版本,与上面的等价 */
#define configMAX_API_CALL_INTERRUPT_PRIORITY
configMAX_SYSCALL_INTERRUPT_PRIORITY

/* 5. 时间片轮转 */
/* 抢占式调度 */
#define configUSE_PREEMPTION 1
/* 时间片轮转 */
#define configUSE_TIME_SLICING 1

// /* 6. 任务相关 api */
// #define INCLUDE_xTaskGetHandle 1
// #define INCLUDE_uxTaskGetStackHighWaterMark 1

// /* 7. 运行时间统计 */
// /* 开启时间统计 */

// #define configGENERATE_RUN_TIME_STATS 1
// /* 更改精度的计数器青 0 */
// #define portCONFIGURE_TIMER_FOR_RUN_TIME_STATS() (highCount = 0)
// /* 返回当前计数器的值 */
// #define portGET_RUN_TIME_COUNTER_VALUE() (highCount)

// /* 8. 互斥信号量 */
```

```
// #define configUSE_MUTEXES 1

// /* 9. 队列集 */
// #define configUSE_QUEUE_SETS 1

// /* 10 任务通知 */
// #define configUSE_TASK_NOTIFICATIONS 1
// /* 一个任务最多可以同时缓冲 2 个通知 */
// #define configTASK_NOTIFICATION_ARRAY_ENTRIES 2

// 这个是中断要添加的
//  #include "FreeRTOS.h"
//  #include "task.h"
//  extern void xPortSysTickHandler(void);

// if (xTaskGetSchedulerState() != taskSCHEDULER_NOT_STARTED)
// {
//     xPortSysTickHandler();

#endif /* FREERTOS_CONFIG_H */
// }
```

7.1.3 stm32f1xx_it.c

```
/* USER CODE BEGIN Header */
/**
 ******************************************************************************
 * @file    stm32f1xx_it.c
 * @brief   Interrupt Service Routines.
 ******************************************************************************
 * @attention
 *
 * Copyright (c) 2024 STMicroelectronics.
 * All rights reserved.
 *
 * This software is licensed under terms that can be found in the LICENSE file
 * in the root directory of this software component.
 * If no LICENSE file comes with this software, it is provided AS-IS.
 *
 ******************************************************************************
 */
```

```
/* USER CODE END Header */

/* Includes ------------------------------------------------------------
-------*/
#include "main.h"
#include "stm32f1xx_it.h"
/* Private includes ----------------------------------------------------
-------*/
/* USER CODE BEGIN Includes */

#include "FreeRTOS.h"
#include "task.h"
#include "usart.h"
/* USER CODE END Includes */

/* Private typedef -----------------------------------------------------
-------*/
/* USER CODE BEGIN TD */
/* USER CODE END TD */

/* Private define ------------------------------------------------------
-------*/
/* USER CODE BEGIN PD */

/* USER CODE END PD */

/* Private macro -------------------------------------------------------
-------*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables ---------------------------------------------------
-------*/
/* USER CODE BEGIN PV */

/* USER CODE END PV */

/* Private function prototypes -----------------------------------------
-------*/
/* USER CODE BEGIN PFP */

/* USER CODE END PFP */
```

```
/* Private user code ---------------------------------------------------
-------*/
/* USER CODE BEGIN 0 */

extern void xPortSysTickHandler(void);
extern void MilliTimer_Handler(void);
/* USER CODE END 0 */

/* External variables --------------------------------------------------
-------*/
extern DMA_HandleTypeDef hdma_usart2_rx;
extern DMA_HandleTypeDef hdma_usart2_tx;
extern UART_HandleTypeDef huart1;
extern UART_HandleTypeDef huart2;
extern UART_HandleTypeDef huart3;
/* USER CODE BEGIN EV */

/* USER CODE END EV */

/******************************************************************************
********/
/*           Cortex-M3 Processor Interruption and Exception
Handlers          */
/******************************************************************************
********/
/**
  * @brief This function handles Non maskable interrupt.
  */
void NMI_Handler(void)
{
  /* USER CODE BEGIN NonMaskableInt_IRQn 0 */

  /* USER CODE END NonMaskableInt_IRQn 0 */
  /* USER CODE BEGIN NonMaskableInt_IRQn 1 */
  while (1)
  {
  }
  /* USER CODE END NonMaskableInt_IRQn 1 */
}

/**
  * @brief This function handles Hard fault interrupt.
  */
void HardFault_Handler(void)
```

```
{
  /* USER CODE BEGIN HardFault_IRQn 0 */

  /* USER CODE END HardFault_IRQn 0 */
  while (1)
  {
    /* USER CODE BEGIN W1_HardFault_IRQn 0 */
    /* USER CODE END W1_HardFault_IRQn 0 */
  }
}

/**
  * @brief This function handles Memory management fault.
  */
void MemManage_Handler(void)
{
  /* USER CODE BEGIN MemoryManagement_IRQn 0 */

  /* USER CODE END MemoryManagement_IRQn 0 */
  while (1)
  {
    /* USER CODE BEGIN W1_MemoryManagement_IRQn 0 */
    /* USER CODE END W1_MemoryManagement_IRQn 0 */
  }
}

/**
  * @brief This function handles Prefetch fault, memory access fault.
  */
void BusFault_Handler(void)
{
  /* USER CODE BEGIN BusFault_IRQn 0 */

  /* USER CODE END BusFault_IRQn 0 */
  while (1)
  {
    /* USER CODE BEGIN W1_BusFault_IRQn 0 */
    /* USER CODE END W1_BusFault_IRQn 0 */
  }
}

/**
  * @brief This function handles Undefined instruction or illegal
state.
```

```
  */
void UsageFault_Handler(void)
{
  /* USER CODE BEGIN UsageFault_IRQn 0 */

  /* USER CODE END UsageFault_IRQn 0 */
  while (1)
  {
    /* USER CODE BEGIN W1_UsageFault_IRQn 0 */

    /* USER CODE END W1_UsageFault_IRQn 0 */
  }
}

/**
  * @brief This function handles System service call via SWI
instruction.
  */
// void SVC_Handler(void)
// {
//   /* USER CODE BEGIN SVCall_IRQn 0 */

//   /* USER CODE END SVCall_IRQn 0 */
//   /* USER CODE BEGIN SVCall_IRQn 1 */

//   /* USER CODE END SVCall_IRQn 1 */
// }

/**
  * @brief This function handles Debug monitor.
  */
void DebugMon_Handler(void)
{
  /* USER CODE BEGIN DebugMonitor_IRQn 0 */

  /* USER CODE END DebugMonitor_IRQn 0 */
  /* USER CODE BEGIN DebugMonitor_IRQn 1 */

  /* USER CODE END DebugMonitor_IRQn 1 */
}

/**
  * @brief This function handles Pendable request for system service.
  */
```

```
// void PendSV_Handler(void)
// {
//   /* USER CODE BEGIN PendSV_IRQn 0 */

//   /* USER CODE END PendSV_IRQn 0 */
//   /* USER CODE BEGIN PendSV_IRQn 1 */

//   /* USER CODE END PendSV_IRQn 1 */
// }

/**
  * @brief This function handles System tick timer.
  */
void SysTick_Handler(void)
{
  /* USER CODE BEGIN SysTick_IRQn 0 */

  /* USER CODE END SysTick_IRQn 0 */
  HAL_IncTick();
  /* USER CODE BEGIN SysTick_IRQn 1 */

  if (xTaskGetSchedulerState() != taskSCHEDULER_NOT_STARTED)
  {
    xPortSysTickHandler();
  }
  MilliTimer_Handler();
  /* USER CODE END SysTick_IRQn 1 */
}

/******************************************************************************/
/* STM32F1xx Peripheral Interrupt Handlers                                    */
/* Add here the Interrupt Handlers for the used peripherals.                  */
/* For the available peripheral interrupt handler names,                      */
/* please refer to the startup file (startup_stm32f1xx.s).                    */
/******************************************************************************/

/**
  * @brief This function handles DMA1 channel6 global interrupt.
```

```
  */
void DMA1_Channel6_IRQHandler(void)
{
  /* USER CODE BEGIN DMA1_Channel6_IRQn 0 */

  /* USER CODE END DMA1_Channel6_IRQn 0 */
  HAL_DMA_IRQHandler(&hdma_usart2_rx);
  /* USER CODE BEGIN DMA1_Channel6_IRQn 1 */

  /* USER CODE END DMA1_Channel6_IRQn 1 */
}

/**
  * @brief This function handles DMA1 channel7 global interrupt.
  */
void DMA1_Channel7_IRQHandler(void)
{
  /* USER CODE BEGIN DMA1_Channel7_IRQn 0 */

  /* USER CODE END DMA1_Channel7_IRQn 0 */
  HAL_DMA_IRQHandler(&hdma_usart2_tx);
  /* USER CODE BEGIN DMA1_Channel7_IRQn 1 */

  /* USER CODE END DMA1_Channel7_IRQn 1 */
}

/**
  * @brief This function handles USART1 global interrupt.
  */
void USART1_IRQHandler(void)
{
  /* USER CODE BEGIN USART1_IRQn 0 */

  /* USER CODE END USART1_IRQn 0 */
  HAL_UART_IRQHandler(&huart1);
  /* USER CODE BEGIN USART1_IRQn 1 */

  /* USER CODE END USART1_IRQn 1 */
}

/**
  * @brief This function handles USART2 global interrupt.
  */
void USART2_IRQHandler(void)
```

```
{
  /* USER CODE BEGIN USART2_IRQn 0 */

  /* USER CODE END USART2_IRQn 0 */
  HAL_UART_IRQHandler(&huart2);
  /* USER CODE BEGIN USART2_IRQn 1 */

  /* USER CODE END USART2_IRQn 1 */
}

/**
  * @brief This function handles USART3 global interrupt.
  */
void USART3_IRQHandler(void)
{
  /* USER CODE BEGIN USART3_IRQn 0 */

  /* USER CODE END USART3_IRQn 0 */
  HAL_UART_IRQHandler(&huart3);
  /* USER CODE BEGIN USART3_IRQn 1 */

  /* USER CODE END USART3_IRQn 1 */
}

/* USER CODE BEGIN 1 */

/* USER CODE END 1 */
```

7.2 工具类

7.2.1 Log.h

```
#ifndef __LOG_H__
#define __LOG_H__

#include <stdio.h>

// 日志级别定义
#define LOG_LEVEL_DEBUG 0
#define LOG_LEVEL_INFO  1
#define LOG_LEVEL_WARN  2
#define LOG_LEVEL_ERROR 3
#define LOG_LEVEL_FATAL 4
```

```
// 当前日志级别，默认为 INFO
#ifndef CURRENT_LOG_LEVEL
#define CURRENT_LOG_LEVEL LOG_LEVEL_INFO
#endif

// 日志输出宏
#define LOG(level, format, ...) \
    do { \
            printf(level " [%s]-%d: " format "\n", __FILE__, __LINE__,
##__VA_ARGS__); \
    } while (0)

// 各日志级别宏
#if CURRENT_LOG_LEVEL <= LOG_LEVEL_DEBUG
#define log_debug(format, ...) LOG("DEBUG", format, ##__VA_ARGS__)
#else
#define log_debug(format, ...)
#endif

#if CURRENT_LOG_LEVEL <= LOG_LEVEL_INFO
#define log_info(format, ...) LOG("INFO", format, ##__VA_ARGS__)
#else
#define log_info(format, ...)
#endif

#if CURRENT_LOG_LEVEL <= LOG_LEVEL_WARN
#define log_warn(format, ...) LOG("WARN", format, ##__VA_ARGS__)
#else
#define log_warn(format, ...)
#endif

#if CURRENT_LOG_LEVEL <= LOG_LEVEL_ERROR
#define log_error(format, ...) LOG("ERROR", format, ##__VA_ARGS__)
#else
#define log_error(format, ...)
#endif

#if CURRENT_LOG_LEVEL <= LOG_LEVEL_FATAL
#define log_fatal(format, ...) LOG("FATAL", format, ##__VA_ARGS__)
#else
#define log_fatal(format, ...)
#endif
```

```
#endif // __LOG_H__
```

7.2.2 usart.c

```
/* USER CODE BEGIN Header */
/**
 ******************************************************************************
 * @file    usart.c
 * @brief   This file provides code for the configuration
 *          of the USART instances.
 ******************************************************************************
 * @attention
 *
 * Copyright (c) 2024 STMicroelectronics.
 * All rights reserved.
 *
 * This software is licensed under terms that can be found in the LICENSE file
 * in the root directory of this software component.
 * If no LICENSE file comes with this software, it is provided AS-IS.
 *
 ******************************************************************************
 */
/* USER CODE END Header */
/* Includes ------------------------------------------------------------------*/
#include "usart.h"

/* USER CODE BEGIN 0 */
#include "log.h"
/* USER CODE END 0 */

UART_HandleTypeDef huart1;
UART_HandleTypeDef huart2;
UART_HandleTypeDef huart3;
DMA_HandleTypeDef hdma_usart2_rx;
DMA_HandleTypeDef hdma_usart2_tx;

/* USART1 init function */

void MX_USART1_UART_Init(void)
```

```
{

  /* USER CODE BEGIN USART1_Init 0 */

  /* USER CODE END USART1_Init 0 */

  /* USER CODE BEGIN USART1_Init 1 */

  /* USER CODE END USART1_Init 1 */
  huart1.Instance = USART1;
  huart1.Init.BaudRate = 115200;
  huart1.Init.WordLength = UART_WORDLENGTH_8B;
  huart1.Init.StopBits = UART_STOPBITS_1;
  huart1.Init.Parity = UART_PARITY_NONE;
  huart1.Init.Mode = UART_MODE_TX_RX;
  huart1.Init.HwFlowCtl = UART_HWCONTROL_NONE;
  huart1.Init.OverSampling = UART_OVERSAMPLING_16;
  if (HAL_UART_Init(&huart1) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN USART1_Init 2 */

  /* USER CODE END USART1_Init 2 */

}
/* USART2 init function */

void MX_USART2_UART_Init(void)
{

  /* USER CODE BEGIN USART2_Init 0 */

  /* USER CODE END USART2_Init 0 */

  /* USER CODE BEGIN USART2_Init 1 */

  /* USER CODE END USART2_Init 1 */
  huart2.Instance = USART2;
  huart2.Init.BaudRate = 115200;
  huart2.Init.WordLength = UART_WORDLENGTH_8B;
  huart2.Init.StopBits = UART_STOPBITS_1;
  huart2.Init.Parity = UART_PARITY_NONE;
  huart2.Init.Mode = UART_MODE_TX_RX;
```

```
  huart2.Init.HwFlowCtl = UART_HWCONTROL_NONE;
  huart2.Init.OverSampling = UART_OVERSAMPLING_16;
  if (HAL_UART_Init(&huart2) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN USART2_Init 2 */

  /* USER CODE END USART2_Init 2 */

}
/* USART3 init function */

void MX_USART3_UART_Init(void)
{

  /* USER CODE BEGIN USART3_Init 0 */

  /* USER CODE END USART3_Init 0 */

  /* USER CODE BEGIN USART3_Init 1 */

  /* USER CODE END USART3_Init 1 */
  huart3.Instance = USART3;
  huart3.Init.BaudRate = 9600;
  huart3.Init.WordLength = UART_WORDLENGTH_8B;
  huart3.Init.StopBits = UART_STOPBITS_1;
  huart3.Init.Parity = UART_PARITY_NONE;
  huart3.Init.Mode = UART_MODE_TX_RX;
  huart3.Init.HwFlowCtl = UART_HWCONTROL_NONE;
  huart3.Init.OverSampling = UART_OVERSAMPLING_16;
  if (HAL_UART_Init(&huart3) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN USART3_Init 2 */

  /* USER CODE END USART3_Init 2 */

}

void HAL_UART_MspInit(UART_HandleTypeDef* uartHandle)
{
```

```
GPIO_InitTypeDef GPIO_InitStruct = {0};
if(uartHandle->Instance==USART1)
{
/* USER CODE BEGIN USART1_MspInit 0 */

/* USER CODE END USART1_MspInit 0 */
  /* USART1 clock enable */
  __HAL_RCC_USART1_CLK_ENABLE();

  __HAL_RCC_GPIOA_CLK_ENABLE();
  /**USART1 GPIO Configuration
  PA9     ------> USART1_TX
  PA10     ------> USART1_RX
  */
  GPIO_InitStruct.Pin = GPIO_PIN_9;
  GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
  HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

  GPIO_InitStruct.Pin = GPIO_PIN_10;
  GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

  /* USART1 interrupt Init */
  HAL_NVIC_SetPriority(USART1_IRQn, 4, 0);
  HAL_NVIC_EnableIRQ(USART1_IRQn);
/* USER CODE BEGIN USART1_MspInit 1 */

/* USER CODE END USART1_MspInit 1 */
}
else if(uartHandle->Instance==USART2)
{
/* USER CODE BEGIN USART2_MspInit 0 */

/* USER CODE END USART2_MspInit 0 */
  /* USART2 clock enable */
  __HAL_RCC_USART2_CLK_ENABLE();

  __HAL_RCC_GPIOA_CLK_ENABLE();
  /**USART2 GPIO Configuration
  PA2     ------> USART2_TX
  PA3     ------> USART2_RX
  */
```

```
    GPIO_InitStruct.Pin = GPIO_PIN_2;
    GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

    GPIO_InitStruct.Pin = GPIO_PIN_3;
    GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

    /* USART2 DMA Init */
    /* USART2_RX Init */
    hdma_usart2_rx.Instance = DMA1_Channel6;
    hdma_usart2_rx.Init.Direction = DMA_PERIPH_TO_MEMORY;
    hdma_usart2_rx.Init.PeriphInc = DMA_PINC_DISABLE;
    hdma_usart2_rx.Init.MemInc = DMA_MINC_ENABLE;
    hdma_usart2_rx.Init.PeriphDataAlignment = DMA_PDATAALIGN_BYTE;
    hdma_usart2_rx.Init.MemDataAlignment = DMA_MDATAALIGN_BYTE;
    hdma_usart2_rx.Init.Mode = DMA_NORMAL;
    hdma_usart2_rx.Init.Priority = DMA_PRIORITY_LOW;
    if (HAL_DMA_Init(&hdma_usart2_rx) != HAL_OK)
    {
      Error_Handler();
    }

    __HAL_LINKDMA(uartHandle,hdmarx,hdma_usart2_rx);

    /* USART2_TX Init */
    hdma_usart2_tx.Instance = DMA1_Channel7;
    hdma_usart2_tx.Init.Direction = DMA_MEMORY_TO_PERIPH;
    hdma_usart2_tx.Init.PeriphInc = DMA_PINC_DISABLE;
    hdma_usart2_tx.Init.MemInc = DMA_MINC_ENABLE;
    hdma_usart2_tx.Init.PeriphDataAlignment = DMA_PDATAALIGN_BYTE;
    hdma_usart2_tx.Init.MemDataAlignment = DMA_MDATAALIGN_BYTE;
    hdma_usart2_tx.Init.Mode = DMA_NORMAL;
    hdma_usart2_tx.Init.Priority = DMA_PRIORITY_LOW;
    if (HAL_DMA_Init(&hdma_usart2_tx) != HAL_OK)
    {
      Error_Handler();
    }

    __HAL_LINKDMA(uartHandle,hdmatx,hdma_usart2_tx);

    /* USART2 interrupt Init */
```

```
    HAL_NVIC_SetPriority(USART2_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(USART2_IRQn);
  /* USER CODE BEGIN USART2_MspInit 1 */

  /* USER CODE END USART2_MspInit 1 */
  }
  else if(uartHandle->Instance==USART3)
  {
  /* USER CODE BEGIN USART3_MspInit 0 */

  /* USER CODE END USART3_MspInit 0 */
    /* USART3 clock enable */
    __HAL_RCC_USART3_CLK_ENABLE();

    __HAL_RCC_GPIOB_CLK_ENABLE();
    /**USART3 GPIO Configuration
    PB10     ------> USART3_TX
    PB11     ------> USART3_RX
    */
    GPIO_InitStruct.Pin = GPIO_PIN_10;
    GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

    GPIO_InitStruct.Pin = GPIO_PIN_11;
    GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

    /* USART3 interrupt Init */
    HAL_NVIC_SetPriority(USART3_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(USART3_IRQn);
  /* USER CODE BEGIN USART3_MspInit 1 */

  /* USER CODE END USART3_MspInit 1 */
  }
}

void HAL_UART_MspDeInit(UART_HandleTypeDef* uartHandle)
{

  if(uartHandle->Instance==USART1)
  {
  /* USER CODE BEGIN USART1_MspDeInit 0 */
```

```
/* USER CODE END USART1_MspDeInit 0 */
  /* Peripheral clock disable */
  __HAL_RCC_USART1_CLK_DISABLE();

  /**USART1 GPIO Configuration
  PA9     ------> USART1_TX
  PA10     ------> USART1_RX
  */
  HAL_GPIO_DeInit(GPIOA, GPIO_PIN_9|GPIO_PIN_10);

  /* USART1 interrupt Deinit */
  HAL_NVIC_DisableIRQ(USART1_IRQn);
/* USER CODE BEGIN USART1_MspDeInit 1 */

/* USER CODE END USART1_MspDeInit 1 */
}
else if(uartHandle->Instance==USART2)
{
/* USER CODE BEGIN USART2_MspDeInit 0 */

/* USER CODE END USART2_MspDeInit 0 */
  /* Peripheral clock disable */
  __HAL_RCC_USART2_CLK_DISABLE();

  /**USART2 GPIO Configuration
  PA2     ------> USART2_TX
  PA3     ------> USART2_RX
  */
  HAL_GPIO_DeInit(GPIOA, GPIO_PIN_2|GPIO_PIN_3);

  /* USART2 DMA DeInit */
  HAL_DMA_DeInit(uartHandle->hdmarx);
  HAL_DMA_DeInit(uartHandle->hdmatx);

  /* USART2 interrupt Deinit */
  HAL_NVIC_DisableIRQ(USART2_IRQn);
/* USER CODE BEGIN USART2_MspDeInit 1 */

/* USER CODE END USART2_MspDeInit 1 */
}
else if(uartHandle->Instance==USART3)
{
/* USER CODE BEGIN USART3_MspDeInit 0 */
```

```
  /* USER CODE END USART3_MspDeInit 0 */
    /* Peripheral clock disable */
    __HAL_RCC_USART3_CLK_DISABLE();

    /**USART3 GPIO Configuration
    PB10     ------> USART3_TX
    PB11     ------> USART3_RX
    */
    HAL_GPIO_DeInit(GPIOB, GPIO_PIN_10|GPIO_PIN_11);

    /* USART3 interrupt Deinit */
    HAL_NVIC_DisableIRQ(USART3_IRQn);
  /* USER CODE BEGIN USART3_MspDeInit 1 */

  /* USER CODE END USART3_MspDeInit 1 */
  }
}

/* USER CODE BEGIN 1 */


void HAL_UARTEx_RxEventCallback(UART_HandleTypeDef *huart, uint16_t
Size)
{

  if (huart->Instance == USART1)
  {
      // HAL_UART_Transmit_DMA(&huart1, u1_data, Size);
      // HAL_UARTEx_ReceiveToIdle_DMA(&huart1, u1_data, 1024);
  }
  else if (huart->Instance == USART2)
  {
    if (huart->RxEventType == HAL_UART_RXEVENT_TC || huart->RxEventType
== HAL_UART_RXEVENT_IDLE)
    {
      // 数据传输完成后处理
      extern void Inf_Modbus_IRQ_Callback(uint16_t size);
      Inf_Modbus_IRQ_Callback(Size); // rs485 callback
    }

  }

}
```

```
int fputc(int c, FILE *file)
{
  HAL_UART_Transmit(&huart1, (uint8_t *)&c, 1, 1000);
  return c;
}
/* USER CODE END 1 */
```

7.2.3 JSON 库

cJSON.zip

7.3 Modbus 协议驱动

7.3.1 Inf_Modbus_Port.h

```
#ifndef __INF_MODBUS_PORT_H__
#define __INF_MODBUS_PORT_H__

#include "log.h"
#include "main.h"

// 日志输出
#define Debug_Printf log_debug
#define Error_Printf log_error

void Inf_Modbus_Init(void);
void Inf_Modbus_Send(uint8_t *data, uint16_t len);

#endif
```

7.3.2 Inf_Modbus_Port.c

```
#include "Inf_modbus_Port.h"
#include "FreeRTOSConfig.h"
#include "usart.h"
#include "FreeRTOS.h"
#include "task.h"
#include "semphr.h"
#include <string.h>
```

```
#define RECV_BUFFER_SIZE     256
#define RECV_BUFFER_COUNT    4

#define MODBUS_TASK_NAME     "Modbus_Task"
#define MODBUS_TASK_STACK    512
#define MODBUS_TASK_PRIORITY 2


static uint8_t recv_buf[RECV_BUFFER_SIZE * RECV_BUFFER_COUNT] = {0};
static uint16_t recv_buf_size[RECV_BUFFER_COUNT]              = {0};
static volatile uint8_t current_read_index                    = 0;
static volatile uint8_t current_write_index                   = 0;

static SemaphoreHandle_t modbus_msg_semaphore = {0};

void Inf_Modbus_Task(void *arg)
{

    while (1) {
        if (xSemaphoreTake(modbus_msg_semaphore, portMAX_DELAY) ==
pdTRUE)
        {

            // 调用回调函数处理数据
            extern void Inf_ModBus_ReadHandle(uint8_t *data, uint16_t
dataLen);
            Inf_ModBus_ReadHandle(recv_buf + current_read_index *
RECV_BUFFER_SIZE, recv_buf_size[current_read_index]);

            // 清楚缓冲区
            memset(recv_buf + current_read_index * RECV_BUFFER_SIZE, 0,
RECV_BUFFER_SIZE);
            current_read_index++;
            if (current_read_index >= RECV_BUFFER_COUNT) {
                current_read_index = 0;
            }
        }
    }
}
void Inf_Modbus_IRQ_Callback(uint16_t size)
{
    recv_buf_size[current_write_index++] = size;
    if (current_write_index >= RECV_BUFFER_COUNT) {
        current_write_index = 0;
```

```
    }
    xSemaphoreGiveFromISR(modbus_msg_semaphore, NULL);
    HAL_UARTEx_ReceiveToIdle_DMA(&huart2, recv_buf +
current_write_index * RECV_BUFFER_SIZE, RECV_BUFFER_SIZE);
}
void Inf_Modbus_Init()
{

    modbus_msg_semaphore = xSemaphoreCreateCounting(RECV_BUFFER_COUNT,
0);
    // 对串口的初始化
    HAL_UARTEx_ReceiveToIdle_DMA(&huart2, recv_buf +
current_write_index * RECV_BUFFER_SIZE, RECV_BUFFER_SIZE);

    xTaskCreate(Inf_Modbus_Task,
                MODBUS_TASK_NAME,
                MODBUS_TASK_STACK,
                NULL,
                MODBUS_TASK_PRIORITY,
                NULL);
}

// modbus 发送的代码
void Inf_Modbus_Send(uint8_t *data, uint16_t len)
{
    log_info("send:");
    for (uint8_t i = 0; i < len; i++)
    {
        printf("%#x ", data[i]);
    }
    printf("\n");

    HAL_UART_Transmit_DMA(&huart2, data, len);
    HAL_Delay(600); //为了防止发送太快,导致电机板接收不到数据
}
```

7.3.3 Inf_Modbus.h

```
#ifndef __INF_MODBUS_H__
#define __INF_MODBUS_H__

#include <stdlib.h>
#include <stdio.h>
```

```
#include "main.h"


// 打印宏定义

// 发送数据宏定义
typedef enum {
    M_FAIL    = -1,
    M_SUCCESS = 0,
} M_STATUS;

typedef enum {
    MODBUS_READ_COILS              = 0x01, /** 读取线圈状态 */
    MODBUS_READ_DISCRETE_INPUTS    = 0x02, /** 读+++++++++取离散输入状态
*/
    MODBUS_READ_HOLDING_REGISTERS  = 0x03, /** 读取保持寄存器 */
    MODBUS_READ_INPUT_REGISTERS    = 0x04, /** 读取输入寄存器 */
    MODBUS_WRITE_SINGLE_COIL       = 0x05, /** 写单个线圈 */
    MODBUS_WRITE_SINGLE_REGISTER   = 0x06, /** 写单个保持寄存器 */
    MODBUS_WRITE_MULTIPLE_COILS    = 0x0F, /** 写多个线圈 */
    MODBUS_WRITE_MULTIPLE_REGISTERS = 0x10, /** 写多个保持寄存器 */
} MODBUS_FUNCTION_CODE;

typedef enum {
    MODBUS_ERROR_ILLEGAL_FUNCTION         = 0x01, /** 错误：非法功能 */
    MODBUS_ERROR_ILLEGAL_DATA_ADDRESS     = 0x02, /** 错误：非法数据地址
*/
    MODBUS_ERROR_ILLEGAL_DATA_VALUE       = 0x03, /** 错误：非法数据值 */
    MODBUS_ERROR_SLAVE_DEVICE_FAILURE     = 0x04, /** 错误：从机设备故障
*/
    MODBUS_ERROR_ACKNOWLEDGE              = 0x05, /** 错误：确认（请求已
接收，正在处理） */
    MODBUS_ERROR_SLAVE_DEVICE_BUSY        = 0x06, /** 错误：从机设备忙 */
    MODBUS_ERROR_MEMORY_PARITY_ERROR      = 0x07, /** 错误：存储奇偶校验
错误 */
    MODBUS_ERROR_GATEWAY_PATH_UNAVAILABLE = 0x08, /** 错误：不可用网关路
径 */
    MODBUS_ERROR_GATEWAY_TARGET_FAILED    = 0x09, /** 错误：网关目标设备
响应失败 */
    MODBUS_ERROR_TIMEOUT                  = 0x0A, /** 错误：超时 */
} MODBUS_ERROR_CODE;

M_STATUS Inf_Modbus_Read_Coils(uint8_t slave_addr, uint16_t
start_register, uint16_t read_length);
```

```
M_STATUS Inf_Modbus_Read_Discrete_Inputs(uint8_t slave_addr, uint16_t
start_register, uint16_t read_length);
M_STATUS Inf_Modbus_Read_Holding_Registers(uint8_t slave_addr, uint16_t
start_register, uint16_t read_length);
M_STATUS Inf_Modbus_Read_Input_Registers(uint8_t slave_addr, uint16_t
start_register, uint16_t read_length);
M_STATUS Inf_Modbus_Write_Single_Coil(uint8_t slave_addr, uint16_t
register, uint16_t value);
M_STATUS Inf_Modbus_Write_Single_Register(uint8_t slave_addr, uint16_t
register, uint16_t value);
M_STATUS Inf_Modbus_Write_Multiple_Coil(uint8_t slave_addr, uint16_t
start_register, uint16_t number, uint8_t *data);
M_STATUS Inf_Modbus_Write_Multiple_Register(uint8_t slave_addr,
uint16_t start_register, uint16_t number, uint16_t *data);

void Inf_Modbus_RegisterCallback(MODBUS_FUNCTION_CODE code, void
(*callback)(uint8_t *data, uint16_t datalen));
void Inf_ModBus_ReadHandle(uint8_t *data, uint8_t dataLen);

#endif
```

7.3.4 Inf_Modbus.c

```
#include "Inf_Modbus.h"
#include "stm32f1xx.h"
#include "Inf_Modbus_Port.h"
#include "main.h"
#include <string.h>

#define MODBUS_MAX_FRAME_SIZE 256
static void (*callbacks[8])(uint8_t *, uint16_t);

static M_STATUS Inf_ModBus_SendHandle(uint8_t slave_addr,
MODBUS_FUNCTION_CODE function_code, uint8_t *data, uint8_t data_len);
static uint16_t Inf_Modbus_CRC(const uint8_t *data, uint16_t length);

/// @brief 01 读线圈
/// @param slave_addr 从机地址
/// @param start_register 起始寄存器
/// @param read_length 读取寄存器的长度
M_STATUS Inf_Modbus_Read_Coils(uint8_t slave_addr, uint16_t
start_register, uint16_t read_length)
{
    uint8_t pdata[8] = {0};
```

```
    uint8_t index    = 2;

    if (slave_addr == 0 || slave_addr > 247) { // 从机地址范围是 1-247
        return M_FAIL;
    }
    if (start_register > 0xFFFF) { // 寄存器地址范围是 0x0000-0xFFFF
        return M_FAIL;
    }
    if (read_length == 0 || read_length > 2000) {  // 读取长度范围是 1-
2000
        return M_FAIL;
    }

    pdata[index++] = (uint8_t)(start_register >> 8);
    pdata[index++] = (uint8_t)(start_register & 0xFF);
    pdata[index++] = (uint8_t)(read_length >> 8);
    pdata[index++] = (uint8_t)(read_length & 0xFF);

    Inf_ModBus_SendHandle(slave_addr, MODBUS_READ_COILS, pdata, index);
    return M_SUCCESS;
}

/// @brief 02 读离散输入状态
/// @param slave_addr 从机地址
/// @param start_register 起始寄存器
/// @param read_length 读取寄存器的长度
M_STATUS Inf_Modbus_Read_Discrete_Inputs(uint8_t slave_addr, uint16_t
start_register, uint16_t read_length)
{
    uint8_t pdata[8] = {0};
    uint8_t index    = 2;

    if (slave_addr == 0 || slave_addr > 247) {// 从机地址范围是 1-247
        return M_FAIL;
    }
    if (start_register > 0xFFFF) {  // 寄存器地址范围是 0x0000-0xFFFF
        return M_FAIL;
    }
    if (read_length == 0 || read_length > 2000) { // 读取长度范围是 1-
2000
        return M_FAIL;
    }

    pdata[index++] = (uint8_t)(start_register >> 8);
```

```
    pdata[index++] = (uint8_t)(start_register & 0xFF);
    pdata[index++] = (uint8_t)(read_length >> 8);
    pdata[index++] = (uint8_t)(read_length & 0xFF);

    Inf_ModBus_SendHandle(slave_addr, MODBUS_READ_DISCRETE_INPUTS,
pdata, index);
    return M_SUCCESS;
}

/// @brief 03 读保持寄存器代码
/// @param slave_addr 从机地址
/// @param start_register 起始寄存器
/// @param read_length 读取寄存器的长度
M_STATUS Inf_Modbus_Read_Holding_Registers(uint8_t slave_addr, uint16_t
start_register, uint16_t read_length)
{
    uint8_t pdata[8] = {0};
    uint8_t index    = 2;

    if (slave_addr == 0 || slave_addr > 247) { // 从机地址范围是 1-247
        return M_FAIL;
    }
    if (start_register > 0xFFFF) {// 寄存器地址范围是 0x0000-0xFFFF
        return M_FAIL;
    }
    if (read_length == 0 || read_length > 2000) {  // 读取长度范围是 1-
2000
        return M_FAIL;
    }

    pdata[index++] = (uint8_t)(start_register >> 8);
    pdata[index++] = (uint8_t)(start_register & 0xFF);
    pdata[index++] = (uint8_t)(read_length >> 8);
    pdata[index++] = (uint8_t)(read_length & 0xFF);

    Inf_ModBus_SendHandle(slave_addr, MODBUS_READ_HOLDING_REGISTERS,
pdata, index);
    return M_SUCCESS;
}

/// @brief 04 读输入寄存器代码
/// @param slave_addr 从机地址
/// @param start_register 起始寄存器
/// @param read_length 读取寄存器的长度
```

```
M_STATUS Inf_Modbus_Read_Input_Registers(uint8_t slave_addr, uint16_t
start_register, uint16_t read_length)
{
    uint8_t pdata[8] = {0};
    uint8_t index    = 2;

    if (slave_addr == 0 || slave_addr > 247) { // 从机地址范围是 1-247
        return M_FAIL;
    }
    if (start_register > 0xFFFF) { // 寄存器地址范围是 0x0000-0xFFFF
        return M_FAIL;
    }
    if (read_length == 0 || read_length > 2000) { // 读取长度范围是 1-
2000
        return M_FAIL;
    }

    pdata[index++] = (uint8_t)(start_register >> 8);
    pdata[index++] = (uint8_t)(start_register & 0xFF);
    pdata[index++] = (uint8_t)(read_length >> 8);
    pdata[index++] = (uint8_t)(read_length & 0xFF);
    Inf_ModBus_SendHandle(slave_addr, MODBUS_READ_INPUT_REGISTERS,
pdata, index);
    return M_SUCCESS;
}

/// @brief 05 写单个线圈
/// @param slave_addr 从机地址
/// @param start_register 寄存器
/// @param value 写入的值
M_STATUS Inf_Modbus_Write_Single_Coil(uint8_t slave_addr, uint16_t
start_register, uint16_t value)
{
    uint8_t pdata[8] = {0};
    uint8_t index    = 2;

    if (slave_addr == 0 || slave_addr > 247) { // 从机地址范围是 1-247
        return M_FAIL;
    }
    if (start_register > 0xFFFF) {// 寄存器地址范围是 0x0000-0xFFFF
        return M_FAIL;
    }

    pdata[index++] = (uint8_t)(start_register >> 8);
```

```
    pdata[index++] = (uint8_t)(start_register & 0xFF);
    pdata[index++] = value ? 0xFF : 0x00;
    pdata[index++] = 0;

    Inf_ModBus_SendHandle(slave_addr, MODBUS_WRITE_SINGLE_COIL, pdata,
index);
    return M_SUCCESS;
}

/// @brief 06 写单个寄存器
/// @param slave_addr 从机地址
/// @param start_register 寄存器
/// @param read_length 写入的值
M_STATUS Inf_Modbus_Write_Single_Register(uint8_t slave_addr, uint16_t
start_register, uint16_t value)
{
    uint8_t pdata[8] = {0};
    uint8_t index    = 2;

    if (slave_addr == 0 || slave_addr > 247) { // 从机地址范围是 1-247
        return M_FAIL;
    }
    if (start_register > 0xFFFF) { // 寄存器地址范围是 0x0000-0xFFFF
        return M_FAIL;
    }

    pdata[index++] = (uint8_t)(start_register >> 8);
    pdata[index++] = (uint8_t)(start_register & 0xFF);
    pdata[index++] = (uint8_t)(value >> 8); // 储存读取的寄存器长度
    pdata[index++] = (uint8_t)(value & 0xFF);

    Inf_ModBus_SendHandle(slave_addr, MODBUS_WRITE_SINGLE_REGISTER,
pdata, index);
    return M_SUCCESS;
}

/// @brief 0F 写多个线圈
/// @param slave_addr 从机地址
/// @param start_register 起始线圈地址
/// @param number 线圈数量
/// @param data 控制模式
M_STATUS Inf_Modbus_Write_Multiple_Coil(uint8_t slave_addr, uint16_t
start_register, uint16_t number, uint8_t *data)
{
```

```
    uint8_t pdata[256] = {0};
    uint8_t index      = 2;
    uint8_t write_len  = (uint8_t)((number + 7) / 8); // 确保向上取整

    if (number == 0 || number > 1968) { // 线圈数量范围是 1-1968
        return M_FAIL;
    }
    if (data == NULL) { // 数据指针不能为空
        return M_FAIL;
    }
    if (slave_addr == 0 || slave_addr > 247) { // 从机地址范围是 1-247
        return M_FAIL;
    }
    if (start_register > 0xFFFF) { // 寄存器地址范围是 0x0000-0xFFFF
        return M_FAIL;
    }

    pdata[index++] = (uint8_t)(start_register >> 8);
    pdata[index++] = (uint8_t)(start_register & 0xFF);
    pdata[index++] = (uint8_t)(number >> 8);
    pdata[index++] = (uint8_t)(number & 0xFF);
    pdata[index++] = write_len;

    for (int8_t i = 0; i < write_len; i++) {
        pdata[index++] = data[i];
    }

    Inf_ModBus_SendHandle(slave_addr, MODBUS_WRITE_MULTIPLE_COILS,
pdata, index);
    return M_SUCCESS;
}

/// @brief 10 写多个寄存器
/// @param slave_addr 从机地址
/// @param start_register 起始寄存器地址
/// @param number 寄存器数量
/// @param write_len 写入几个
/// @param control_mode 控制模式
M_STATUS Inf_Modbus_Write_Multiple_Register(uint8_t slave_addr,
uint16_t start_register, uint16_t number, uint16_t *data)
{
    uint8_t pdata[256] = {0};
    uint8_t index      = 2;
```

```
    if (number == 0 || number > 123) { // 线圈数量范围是 1-1968
        return M_FAIL;
    }
    if (data == NULL) { // 数据指针不能为空
        return M_FAIL;
    }
    if (slave_addr == 0 || slave_addr > 247) { // 从机地址范围是 1-247
        return M_FAIL;
    }
    if (start_register > 0xFFFF) { // 寄存器地址范围是 0x0000-0xFFFF
        return M_FAIL;
    }

    pdata[index++] = (uint8_t)(start_register >> 8);
    pdata[index++] = (uint8_t)(start_register & 0xFF);
    pdata[index++] = (uint8_t)(number >> 8);
    pdata[index++] = (uint8_t)(number & 0xFF);

    pdata[index++] = number * 2;

    for (uint8_t i = 0; i < number; i++) {
        pdata[index++] = (uint8_t)(data[i] >> 8);
        pdata[index++] = (uint8_t)(data[i] & 0xFF);
    }
    Inf_ModBus_SendHandle(slave_addr, MODBUS_WRITE_MULTIPLE_REGISTERS,
pdata, index);
    return M_SUCCESS;
}

/// @brief 对要发送的数据进行发送
/// @param slave_addr 从机地址
/// @param function_code 功能码
/// @param data 数据
/// @param data_len 数据的长度
/// @param crc crc 计算
/// @return
static M_STATUS Inf_ModBus_SendHandle(uint8_t slave_addr,
MODBUS_FUNCTION_CODE function_code, uint8_t *data, uint8_t data_len)
{

    data[0] = slave_addr;              // 赋值从设备地址
    data[1] = (uint8_t)function_code; // 赋值功能码
    // 璁＄畻 crc
    uint16_t crc = Inf_Modbus_CRC(data, data_len);
```

```
    data[data_len++] = crc & 0xff;
    data[data_len++] = crc >> 8 & 0xff;
    Inf_Modbus_Send(data, data_len);
    return M_SUCCESS;
}

void Inf_Modbus_RegisterCallback(MODBUS_FUNCTION_CODE code, void
(*callback)(uint8_t *data, uint16_t datalen))
{
    switch (code) {
        case MODBUS_READ_COILS:
            /* 处理读取线圈状态 */
            callbacks[0] = callback;
            break;
        case MODBUS_READ_DISCRETE_INPUTS:
            /* 处理读取离散输入状态? */
            callbacks[1] = callback;
            break;

        case MODBUS_READ_HOLDING_REGISTERS:
            /* 处理读取保持寄存器? */
            callbacks[2] = callback;
            break;

        case MODBUS_READ_INPUT_REGISTERS:
            /* 处理读取输入寄存器? */
            callbacks[3] = callback;
            break;

        case MODBUS_WRITE_SINGLE_COIL:
            /* 处理写单个线圈 */
            callbacks[4] = callback;
            break;

        case MODBUS_WRITE_SINGLE_REGISTER:
            /* 处理写单个保持寄存器 */
            callbacks[5] = callback;
            break;

        case MODBUS_WRITE_MULTIPLE_COILS:
            /* 处理写多个线圈 */
            callbacks[6] = callback;
            break;
```

```
        case MODBUS_WRITE_MULTIPLE_REGISTERS:
            /* 处理写多个保持寄存器 */
            callbacks[7] = callback;
            break;

        default:
             /* 处理未定义的功能码 */
            break;
    }
}

/// @brief 对串口收到的数据进行处理
/// @param data 串口收到的完整数据
/// @param data_len 数据的长度
void Inf_ModBus_ReadHandle(uint8_t *data, uint8_t data_len)
{
    printf("Inf_ModBus_ReadHandle receive data:");
    for (uint8_t i = 0; i < data_len; i++)
    {
        printf("%#x ", data[i]);
    }
    printf("\n");


    // 计算 crc,判断数据是否正确
    uint16_t crc = Inf_Modbus_CRC(data, data_len - 2);
    if (memcmp(&crc, data + data_len - 2, 2) != 0) {
        Error_Printf("modbus crc error");
        return;
    }
    // 错误响应帧
    if (data[1] & 0x80) {
        uint8_t error_code = data[2];  // 错误码位于数据帧的第 3 字节

        switch (error_code) {
            case MODBUS_ERROR_ILLEGAL_FUNCTION:
                Error_Printf("modbus error: illegal function (0x01)");
                break;
            case MODBUS_ERROR_ILLEGAL_DATA_ADDRESS:
                Error_Printf("modbus error: illegal data address
(0x02)");
                break;
            case MODBUS_ERROR_ILLEGAL_DATA_VALUE:
```

```
                Error_Printf("modbus error: illegal data value
(0x03)");
                break;
            case MODBUS_ERROR_SLAVE_DEVICE_FAILURE:
                Error_Printf("modbus error: slave device failure
(0x04)");
                break;
            case MODBUS_ERROR_ACKNOWLEDGE:
                Error_Printf("modbus error: acknowledge (0x05)");
                break;
            case MODBUS_ERROR_SLAVE_DEVICE_BUSY:
                Error_Printf("modbus error: slave device busy (0x06)");
                break;
            case MODBUS_ERROR_MEMORY_PARITY_ERROR:
                Error_Printf("modbus error: memory parity error
(0x07)");
                break;
            case MODBUS_ERROR_GATEWAY_PATH_UNAVAILABLE:
                Error_Printf("modbus error: gateway path unavailable
(0x08)");
                break;
            case MODBUS_ERROR_GATEWAY_TARGET_FAILED:
                Error_Printf("modbus error: gateway target failed
(0x09)");
                break;
            case MODBUS_ERROR_TIMEOUT:
                Error_Printf("modbus error: timeout (0x0A)");
                break;
            default:
                Error_Printf("modbus error: unknown error code
(0x%02X)", error_code);
                break;
        }
        return;
    }

    // 数据正确
    MODBUS_FUNCTION_CODE function_code = (MODBUS_FUNCTION_CODE)data[1];
    switch (function_code) // 根据不同的功能码来调用不同的函数
    {
        case MODBUS_READ_COILS:
            /* 处理读取线圈状态 */
            if (callbacks[0])
            {
```

```
                callbacks[0](data, data_len);
            }
            break;
        case MODBUS_READ_DISCRETE_INPUTS:
            /* 处理读取离散输入状态 */
            if (callbacks[1]) {
                callbacks[1](data, data_len);
            }
            break;

        case MODBUS_READ_HOLDING_REGISTERS:
            /* 处理读取保持寄存器 */
            if (callbacks[2]) {
                callbacks[2](data, data_len);
            }
            break;

        case MODBUS_READ_INPUT_REGISTERS:
            /* 处理读取输入寄存器 */
            if (callbacks[3]) {
                callbacks[3](data, data_len);
            }
            break;

        case MODBUS_WRITE_SINGLE_COIL:
            /* 处理写单个线圈 */
            if (callbacks[4]) {
                callbacks[4](data, data_len);
            }
            break;

        case MODBUS_WRITE_SINGLE_REGISTER:
            /* 处理写单个保持寄存器 */
            if (callbacks[5]) {
                callbacks[5](data, data_len);
            }

            break;

        case MODBUS_WRITE_MULTIPLE_COILS:
            /* 处理写多个线圈 */
            if (callbacks[6]) {
                callbacks[6](data, data_len);
            }
```

```
            break;

        case MODBUS_WRITE_MULTIPLE_REGISTERS:
            /* 处理写多个保持寄存器 */
            if (callbacks[7]) {
                callbacks[7](data, data_len);
            }
            break;

        default:
            /* 处理未定义的功能码 */
            break;
    }
}

/// @brief 485 计算 crc 函数
/// @param data 数据
/// @param length 数据长度
/// @return 0xDDFF, DD 为 crc 的最后一个，FF 为 crc 的前面一个
// temp[7] = Inf_Modbus_CRC(temp, 6) >> 8 & 0xff;
// temp[6] = Inf_Modbus_CRC(temp, 6) & 0xff;

static uint16_t Inf_Modbus_CRC(const uint8_t *data, uint16_t length)
{
    static const uint16_t crc_table[256] = {
        0x0000, 0xC0C1, 0xC181, 0x0140, 0xC301, 0x03C0, 0x0280, 0xC241,
        0xC601, 0x06C0, 0x0780, 0xC741, 0x0500, 0xC5C1, 0xC481, 0x0440,
        0xCC01, 0x0CC0, 0x0D80, 0xCD41, 0x0F00, 0xCFC1, 0xCE81, 0x0E40,
        0x0A00, 0xCAC1, 0xCB81, 0x0B40, 0xC901, 0x09C0, 0x0880, 0xC841,
        0xD801, 0x18C0, 0x1980, 0xD941, 0x1B00, 0xDBC1, 0xDA81, 0x1A40,
        0x1E00, 0xDEC1, 0xDF81, 0x1F40, 0xDD01, 0x1DC0, 0x1C80, 0xDC41,
        0x1400, 0xD4C1, 0xD581, 0x1540, 0xD701, 0x17C0, 0x1680, 0xD641,
        0xD201, 0x12C0, 0x1380, 0xD341, 0x1100, 0xD1C1, 0xD081, 0x1040,
        0xF001, 0x30C0, 0x3180, 0xF141, 0x3300, 0xF3C1, 0xF281, 0x3240,
        0x3600, 0xF6C1, 0xF781, 0x3740, 0xF501, 0x35C0, 0x3480, 0xF441,
        0x3C00, 0xFCC1, 0xFD81, 0x3D40, 0xFF01, 0x3FC0, 0x3E80, 0xFE41,
        0xFA01, 0x3AC0, 0x3B80, 0xFB41, 0x3900, 0xF9C1, 0xF881, 0x3840,
        0x2800, 0xE8C1, 0xE981, 0x2940, 0xEB01, 0x2BC0, 0x2A80, 0xEA41,
        0xEE01, 0x2EC0, 0x2F80, 0xEF41, 0x2D00, 0xEDC1, 0xEC81, 0x2C40,
        0xE401, 0x24C0, 0x2580, 0xE541, 0x2700, 0xE7C1, 0xE681, 0x2640,
        0x2200, 0xE2C1, 0xE381, 0x2340, 0xE101, 0x21C0, 0x2080, 0xE041,
        0xA001, 0x60C0, 0x6180, 0xA141, 0x6300, 0xA3C1, 0xA281, 0x6240,
        0x6600, 0xA6C1, 0xA781, 0x6740, 0xA501, 0x65C0, 0x6480, 0xA441,
        0x6C00, 0xACC1, 0xAD81, 0x6D40, 0xAF01, 0x6FC0, 0x6E80, 0xAE41,
```

```
        0xAA01, 0x6AC0, 0x6B80, 0xAB41, 0x6900, 0xA9C1, 0xA881, 0x6840,
        0x7800, 0xB8C1, 0xB981, 0x7940, 0xBB01, 0x7BC0, 0x7A80, 0xBA41,
        0xBE01, 0x7EC0, 0x7F80, 0xBF41, 0x7D00, 0xBDC1, 0xBC81, 0x7C40,
        0xB401, 0x74C0, 0x7580, 0xB541, 0x7700, 0xB7C1, 0xB681, 0x7640,
        0x7200, 0xB2C1, 0xB381, 0x7340, 0xB101, 0x71C0, 0x7080, 0xB041,
        0x5000, 0x90C1, 0x9181, 0x5140, 0x9301, 0x53C0, 0x5280, 0x9241,
        0x9601, 0x56C0, 0x5780, 0x9741, 0x5500, 0x95C1, 0x9481, 0x5440,
        0x9C01, 0x5CC0, 0x5D80, 0x9D41, 0x5F00, 0x9FC1, 0x9E81, 0x5E40,
        0x5A00, 0x9AC1, 0x9B81, 0x5B40, 0x9901, 0x59C0, 0x5880, 0x9841,
        0x8801, 0x48C0, 0x4980, 0x8941, 0x4B00, 0x8BC1, 0x8A81, 0x4A40,
        0x4E00, 0x8EC1, 0x8F81, 0x4F40, 0x8D01, 0x4DC0, 0x4C80, 0x8C41,
        0x4400, 0x84C1, 0x8581, 0x4540, 0x8701, 0x47C0, 0x4680, 0x8641,
        0x8201, 0x42C0, 0x4380, 0x8341, 0x4100, 0x81C1, 0x8081,
0x4040};

    uint16_t crc = 0xFFFF;
    for (uint16_t pos = 0; pos < length; pos++) {
        uint8_t index = (crc ^ data[pos]) & 0xFF;
        crc           = (crc >> 8) ^ crc_table[index];
    }
    return crc;
}
```

7.4 以太网和 MQTT 消息接收

7.4.1 Mqtt 和以太网库

ETH.zip

7.4.2 Inf_mqtt.c

```
#include "Inf_mqtt.h"
#include "log.h"
#include <usart.h>
#include <stdlib.h>
#include <string.h>

#include "MQTTClient.h"
#include "mqtt_interface.h"
#include "socket.h"
#include "cJSON.h"
#include "FreeRTOS.h"
```

```
#include "task.h"
#include "semphr.h"
#include "w5500.h"

#define TCP_SOCKET  0
#define BUFFER_SIZE 1024
#define TARGET_IP   {44, 232, 241, 40}
#define TARGET_PORT 1883 // mqtt server port

#define PULL_TOPIC  "pull-atguigu"
#define PUSH_TOPIC  "push-atguigu"
#define MAC_ADDR    {110, 120, 130, 142, 150, 162}
#define IP_ADDR     {172, 17, 0, 55}
#define IP_MASK     {255, 255, 255, 0}
#define IP_GATEWAY  {172, 17, 0, 1}

static unsigned char recvBuf[BUFFER_SIZE] = {};
static unsigned char sendBuf[BUFFER_SIZE] = {};

static MQTTClient c;
Network n;

static void (*message_callback)(void *data, size_t data_len);

void messageArrived(MessageData *md)
{

    MQTTMessage *message = md->message;
    // 选择是否打印
    log_debug("%.*s", (int)message->payloadlen, (char
*)message->payload);
    message_callback(message->payload, message->payloadlen);
}

static void ETH_Reset(void)
{

    HAL_GPIO_WritePin(W5500_RST_GPIO_Port, W5500_RST_Pin,
GPIO_PIN_RESET);
    HAL_Delay(800);

    HAL_GPIO_WritePin(W5500_RST_GPIO_Port, W5500_RST_Pin,
GPIO_PIN_SET);
    log_info("W5500 init success\r\n");
```

```
}

static void Inf_ETH_Init(void)
{
    uint8_t mac[6]     = MAC_ADDR;    // myself MAC
    uint8_t ip[4]      = IP_ADDR;     // myself IP
    uint8_t submask[4] = IP_MASK;     // myself submask
    uint8_t gateway[4] = IP_GATEWAY; // myself gateway

    user_register_function();

    ETH_Reset();

    setSHAR(mac);

    setSIPR(ip);

    setSUBR(submask);

    setGAR(gateway);
}

// mqtt 初始化


void Inf_Mqtt_Init(void)
{
    int rc = 0;
    MQTTPacket_connectData data = MQTTPacket_connectData_initializer;
    uint8_t targetIP[4]         = TARGET_IP; // mqtt server IP
    Inf_ETH_Init();                          // 初始化网络接口并配置 IP 地
址
    HAL_Delay(8000);                         // 等待网络稳定

    printf("W5500 init success\r\n");
    // 选择对应的 socket
    NewNetwork(&n, 5);
    rc = ConnectNetwork(&n, targetIP, TARGET_PORT);
    if (rc != SOCK_OK) {
        printf("Error connecting network\n");
    }
    printf("success connected network\n");

    MQTTClientInit(&c, &n, 1000, sendBuf, 1024, recvBuf, 1024);
```

```
    data.willFlag         = 0;
    data.MQTTVersion      = 3;
    data.clientID.cstring = (char *)"stdout-subscriber-zc";
    data.username.cstring = NULL;
    data.password.cstring = NULL;

    data.keepAliveInterval = 60;
    data.cleansession      = 1;

    rc = MQTTConnect(&c, &data);
    if (rc != SUCCESSS) {
        printf("Error connecting MQTT.\n");
    }
    printf("Connected %d\r\n", rc);

    printf("Subscribing to %s\r\n", PULL_TOPIC);
    // 接收来自这个的信息 PULL_TOPIC
    rc = MQTTSubscribe(&c, PULL_TOPIC, QOS0, messageArrived);
    printf("Subscribed %d\r\n", rc);
}

// 循环调用的函数，用来处理接收回调
void Inf_Mqtt_Yield()
{
    MQTTYield(&c, 100);
}

// 通过 mqtt 发送数据到服务器
void Inf_Mqtt_SendData(const char *data)
{

    MQTTMessage message;
    message.dup        = 0;
    message.qos        = QOS0;
    message.retained   = 0;
    message.payload    = (void *)data;
    message.payloadlen = strlen(data);
    MQTTPublish(&c, PUSH_TOPIC, &message);
}

void Inf_Mqtt_RegisterRecvCallback(void (*callback)(void *data, size_t
data_len))
{
```

```
    message_callback = callback;
}
```

7.4.3 Inf_mqtt.h

```
/*
 * @Author: wushengran
 * @Date: 2024-11-20 15:37:35
 * @Description:
 *
 * Copyright (c) 2024 by atguigu, All Rights Reserved.
 */
#ifndef __ETH_H
#define __ETH_H

#include <stdio.h>

void Inf_Mqtt_Init(void);
void Inf_Mqtt_Yield(void);

void Inf_Mqtt_SendData(const char *data);
void Inf_Mqtt_RegisterRecvCallback(void (*callback)(void *data, size_t
data_len));
#endif
```

7.5 应用层实现

7.5.1 App_Task.c

```
#include "App_Task.h"
#include <string.h>
#include "FreeRTOS.h"
#include "task.h"
#include "semphr.h"
#include "usart.h"
#include "cJSON.h"
#include "Inf_mqtt.h"
#include "queue.h"
#include "Inf_Modbus_Port.h"
#include "Inf_Modbus.h"
#include "log.h"

#define START_TASK_NAME         "Start_Task"
#define START_TASK_STACK        128
#define START_TASK_PRIORITY     5
```

```
#define MQTTYIELD_TASK_NAME     "Yield_Task"
#define MQTTYIELD_TASK_STACK    1024
#define MQTTYIELD_TASK_PRIORITY 1

static void App_MQTTYield_Task(void *pvParameters);
static void App_MessageCallback(void *data, size_t datalen);
static void App_Task_ModbusCallback(uint8_t *data, uint16_t datalen);

static void App_Start_Task(void *pvParameters)
{
    // 进入临界区
    Inf_Mqtt_RegisterRecvCallback(App_MessageCallback);
    Inf_Mqtt_Init();
    Inf_Modbus_RegisterCallback(MODBUS_READ_COILS,
App_Task_ModbusCallback);
    Inf_Modbus_RegisterCallback(MODBUS_READ_DISCRETE_INPUTS,
App_Task_ModbusCallback);
    Inf_Modbus_RegisterCallback(MODBUS_READ_HOLDING_REGISTERS,
App_Task_ModbusCallback);
    Inf_Modbus_RegisterCallback(MODBUS_READ_INPUT_REGISTERS,
App_Task_ModbusCallback);
    Inf_Modbus_Init();

    taskENTER_CRITICAL();

    // 创建任务
    xTaskCreate(App_MQTTYield_Task,
                MQTTYIELD_TASK_NAME,
                MQTTYIELD_TASK_STACK,
                NULL,
                MQTTYIELD_TASK_PRIORITY,
                NULL);

    // 销毁自身
    log_info("App_Start_over");

    vTaskDelete(NULL);
    taskEXIT_CRITICAL();
}

// 启动任务
void App_MQTTYield_Task(void *pvParameters)
{
```

```
    while (1) {
        Inf_Mqtt_Yield();
        HAL_Delay(200);
    }
}

static void App_MessageCallback(void *data, size_t datalen)
{
    // 解析 JSON 数据
    cJSON *root = cJSON_ParseWithLength((char *)data, datalen);
    if (root == NULL) {
        log_error("Error parsing JSON string.");
        return;
    }

    // 提取必填字段
    cJSON *connection_type = cJSON_GetObjectItem(root,
"connection_type");
    cJSON *device_id       = cJSON_GetObjectItem(root, "device_id");

    // 检查必填字段是否存在
    if (!cJSON_IsString(connection_type) || !cJSON_IsNumber(device_id))
{
        log_error("Invalid JSON format: missing required fields.");
        cJSON_Delete(root);
        return;
    }

    // 检查连接类型是否为 RS485
    if (strcmp(connection_type->valuestring, "rs485") != 0) {
        log_error("Unsupported connection type: %s",
connection_type->valuestring);
        cJSON_Delete(root);
        return;
    }
    log_info("Connection type: %s", connection_type->valuestring);

    // 获取设备地址
    uint8_t slave_addr = (uint8_t)device_id->valueint;

    // 检查是否为查询请求
    cJSON *motor_status     = cJSON_GetObjectItem(root,
"motor_status");
```

```
    cJSON *target_direction = cJSON_GetObjectItem(root,
"target_direction");
    cJSON *target_speed     = cJSON_GetObjectItem(root,
"target_speed");

    M_STATUS status;

    // 写入线圈状态（电机启停）
    if (cJSON_IsBool(motor_status)) {
        uint16_t value = motor_status->valueint ? 1 : 0; // MODBUS 绾垮湀
鐩舵€佸€?
        status         = Inf_Modbus_Write_Single_Coil(slave_addr, 2,
value);
        if (status != M_SUCCESS) {
            log_error("Failed to write coil status.");
        }

    }

    // 写入线圈状态（目标方向）
    if (cJSON_IsBool(target_direction)) {
        uint16_t value = target_direction->valueint ? 1 : 0; // MODBUS
绾垮湀鐩舵€佸€?
        status         = Inf_Modbus_Write_Single_Coil(slave_addr, 3,
value);
        if (status != M_SUCCESS) {
            log_error("Failed to write coil status for target
direction.");
        }

    }

    // 写入保持寄存器（目标转速）
    if (cJSON_IsNumber(target_speed)) {
        uint16_t value = (uint16_t)target_speed->valueint;
        status         = Inf_Modbus_Write_Single_Register(slave_addr,
2, value);
        if (status != M_SUCCESS) {
            log_error("Failed to write holding register for target
speed.");
        }
    }
```

```
    // 设置完成后，查询所有寄存器状态 ，总共 5 个
    // 查询电机启停
    status = Inf_Modbus_Read_Coils(slave_addr, 2, 1);
    if (status != M_SUCCESS) {
        log_error("Failed to read coil status.");
    }

    // 查询目标方向
    status = Inf_Modbus_Read_Coils(slave_addr, 3, 1);
    if (status != M_SUCCESS) {
        log_error("Failed to read coil status.");
    }

    // 查询当前方向
    status = Inf_Modbus_Read_Discrete_Inputs(slave_addr, 3, 1);
    if (status != M_SUCCESS) {
        log_error("Failed to read discrete input status.");
    }

    // 查询当前转速
    status = Inf_Modbus_Read_Input_Registers(slave_addr, 2, 1);
    if (status != M_SUCCESS) {
        log_error("Failed to read input register.");
    }

    // 查询目标转速
    status = Inf_Modbus_Read_Holding_Registers(slave_addr, 2, 1);
    if (status != M_SUCCESS) {
        log_error("Failed to read holding register.");
    }

    // 释放 JSON 对象
    cJSON_Delete(root);
}

static void App_Task_ModbusCallback(uint8_t *data, uint16_t datalen)
{

    static cJSON *obj = NULL;
    if (obj == NULL) {
        obj = cJSON_CreateObject();
        if (obj == NULL) {
            log_error("Failed to create JSON object.");
            return;
```

```
        }
        log_info("receive device_id: %d", data[0]);
        // 添加固定字段
        cJSON_AddStringToObject(obj, "connection_type", "rs485");
        cJSON_AddNumberToObject(obj, "device_id", data[0]); // 设备地址
为 MODBUS 响应帧的第 1 字节
    }


    // 解析 MODBUS 响应帧的功能码
    uint8_t function_code = data[1];

    // 根据功能码处理数据
    switch (function_code) {
        case MODBUS_READ_COILS: {
            // 处理读取线圈状态的响应
            // 数据格式：[Slave Addr][Func Code][Byte Count][Coil Data]
            if (datalen >= 4) {
                uint8_t coil_data   =
data[3];                                    // 第 3 字节为线圈状态
                cJSON *motor_status = cJSON_GetObjectItem(obj,
"motor_status"); // 获取电机启停状态有没有
                // 如果电机启停状态有，代表进来了一次，这个是第二次进来，那就
是获取目标方向
                if (cJSON_IsBool(motor_status)) {
                    log_info("get target_direction");
                    int8_t target_direction = (coil_data & 0x01) != 0;
// 目标方向（00003）
                    cJSON_AddBoolToObject(obj, "target_direction",
target_direction);
                } else // 如果没有代表第一次进来，那就是获取电机启停
                {
                    log_info("get motor_status");
                    int8_t motor_status = (coil_data & 0x01) != 0; // 电
机启停状态（00002）
                    cJSON_AddBoolToObject(obj, "motor_status",
motor_status);
                }
            }
            break;
        }

        case MODBUS_READ_DISCRETE_INPUTS: {
            // 处理读取离散输入的响应
```

```
            // 数据格式：[Slave Addr][Func Code][Byte Count][Input Data]
            if (datalen >= 4) {
                log_info("get current_direction");
                uint8_t input_data       = data[3];                  //
第 3 字节为离散输入状态
                int8_t current_direction = (input_data & 0x01) != 0; //
当前方向（10003）
                cJSON_AddBoolToObject(obj, "current_direction",
current_direction);
            }
            break;
        }

        case MODBUS_READ_INPUT_REGISTERS: {
            // 处理读取输入寄存器的响应
            // 数据格式：[Slave Addr][Func Code][Byte Count][Register
Data High][Register Data Low]
            if (datalen >= 5) {
                log_info("get current_speed");
                uint16_t current_speed = (data[3] << 8) | data[4]; // 当
前转速（40002）
                cJSON_AddNumberToObject(obj, "current_speed",
current_speed);
            }
            break;
        }

        case MODBUS_READ_HOLDING_REGISTERS: {
            // 处理读取保持寄存器的响应
            // 数据格式：[Slave Addr][Func Code][Byte Count][Register
Data High][Register Data Low]
            if (datalen >= 5) {
                log_info("get target_speed");
                uint16_t target_speed = (data[3] << 8) | data[4]; // 目
标转速（30002）
                cJSON_AddNumberToObject(obj, "target_speed",
target_speed);
            }
            break;
        }

        default:
            log_warn("Unsupported function code: %02X", function_code);
            break;
```

```
    }



    // 检查是否所有字段都已填充
    if (cJSON_HasObjectItem(obj, "motor_status") &&
        cJSON_HasObjectItem(obj, "target_direction") &&
        cJSON_HasObjectItem(obj, "current_direction") &&
        cJSON_HasObjectItem(obj, "current_speed") &&
        cJSON_HasObjectItem(obj, "target_speed")) {
        log_info("All fields filled, sending data.");
        // 将 JSON 对象转换为字符串
        char *json_str = cJSON_Print(obj);
        cJSON_DeleteItemFromObject(obj, "motor_status");
        cJSON_DeleteItemFromObject(obj, "target_direction");
        cJSON_DeleteItemFromObject(obj, "current_direction");
        cJSON_DeleteItemFromObject(obj, "current_speed");
        cJSON_DeleteItemFromObject(obj, "target_speed");
        if (json_str != NULL) {
            log_info("json_str: %s", json_str);
            // 通过 MQTT 发送数据
            Inf_Mqtt_SendData(json_str);
            // 释放 JSON 字符串内存
            cJSON_free(json_str);
        } else {
            log_error("Failed to convert JSON to string.");
        }
    }

}

void App_FreeRTOS_Start()
{
    log_info("FreeRTOS Start");

    // mqtt 的初始化
    xTaskCreate(App_Start_Task,
                START_TASK_NAME,
                START_TASK_STACK,
                NULL,
                START_TASK_PRIORITY,
                NULL);

    vTaskStartScheduler();
```

```
    // 启动任务调度器
    log_info("Task Scheduler Started");
}
```

7.5.2 App_Task.h

```
#ifndef __APP_TASK_H__
#define __APP_TASK_H__

void App_FreeRTOS_Start(void);
#endif
```

7.5.3 Main.c

```
/* USER CODE BEGIN Header */
/**
 ******************************************************************************
 * @file           : main.c
 * @brief          : Main program body
 ******************************************************************************
 * @attention
 *
 * Copyright (c) 2024 STMicroelectronics.
 * All rights reserved.
 *
 * This software is licensed under terms that can be found in the LICENSE file
 * in the root directory of this software component.
 * If no LICENSE file comes with this software, it is provided AS-IS.
 *
 ******************************************************************************
 */
/* USER CODE END Header */
/* Includes ------------------------------------------------------------------*/
#include "main.h"
#include "dma.h"
#include "i2c.h"
#include "spi.h"
#include "usart.h"
```

```
#include "gpio.h"

/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */
#include "App_Task.h"
#include "FreeRTOS.h"
#include "task.h"
/* USER CODE END Includes */

/* Private typedef -----------------------------------------------------------*/
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define ------------------------------------------------------------*/
/* USER CODE BEGIN PD */

/* USER CODE END PD */

/* Private macro -------------------------------------------------------------*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables ---------------------------------------------------------*/

/* USER CODE BEGIN PV */

/* USER CODE END PV */

/* Private function prototypes -----------------------------------------------*/
void SystemClock_Config(void);
/* USER CODE BEGIN PFP */

/* USER CODE END PFP */

/* Private user code ---------------------------------------------------------*/
```

```
/* USER CODE BEGIN 0 */

/* USER CODE END 0 */

/**
 * @brief  The application entry point.
 * @retval int
 */
int main(void)
{
    /* USER CODE BEGIN 1 */

    /* USER CODE END 1 */

    /* MCU Configuration--------------------------------------------------
---------*/

    /* Reset of all peripherals, Initializes the Flash interface and
the Systick. */
    HAL_Init();

    /* USER CODE BEGIN Init */

    /* USER CODE END Init */

    /* Configure the system clock */
    SystemClock_Config();

    /* USER CODE BEGIN SysInit */

    /* USER CODE END SysInit */

    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    MX_DMA_Init();
    MX_USART1_UART_Init();
    MX_SPI1_Init();
    MX_USART2_UART_Init();
    MX_SPI2_Init();
    MX_I2C1_Init();
    MX_USART3_UART_Init();
    /* USER CODE BEGIN 2 */

    App_FreeRTOS_Start();
```

```
    /* USER CODE END 2 */

    /* Infinite loop */
    /* USER CODE BEGIN WHILE */

    while (1) {
        /* USER CODE END WHILE */

        /* USER CODE BEGIN 3 */
    }
    /* USER CODE END 3 */
}

/**
 * @brief System Clock Configuration
 * @retval None
 */
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

    /** Initializes the RCC Oscillators according to the specified
parameters
     * in the RCC_OscInitTypeDef structure.
     */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
    RCC_OscInitStruct.HSEState       = RCC_HSE_ON;
    RCC_OscInitStruct.HSEPredivValue = RCC_HSE_PREDIV_DIV1;
    RCC_OscInitStruct.HSIState       = RCC_HSI_ON;
    RCC_OscInitStruct.PLL.PLLState   = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource  = RCC_PLLSOURCE_HSE;
    RCC_OscInitStruct.PLL.PLLMUL     = RCC_PLL_MUL9;
    if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK) {
        Error_Handler();
    }

    /** Initializes the CPU, AHB and APB buses clocks
     */
    RCC_ClkInitStruct.ClockType      = RCC_CLOCKTYPE_HCLK |
RCC_CLOCKTYPE_SYSCLK | RCC_CLOCKTYPE_PCLK1 | RCC_CLOCKTYPE_PCLK2;
    RCC_ClkInitStruct.SYSCLKSource   = RCC_SYSCLKSOURCE_PLLCLK;
    RCC_ClkInitStruct.AHBCLKDivider  = RCC_SYSCLK_DIV1;
    RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
```

```
    RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

    if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) !=
HAL_OK) {
        Error_Handler();
    }
}

/* USER CODE BEGIN 4 */
void HAL_Delay(uint32_t Delay)
{
    vTaskDelay(Delay / portTICK_PERIOD_MS);
}
/* USER CODE END 4 */

/**
 * @brief  This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void)
{
    /* USER CODE BEGIN Error_Handler_Debug */
    /* User can add his own implementation to report the HAL error
return state */
    __disable_irq();
    while (1) {
    }
    /* USER CODE END Error_Handler_Debug */
}

#ifdef USE_FULL_ASSERT
/**
 * @brief  Reports the name of the source file and the source line
number
 *         where the assert_param error has occurred.
 * @param  file: pointer to the source file name
 * @param  line: assert_param error line source number
 * @retval None
 */
void assert_failed(uint8_t *file, uint32_t line)
{
    /* USER CODE BEGIN 6 */
    /* User can add his own implementation to report the file name and
line number,
```

```
       ex: printf("Wrong parameters value: file %s on line %d\r\n",
file, line) */
    /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */
```

第 8 章 Lora 网关实现

首先新建工程，移植我们的 FreeRTOS

8.1 FreeRTOS 移植

8.1.1 FreeRTOS 库

FreeRTOS.zip

8.1.2 FreeRTOSConfig.h

```
 * FreeRTOS V202212.01
 * Copyright (C) 2020 Amazon.com, Inc. or its affiliates.  All Rights
Reserved.
 *
 * Permission is hereby granted, free of charge, to any person
obtaining a copy of
 * this software and associated documentation files (the "Software"),
to deal in
 * the Software without restriction, including without limitation the
rights to
 * use, copy, modify, merge, publish, distribute, sublicense, and/or
sell copies of
 * the Software, and to permit persons to whom the Software is
furnished to do so,
 * subject to the following conditions:
 *
 * The above copyright notice and this permission notice shall be
included in all
 * copies or substantial portions of the Software.
 *
 * THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
EXPRESS OR
 * IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
MERCHANTABILITY, FITNESS
```

```
 * FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR
 * COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER
 * IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR
IN
 * CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
 *
 * https://www.FreeRTOS.org
 * https://github.com/FreeRTOS
 *
 */

#ifndef FREERTOS_CONFIG_H
#define FREERTOS_CONFIG_H

/*-----------------------------------------------------------
 * Application specific definitions.
 *
 * These definitions should be adjusted for your particular hardware
and
 * application requirements.
 *
 * THESE PARAMETERS ARE DESCRIBED WITHIN THE 'CONFIGURATION' SECTION OF
THE
 * FreeRTOS API DOCUMENTATION AVAILABLE ON THE FreeRTOS.org WEB SITE.
 *
 * See http://www.freertos.org/a00110.html
 *----------------------------------------------------------*/

// #define configUSE_PREEMPTION 1
#define configUSE_IDLE_HOOK 0
#define configUSE_TICK_HOOK 0
#define configCPU_CLOCK_HZ ((unsigned long)72000000)
#define configTICK_RATE_HZ ((TickType_t)1000) /**/
#define configMAX_PRIORITIES (32)
#define configMINIMAL_STACK_SIZE ((unsigned short)128)
#define configTOTAL_HEAP_SIZE ((size_t)(10 * 1024))
#define configMAX_TASK_NAME_LEN (16)
// #define configUSE_TRACE_FACILITY 0
#define configUSE_16_BIT_TICKS 0 /* 时间组最多有 24 个事件 */
#define configIDLE_SHOULD_YIELD 1
```

```
/* Set the following definitions to 1 to include the API function, or
zero
to exclude the API function. */

#define INCLUDE_vTaskPrioritySet 1
#define INCLUDE_uxTaskPriorityGet 1
#define INCLUDE_vTaskDelete 1 /* 删除任务 */
#define INCLUDE_vTaskCleanUpResources 0
#define INCLUDE_vTaskSuspend 1 /* 挂起任务(暂停) */
#define INCLUDE_vTaskDelayUntil 1
#define INCLUDE_vTaskDelay 1 /* 延时 */

/* This is the raw value as per the Cortex-M3 NVIC.  Values can be 255
(lowest) to 0 (1?) (highest). */
// #define configKERNEL_INTERRUPT_PRIORITY 255
/* !!!! configMAX_SYSCALL_INTERRUPT_PRIORITY must not be set to
zero !!!!
See http://www.FreeRTOS.org/RTOS-Cortex-M3-M4.html. */
// #define configMAX_SYSCALL_INTERRUPT_PRIORITY 191 /* equivalent to
0xb0, or priority 11. */

/* This is the value being used as per the ST library which permits 16
priority values, 0 to 15.  This must correspond to the
configKERNEL_INTERRUPT_PRIORITY setting.  Here 15 corresponds to the
lowest
NVIC value of 255. */
#define configLIBRARY_KERNEL_INTERRUPT_PRIORITY 15

/* 1. 基础的必须的配置 */
#define xPortPendSVHandler PendSV_Handler /* 用来处理 PendSV 中断:系统的任
务的切换 */
#define vPortSVCHandler SVC_Handler       /* svc: 实现操作系统的系统调用
*/

#define INCLUDE_xTaskGetSchedulerState 1 /* 用来获取调度状态  */

/* 2. 动态创建任务 */
/* 动态内存分配 */
#define configSUPPORT_DYNAMIC_ALLOCATION 1
#define configUSE_COUNTING_SEMAPHORES 1
#define configFRTOS_MEMORY_SCHEME 4  // 或者选择其他方案，根据你的需求

/* 3. 显示任务的状态 */
#define configUSE_TRACE_FACILITY 1              /* 使用追功能 */
```

```
#define configUSE_STATS_FORMATTING_FUNCTIONS 1 /* 字符格式化 */

/* 4. 中断相关 */
/* rtos 内核的中断优先级: 配置为最低优先级, 主要是为了不影响其他中断 */
#define configKERNEL_INTERRUPT_PRIORITY (15 << 4)
/* rtos 管理的中断的阈值 */
#define configMAX_SYSCALL_INTERRUPT_PRIORITY (5 << 4)
/* 是新版本,与上面的等价 */
#define configMAX_API_CALL_INTERRUPT_PRIORITY
configMAX_SYSCALL_INTERRUPT_PRIORITY

/* 5. 时间片轮转 */
/* 抢占式调度 */
#define configUSE_PREEMPTION 1
/* 时间片轮转 */
#define configUSE_TIME_SLICING 1

// /* 6. 任务相关 api */
// #define INCLUDE_xTaskGetHandle 1
// #define INCLUDE_uxTaskGetStackHighWaterMark 1

// /* 7. 运行时间统计 */
// /* 开启时间统计 */

// #define configGENERATE_RUN_TIME_STATS 1
// /* 更改精度的计数器青 0 */
// #define portCONFIGURE_TIMER_FOR_RUN_TIME_STATS() (highCount = 0)
// /* 返回当前计数器的值 */
// #define portGET_RUN_TIME_COUNTER_VALUE() (highCount)

// /* 8. 互斥信号量 */
// #define configUSE_MUTEXES 1

// /* 9. 队列集 */
// #define configUSE_QUEUE_SETS 1

// /* 10 任务通知 */
// #define configUSE_TASK_NOTIFICATIONS 1
// /* 一个任务最多可以同时缓冲 2 个通知 */
// #define configTASK_NOTIFICATION_ARRAY_ENTRIES 2

// 这个是中断要添加的
//  #include "FreeRTOS.h"
//  #include "task.h"
```

```
//  extern void xPortSysTickHandler(void);

// if (xTaskGetSchedulerState() != taskSCHEDULER_NOT_STARTED)
// {
//     xPortSysTickHandler();
// }

#endif /* FREERTOS_CONFIG_H */
```

8.1.3 stm32f1xx_it.c

```
/* USER CODE BEGIN Header */
/**
 ******************************************************************************
 * @file    stm32f1xx_it.c
 * @brief   Interrupt Service Routines.
 ******************************************************************************
 * @attention
 *
 * Copyright (c) 2024 STMicroelectronics.
 * All rights reserved.
 *
 * This software is licensed under terms that can be found in the LICENSE file
 * in the root directory of this software component.
 * If no LICENSE file comes with this software, it is provided AS-IS.
 *
 ******************************************************************************
 */
/* USER CODE END Header */

/* Includes ------------------------------------------------------------------*/
#include "main.h"
#include "stm32f1xx_it.h"
/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */

#include "FreeRTOS.h"
#include "task.h"
```

```
#include "usart.h"
/* USER CODE END Includes */

/* Private typedef -----------------------------------------------------------*/
/* USER CODE BEGIN TD */
/* USER CODE END TD */

/* Private define ------------------------------------------------------------*/
/* USER CODE BEGIN PD */

/* USER CODE END PD */

/* Private macro -------------------------------------------------------------*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables ---------------------------------------------------------*/
/* USER CODE BEGIN PV */

/* USER CODE END PV */

/* Private function prototypes -----------------------------------------------*/
/* USER CODE BEGIN PFP */

/* USER CODE END PFP */

/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */

extern void xPortSysTickHandler(void);
extern void MilliTimer_Handler(void);
/* USER CODE END 0 */

/* External variables --------------------------------------------------------*/
extern UART_HandleTypeDef huart1;
/* USER CODE BEGIN EV */
```

```
/* USER CODE END EV */

/******************************************************************************
********/
/*           Cortex-M3 Processor Interruption and Exception
Handlers          */
/******************************************************************************
********/
/**
  * @brief This function handles Non maskable interrupt.
  */
void NMI_Handler(void)
{
  /* USER CODE BEGIN NonMaskableInt_IRQn 0 */

  /* USER CODE END NonMaskableInt_IRQn 0 */
  /* USER CODE BEGIN NonMaskableInt_IRQn 1 */
  while (1)
  {
  }
  /* USER CODE END NonMaskableInt_IRQn 1 */
}

/**
  * @brief This function handles Hard fault interrupt.
  */
void HardFault_Handler(void)
{
  /* USER CODE BEGIN HardFault_IRQn 0 */

  /* USER CODE END HardFault_IRQn 0 */
  while (1)
  {
    /* USER CODE BEGIN W1_HardFault_IRQn 0 */
    /* USER CODE END W1_HardFault_IRQn 0 */
  }
}

/**
  * @brief This function handles Memory management fault.
  */
void MemManage_Handler(void)
{
```

```
  /* USER CODE BEGIN MemoryManagement_IRQn 0 */

  /* USER CODE END MemoryManagement_IRQn 0 */
  while (1)
  {
    /* USER CODE BEGIN W1_MemoryManagement_IRQn 0 */
    /* USER CODE END W1_MemoryManagement_IRQn 0 */
  }
}

/**
  * @brief This function handles Prefetch fault, memory access fault.
  */
void BusFault_Handler(void)
{
  /* USER CODE BEGIN BusFault_IRQn 0 */

  /* USER CODE END BusFault_IRQn 0 */
  while (1)
  {
    /* USER CODE BEGIN W1_BusFault_IRQn 0 */
    /* USER CODE END W1_BusFault_IRQn 0 */
  }
}

/**
  * @brief This function handles Undefined instruction or illegal
state.
  */
void UsageFault_Handler(void)
{
  /* USER CODE BEGIN UsageFault_IRQn 0 */

  /* USER CODE END UsageFault_IRQn 0 */
  while (1)
  {
    /* USER CODE BEGIN W1_UsageFault_IRQn 0 */

    /* USER CODE END W1_UsageFault_IRQn 0 */
  }
}

/**
```

```
  * @brief This function handles System service call via SWI
instruction.
  */
// void SVC_Handler(void)
// {
//   /* USER CODE BEGIN SVCall_IRQn 0 */

//   /* USER CODE END SVCall_IRQn 0 */
//   /* USER CODE BEGIN SVCall_IRQn 1 */

//   /* USER CODE END SVCall_IRQn 1 */
// }

/**
  * @brief This function handles Debug monitor.
  */
void DebugMon_Handler(void)
{
  /* USER CODE BEGIN DebugMonitor_IRQn 0 */

  /* USER CODE END DebugMonitor_IRQn 0 */
  /* USER CODE BEGIN DebugMonitor_IRQn 1 */

  /* USER CODE END DebugMonitor_IRQn 1 */
}

/**
  * @brief This function handles Pendable request for system service.
  */
// void PendSV_Handler(void)
// {
//   /* USER CODE BEGIN PendSV_IRQn 0 */

//   /* USER CODE END PendSV_IRQn 0 */
//   /* USER CODE BEGIN PendSV_IRQn 1 */

//   /* USER CODE END PendSV_IRQn 1 */
// }

/**
  * @brief This function handles System tick timer.
  */
void SysTick_Handler(void)
{
```

```
  /* USER CODE BEGIN SysTick_IRQn 0 */

  /* USER CODE END SysTick_IRQn 0 */
  HAL_IncTick();
  /* USER CODE BEGIN SysTick_IRQn 1 */

  if (xTaskGetSchedulerState() != taskSCHEDULER_NOT_STARTED)
  {
    xPortSysTickHandler();
  }
  MilliTimer_Handler();
  /* USER CODE END SysTick_IRQn 1 */
}

/******************************************************************************
********/
/* STM32F1xx Peripheral Interrupt
Handlers                                          */
/* Add here the Interrupt Handlers for the used
peripherals.                  */
/* For the available peripheral interrupt handler
names,                       */
/* please refer to the startup file
(startup_stm32f1xx.s).                    */
/******************************************************************************
********/

/**
  * @brief This function handles USART1 global interrupt.
  */
void USART1_IRQHandler(void)
{
  /* USER CODE BEGIN USART1_IRQn 0 */

  /* USER CODE END USART1_IRQn 0 */
  HAL_UART_IRQHandler(&huart1);
  /* USER CODE BEGIN USART1_IRQn 1 */

  /* USER CODE END USART1_IRQn 1 */
}

/* USER CODE BEGIN 1 */

/* USER CODE END 1 */
```

8.2 工具类

8.2.1 Log.h

```
#ifndef __LOG_H__
#define __LOG_H__

#include <stdio.h>

// 日志级别定义
#define LOG_LEVEL_DEBUG 0
#define LOG_LEVEL_INFO  1
#define LOG_LEVEL_WARN  2
#define LOG_LEVEL_ERROR 3
#define LOG_LEVEL_FATAL 4

// 当前日志级别，默认为 INFO
#ifndef CURRENT_LOG_LEVEL
#define CURRENT_LOG_LEVEL LOG_LEVEL_INFO
#endif

// 日志输出宏
#define LOG(level, format, ...) \
    do { \
            printf(level " [%s]-%d: " format "\n", __FILE__, __LINE__,
##__VA_ARGS__); \
    } while (0)

// 各日志级别宏
#if CURRENT_LOG_LEVEL <= LOG_LEVEL_DEBUG
#define log_debug(format, ...) LOG("DEBUG", format, ##__VA_ARGS__)
#else
#define log_debug(format, ...)
#endif

#if CURRENT_LOG_LEVEL <= LOG_LEVEL_INFO
#define log_info(format, ...) LOG("INFO", format, ##__VA_ARGS__)
#else
#define log_info(format, ...)
#endif

#if CURRENT_LOG_LEVEL <= LOG_LEVEL_WARN
```

```
#define log_warn(format, ...) LOG("WARN", format, ##__VA_ARGS__)
#else
#define log_warn(format, ...)
#endif

#if CURRENT_LOG_LEVEL <= LOG_LEVEL_ERROR
#define log_error(format, ...) LOG("ERROR", format, ##__VA_ARGS__)
#else
#define log_error(format, ...)
#endif

#if CURRENT_LOG_LEVEL <= LOG_LEVEL_FATAL
#define log_fatal(format, ...) LOG("FATAL", format, ##__VA_ARGS__)
#else
#define log_fatal(format, ...)
#endif

#endif // __LOG_H__
```

8.2.2 CJSON

cJSON.zip

8.3 LoRa 协议驱动实现

8.3.1 E220

E220.zip

8.3.2 Inf_Lora.c

```
#include "Inf_LoRa.h"

llcc68_handle_t gs_handle = {
    .spi_init = llcc68_interface_spi_init,
    .spi_deinit = llcc68_interface_spi_deinit,
    .spi_write_read = llcc68_interface_spi_write_read,
    .reset_gpio_init = llcc68_interface_reset_gpio_init,
    .reset_gpio_deinit = llcc68_interface_reset_gpio_deinit,
    .reset_gpio_write = llcc68_interface_reset_gpio_write,
    .busy_gpio_init = llcc68_interface_busy_gpio_init,
    .busy_gpio_deinit = llcc68_interface_busy_gpio_deinit,
```

```
    .busy_gpio_read = llcc68_interface_busy_gpio_read,
    .delay_ms = llcc68_interface_delay_ms,
    .debug_print = llcc68_interface_debug_print,
    .receive_callback = llcc68_interface_receive_callback,
};

void Inf_LoRa_Init(void)
{
    uint8_t res;
    uint32_t reg;
    uint8_t modulation;
    uint8_t config;

    // 判断结构体内部的函数指针是否完整
    /* init the llcc68 */
    res = llcc68_init(&gs_handle);
    if (res != 0)
    {
        llcc68_interface_debug_print("llcc68: init failed.\n");
    }

    // 进入待机模式
    /* enter standby */
    res = llcc68_set_standby(&gs_handle,
LLCC68_CLOCK_SOURCE_XTAL_32MHZ);
    if (res != 0)
    {
        llcc68_interface_debug_print("llcc68: set standby failed.\n");
        (void)llcc68_deinit(&gs_handle);
    }

    // 停止定时器
    /* set stop timer on preamble */
    res = llcc68_set_stop_timer_on_preamble(&gs_handle,
LLCC68_LORA_DEFAULT_STOP_TIMER_ON_PREAMBLE);
    if (res != 0)
    {
        llcc68_interface_debug_print("llcc68: stop timer on preamble
failed.\n");
        (void)llcc68_deinit(&gs_handle);
    }

    // 配置电源模式
    /* set regulator mode */
```

```
    res = llcc68_set_regulator_mode(&gs_handle,
LLCC68_LORA_DEFAULT_REGULATOR_MODE);
    if (res != 0)
    {
        llcc68_interface_debug_print("llcc68: set regulator mode
failed.\n");
        (void)llcc68_deinit(&gs_handle);
    }

    // 配置功率
    /* set pa config */
    res = llcc68_set_pa_config(&gs_handle,
LLCC68_LORA_DEFAULT_PA_CONFIG_DUTY_CYCLE,
LLCC68_LORA_DEFAULT_PA_CONFIG_HP_MAX);
    if (res != 0)
    {
        llcc68_interface_debug_print("llcc68: set pa config
failed.\n");
        (void)llcc68_deinit(&gs_handle);
    }

    // 进入修改模式
    /* enter to stdby xosc mode */
    res = llcc68_set_rx_tx_fallback_mode(&gs_handle,
LLCC68_RX_TX_FALLBACK_MODE_STDBY_XOSC);
    if (res != 0)
    {
        llcc68_interface_debug_print("llcc68: set rx tx fallback mode
failed.\n");
        (void)llcc68_deinit(&gs_handle);
    }
    // 设置 lora 模式
    /* set lora mode */
    res = llcc68_set_packet_type(&gs_handle, LLCC68_PACKET_TYPE_LORA);
    if (res != 0)
    {
        llcc68_interface_debug_print("llcc68: set packet type
failed.\n");
        (void)llcc68_deinit(&gs_handle);
    }

    // 设置输出的参数
    /* set tx params */
```

```
    res = llcc68_set_tx_params(&gs_handle, LLCC68_LORA_DEFAULT_TX_DBM,
LLCC68_LORA_DEFAULT_RAMP_TIME);
    if (res != 0)
    {
        llcc68_interface_debug_print("llcc68: set tx params
failed.\n");
        (void)llcc68_deinit(&gs_handle);
    }

    // 设置调制解调参数
    /* set lora modulation params */
    res = llcc68_set_lora_modulation_params(&gs_handle,
LLCC68_LORA_DEFAULT_SF, LLCC68_LORA_DEFAULT_BANDWIDTH,
                                            LLCC68_LORA_DEFAULT_CR,
LLCC68_LORA_DEFAULT_LOW_DATA_RATE_OPTIMIZE);
    if (res != 0)
    {
        llcc68_interface_debug_print("llcc68: set lora modulation
params failed.\n");
        (void)llcc68_deinit(&gs_handle);
    }

    // 转换频率
    /* convert the frequency */
    res = llcc68_frequency_convert_to_register(&gs_handle,
LLCC68_LORA_DEFAULT_RF_FREQUENCY, (uint32_t *)&reg);
    if (res != 0)
    {
        llcc68_interface_debug_print("llcc68: convert to register
failed.\n");
        (void)llcc68_deinit(&gs_handle);
    }

    // 设置频率
    /* set the frequency */
    res = llcc68_set_rf_frequency(&gs_handle, reg);
    if (res != 0)
    {
        llcc68_interface_debug_print("llcc68: set rf frequency
failed.\n");
        (void)llcc68_deinit(&gs_handle);
    }

    // 设置基地址
```

```
    /* set base address */
    res = llcc68_set_buffer_base_address(&gs_handle, 0x00, 0x00);
    if (res != 0)
    {
        llcc68_interface_debug_print("llcc68: set buffer base address
failed.\n");
        (void)llcc68_deinit(&gs_handle);
    }

    // 设置同步的超时时间
    /* set lora symb num */
    res = llcc68_set_lora_symb_num_timeout(&gs_handle,
LLCC68_LORA_DEFAULT_SYMB_NUM_TIMEOUT);
    if (res != 0)
    {
        llcc68_interface_debug_print("llcc68: set lora symb num timeout
failed.\n");
        (void)llcc68_deinit(&gs_handle);
    }

    // 重置状态
    /* reset stats */
    res = llcc68_reset_stats(&gs_handle, 0x0000, 0x0000, 0x0000);
    if (res != 0)
    {
        llcc68_interface_debug_print("llcc68: reset stats failed.\n");
        (void)llcc68_deinit(&gs_handle);
    }

    // 清除错误
    /* clear device errors */
    res = llcc68_clear_device_errors(&gs_handle);
    if (res != 0)
    {
        llcc68_interface_debug_print("llcc68: clear device errors
failed.\n");
        (void)llcc68_deinit(&gs_handle);
    }

    // 重要: 设置同步字 同步字相同的设备才能通信
    /* set the lora sync word */
    res = llcc68_set_lora_sync_word(&gs_handle,
LLCC68_LORA_DEFAULT_SYNC_WORD);
    if (res != 0)
```

```
    {
        llcc68_interface_debug_print("llcc68: set lora sync word
failed.\n");
        (void)llcc68_deinit(&gs_handle);
    }

    // 输出调制
    /* get tx modulation */
    res = llcc68_get_tx_modulation(&gs_handle, (uint8_t *)&modulation);
    if (res != 0)
    {
        llcc68_interface_debug_print("llcc68: get tx modulation
failed.\n");
        (void)llcc68_deinit(&gs_handle);
    }
    modulation |= 0x04;

    /* set the tx modulation */
    res = llcc68_set_tx_modulation(&gs_handle, modulation);
    if (res != 0)
    {
        llcc68_interface_debug_print("llcc68: set tx modulation
failed.\n");
        (void)llcc68_deinit(&gs_handle);
    }

    // 设置输入的增益
    /* set the rx gain */
    res = llcc68_set_rx_gain(&gs_handle, LLCC68_LORA_DEFAULT_RX_GAIN);
    if (res != 0)
    {
        llcc68_interface_debug_print("llcc68: set rx gain failed.\n");
        (void)llcc68_deinit(&gs_handle);
    }

    // 设置过电流保护
    /* set the ocp */
    res = llcc68_set_ocp(&gs_handle, LLCC68_LORA_DEFAULT_OCP);
    if (res != 0)
    {
        llcc68_interface_debug_print("llcc68: set ocp failed.\n");
        (void)llcc68_deinit(&gs_handle);
    }
```

```
    // 发射器限流配置
    /* get the tx clamp config */
    res = llcc68_get_tx_clamp_config(&gs_handle, (uint8_t *)&config);
    if (res != 0)
    {
        llcc68_interface_debug_print("llcc68: get tx clamp config
failed.\n");
        (void)llcc68_deinit(&gs_handle);
    }
    config |= 0x1E;

    /* set the tx clamp config */
    res = llcc68_set_tx_clamp_config(&gs_handle, config);
    if (res != 0)
    {
        llcc68_interface_debug_print("llcc68: set tx clamp config
failed.\n");
        (void)llcc68_deinit(&gs_handle);
    }

    // 设置默认的 LoRa 模式 => 接收模式
    Inf_LoRa_Enter_RX_Mode();
}

/**
 * return: 1 失败 0 成功
 */
uint8_t Inf_LoRa_Enter_RX_Mode(void)
{
    // 变换引脚
    // HAL_GPIO_WritePin(LoRa_RXEN_GPIO_Port, LoRa_RXEN_Pin,
GPIO_PIN_SET);
    // HAL_GPIO_WritePin(LoRa_TXEN_GPIO_Port, LoRa_TXEN_Pin,
GPIO_PIN_RESET);

    uint8_t setup;

    /* set dio irq */
    if (llcc68_set_dio_irq_params(&gs_handle, LLCC68_IRQ_RX_DONE |
LLCC68_IRQ_TIMEOUT | LLCC68_IRQ_CRC_ERR | LLCC68_IRQ_CAD_DONE |
LLCC68_IRQ_CAD_DETECTED,
                                  LLCC68_IRQ_RX_DONE |
LLCC68_IRQ_TIMEOUT | LLCC68_IRQ_CRC_ERR | LLCC68_IRQ_CAD_DONE |
LLCC68_IRQ_CAD_DETECTED,
```

```
                                    0x0000, 0x0000) != 0)
    {
        return 1;
    }

    /* clear irq status */
    if (llcc68_clear_irq_status(&gs_handle, 0x03FFU) != 0)
    {
        return 1;
    }

    /* set lora packet params */
    if (llcc68_set_lora_packet_params(&gs_handle,
LLCC68_LORA_DEFAULT_PREAMBLE_LENGTH,
                                      LLCC68_LORA_DEFAULT_HEADER,
LLCC68_LORA_DEFAULT_BUFFER_SIZE,
                                      LLCC68_LORA_DEFAULT_CRC_TYPE,
LLCC68_LORA_DEFAULT_INVERT_IQ) != 0)
    {
        return 1;
    }

    /* get iq polarity */
    if (llcc68_get_iq_polarity(&gs_handle, (uint8_t *)&setup) != 0)
    {
        return 1;
    }

    setup |= 1 << 2;

    /* set the iq polarity */
    if (llcc68_set_iq_polarity(&gs_handle, setup) != 0)
    {
        return 1;
    }

    /* start receive */
    if (llcc68_continuous_receive(&gs_handle) != 0)
    {
        return 1;
    }

    return 0;
}
```

```
uint8_t Inf_LoRa_Enter_TX_Mode(void)
{
    // 变换引脚
    // HAL_GPIO_WritePin(LoRa_RXEN_GPIO_Port, LoRa_RXEN_Pin,
GPIO_PIN_RESET);
    // HAL_GPIO_WritePin(LoRa_TXEN_GPIO_Port, LoRa_TXEN_Pin,
GPIO_PIN_SET);

    /* set dio irq */
    if (llcc68_set_dio_irq_params(&gs_handle, LLCC68_IRQ_TX_DONE |
LLCC68_IRQ_TIMEOUT | LLCC68_IRQ_CAD_DONE | LLCC68_IRQ_CAD_DETECTED,
                                  LLCC68_IRQ_TX_DONE |
LLCC68_IRQ_TIMEOUT | LLCC68_IRQ_CAD_DONE | LLCC68_IRQ_CAD_DETECTED,
                                  0x0000, 0x0000) != 0)
    {
        return 1;
    }

    /* clear irq status */
    if (llcc68_clear_irq_status(&gs_handle, 0x03FFU) != 0)
    {
        return 1;
    }

    return 0;
}

uint8_t Inf_LoRa_Send_Data(uint8_t *pData, uint16_t Size)
{
    // 切换到发送模式
    Inf_LoRa_Enter_TX_Mode();

    /* send the data */
    if (llcc68_lora_transmit(&gs_handle,
LLCC68_CLOCK_SOURCE_XTAL_32MHZ,
                             LLCC68_LORA_DEFAULT_PREAMBLE_LENGTH,
LLCC68_LORA_DEFAULT_HEADER,
                             LLCC68_LORA_DEFAULT_CRC_TYPE,
LLCC68_LORA_DEFAULT_INVERT_IQ,
                             (uint8_t *)pData, Size, 0) != 0)
    {
        Inf_LoRa_Enter_RX_Mode();
        return 1;
```

```
    }

    Inf_LoRa_Enter_RX_Mode();

    return 0;
}

uint8_t Inf_LoRa_Receive_Data(uint8_t *pData, uint16_t *Size)
{
    // 调用处理函数
    if (llcc68_irq_handler(&gs_handle) != 0)
    {
        return 1;
    }

    // 判断接收到数据
    if (gs_handle.receive_len > 0)
    {
        // 接收到数据
        memcpy(pData, gs_handle.receive_buf, gs_handle.receive_len);
        *Size = gs_handle.receive_len;

        pData[*Size] = '\0';
        // 处理完接收数据 清空缓冲区
        memset(gs_handle.receive_buf, 0,
sizeof(gs_handle.receive_buf));
        gs_handle.receive_len = 0;
    }
    return 0;
}
```

8.3.3 Inf_Lora.h

```
#ifndef __INF_LORA__
#define __INF_LORA__

#include "driver_llcc68_interface.h"

#define
LLCC68_LORA_DEFAULT_STOP_TIMER_ON_PREAMBLE      LLCC68_BOOL_FALSE
           /**< disable stop timer on preamble */
```

```
#define
LLCC68_LORA_DEFAULT_REGULATOR_MODE              LLCC68_REGULATOR_MODE_D
C_DC_LDO   /**< only ldo */
#define
LLCC68_LORA_DEFAULT_PA_CONFIG_DUTY_CYCLE        0x02
           /**< set +17dBm power */
#define
LLCC68_LORA_DEFAULT_PA_CONFIG_HP_MAX            0x03
           /**< set +17dBm power */
#define
LLCC68_LORA_DEFAULT_TX_DBM                      17
           /**< +17dBm */
#define
LLCC68_LORA_DEFAULT_RAMP_TIME                   LLCC68_RAMP_TIME_10US
           /**< set ramp time 10 us */
#define
LLCC68_LORA_DEFAULT_SF                          LLCC68_LORA_SF_9
           /**< sf9 */
#define
LLCC68_LORA_DEFAULT_BANDWIDTH                   LLCC68_LORA_BANDWIDTH_1
25_KHZ     /**< 125khz */
#define
LLCC68_LORA_DEFAULT_CR                          LLCC68_LORA_CR_4_5
           /**< cr4/5 */
#define
LLCC68_LORA_DEFAULT_LOW_DATA_RATE_OPTIMIZE      LLCC68_BOOL_FALSE
           /**< disable low data rate optimize */
#define
LLCC68_LORA_DEFAULT_RF_FREQUENCY                480000000U
           /**< 480000000Hz */
#define
LLCC68_LORA_DEFAULT_SYMB_NUM_TIMEOUT            0
           /**< 0 */
// 特别重要: 同步字 => 0x3444U
#define
LLCC68_LORA_DEFAULT_SYNC_WORD                   0x3444U
           /**< public network */
#define
LLCC68_LORA_DEFAULT_RX_GAIN                     0x94
           /**< common rx gain */
#define
LLCC68_LORA_DEFAULT_OCP                         0x38
           /**< 140 mA */
```

```
#define
LLCC68_LORA_DEFAULT_PREAMBLE_LENGTH             12
          /**< 12 */
#define
LLCC68_LORA_DEFAULT_HEADER                      LLCC68_LORA_HEADER_EXPL
ICIT      /**< explicit header */
#define
LLCC68_LORA_DEFAULT_BUFFER_SIZE                 255
          /**< 255 */
#define
LLCC68_LORA_DEFAULT_CRC_TYPE                    LLCC68_LORA_CRC_TYPE_ON
          /**< crc on */
#define
LLCC68_LORA_DEFAULT_INVERT_IQ                   LLCC68_BOOL_FALSE
          /**< disable invert iq */
#define
LLCC68_LORA_DEFAULT_CAD_SYMBOL_NUM              LLCC68_LORA_CAD_SYMBOL_
NUM_2     /**< 2 symbol */
#define
LLCC68_LORA_DEFAULT_CAD_DET_PEAK                24
          /**< 24 */
#define
LLCC68_LORA_DEFAULT_CAD_DET_MIN                 10
          /**< 10 */
#define
LLCC68_LORA_DEFAULT_START_MODE                  LLCC68_START_MODE_WARM
          /**< warm mode */
#define
LLCC68_LORA_DEFAULT_RTC_WAKE_UP                 LLCC68_BOOL_TRUE
          /**< enable rtc wake up */


void Inf_LoRa_Init(void);

uint8_t Inf_LoRa_Enter_RX_Mode(void);

uint8_t Inf_LoRa_Enter_TX_Mode(void);

uint8_t Inf_LoRa_Send_Data(uint8_t *pData, uint16_t Size);

uint8_t Inf_LoRa_Receive_Data(uint8_t *pData, uint16_t *Size);

#endif // __INT_LORA__
```

8.4 Modbus 协议驱动

只需要修改底层数据发送逻辑为 LoRa 即可。

8.4.1 Inf_Modbus.c

```
#include "Inf_Modbus.h"
#include "stm32f1xx.h"
#include "Inf_Modbus_Port.h"
#include "main.h"
#include <string.h>

#define MODBUS_MAX_FRAME_SIZE 256
static void (*callbacks[8])(uint8_t *, uint16_t);

static M_STATUS Inf_ModBus_SendHandle(uint8_t slave_addr,
MODBUS_FUNCTION_CODE function_code, uint8_t *data, uint8_t data_len);
static uint16_t Inf_Modbus_CRC(const uint8_t *data, uint16_t length);

/// @brief 01 读线圈
/// @param slave_addr 从机地址
/// @param start_register 起始寄存器
/// @param read_length 读取寄存器的长度
M_STATUS Inf_Modbus_Read_Coils(uint8_t slave_addr, uint16_t
start_register, uint16_t read_length)
{
    uint8_t pdata[8] = {0};
    uint8_t index    = 2;

    if (slave_addr == 0 || slave_addr > 247) { // 从机地址范围是 1-247
        return M_FAIL;
    }
    if (start_register > 0xFFFF) { // 寄存器地址范围是 0x0000-0xFFFF
        return M_FAIL;
    }
    if (read_length == 0 || read_length > 2000) {  // 读取长度范围是 1-
2000
        return M_FAIL;
    }

    pdata[index++] = (uint8_t)(start_register >> 8);
    pdata[index++] = (uint8_t)(start_register & 0xFF);
    pdata[index++] = (uint8_t)(read_length >> 8);
    pdata[index++] = (uint8_t)(read_length & 0xFF);
```

```
    Inf_ModBus_SendHandle(slave_addr, MODBUS_READ_COILS, pdata, index);
    return M_SUCCESS;
}

/// @brief 02 读离散输入状态
/// @param slave_addr 从机地址
/// @param start_register 起始寄存器
/// @param read_length 读取寄存器的长度
M_STATUS Inf_Modbus_Read_Discrete_Inputs(uint8_t slave_addr, uint16_t
start_register, uint16_t read_length)
{
    uint8_t pdata[8] = {0};
    uint8_t index    = 2;

    if (slave_addr == 0 || slave_addr > 247) {// 从机地址范围是 1-247
        return M_FAIL;
    }
    if (start_register > 0xFFFF) {  // 寄存器地址范围是 0x0000-0xFFFF
        return M_FAIL;
    }
    if (read_length == 0 || read_length > 2000) { // 读取长度范围是 1-
2000
        return M_FAIL;
    }

    pdata[index++] = (uint8_t)(start_register >> 8);
    pdata[index++] = (uint8_t)(start_register & 0xFF);
    pdata[index++] = (uint8_t)(read_length >> 8);
    pdata[index++] = (uint8_t)(read_length & 0xFF);

    Inf_ModBus_SendHandle(slave_addr, MODBUS_READ_DISCRETE_INPUTS,
pdata, index);
    return M_SUCCESS;
}

/// @brief 03 读保持寄存器代码
/// @param slave_addr 从机地址
/// @param start_register 起始寄存器
/// @param read_length 读取寄存器的长度
M_STATUS Inf_Modbus_Read_Holding_Registers(uint8_t slave_addr, uint16_t
start_register, uint16_t read_length)
{
    uint8_t pdata[8] = {0};
    uint8_t index    = 2;
```

```
    if (slave_addr == 0 || slave_addr > 247) { // 从机地址范围是 1-247
        return M_FAIL;
    }
    if (start_register > 0xFFFF) {// 寄存器地址范围是 0x0000-0xFFFF
        return M_FAIL;
    }
    if (read_length == 0 || read_length > 2000) {  // 读取长度范围是 1-
2000
        return M_FAIL;
    }

    pdata[index++] = (uint8_t)(start_register >> 8);
    pdata[index++] = (uint8_t)(start_register & 0xFF);
    pdata[index++] = (uint8_t)(read_length >> 8);
    pdata[index++] = (uint8_t)(read_length & 0xFF);

    Inf_ModBus_SendHandle(slave_addr, MODBUS_READ_HOLDING_REGISTERS,
pdata, index);
    return M_SUCCESS;
}

/// @brief 04 读输入寄存器代码
/// @param slave_addr 从机地址
/// @param start_register 起始寄存器
/// @param read_length 读取寄存器的长度
M_STATUS Inf_Modbus_Read_Input_Registers(uint8_t slave_addr, uint16_t
start_register, uint16_t read_length)
{
    uint8_t pdata[8] = {0};
    uint8_t index    = 2;

    if (slave_addr == 0 || slave_addr > 247) { // 从机地址范围是 1-247
        return M_FAIL;
    }
    if (start_register > 0xFFFF) { // 寄存器地址范围是 0x0000-0xFFFF
        return M_FAIL;
    }
    if (read_length == 0 || read_length > 2000) { // 读取长度范围是 1-
2000
        return M_FAIL;
    }

    pdata[index++] = (uint8_t)(start_register >> 8);
```

```
    pdata[index++] = (uint8_t)(start_register & 0xFF);
    pdata[index++] = (uint8_t)(read_length >> 8);
    pdata[index++] = (uint8_t)(read_length & 0xFF);
    Inf_ModBus_SendHandle(slave_addr, MODBUS_READ_INPUT_REGISTERS,
pdata, index);
    return M_SUCCESS;
}

/// @brief 05 写单个线圈
/// @param slave_addr 从机地址
/// @param start_register 寄存器
/// @param value 写入的值
M_STATUS Inf_Modbus_Write_Single_Coil(uint8_t slave_addr, uint16_t
start_register, uint16_t value)
{
    uint8_t pdata[8] = {0};
    uint8_t index    = 2;

    if (slave_addr == 0 || slave_addr > 247) { // 从机地址范围是 1-247
        return M_FAIL;
    }
    if (start_register > 0xFFFF) {// 寄存器地址范围是 0x0000-0xFFFF
        return M_FAIL;
    }

    pdata[index++] = (uint8_t)(start_register >> 8);
    pdata[index++] = (uint8_t)(start_register & 0xFF);
    pdata[index++] = value ? 0xFF : 0x00;
    pdata[index++] = 0;

    Inf_ModBus_SendHandle(slave_addr, MODBUS_WRITE_SINGLE_COIL, pdata,
index);
    return M_SUCCESS;
}

/// @brief 06 写单个寄存器
/// @param slave_addr 从机地址
/// @param start_register 寄存器
/// @param read_length 写入的值
M_STATUS Inf_Modbus_Write_Single_Register(uint8_t slave_addr, uint16_t
start_register, uint16_t value)
{
    uint8_t pdata[8] = {0};
    uint8_t index    = 2;
```

```
    if (slave_addr == 0 || slave_addr > 247) { // 从机地址范围是 1-247
        return M_FAIL;
    }
    if (start_register > 0xFFFF) { // 寄存器地址范围是 0x0000-0xFFFF
        return M_FAIL;
    }

    pdata[index++] = (uint8_t)(start_register >> 8);
    pdata[index++] = (uint8_t)(start_register & 0xFF);
    pdata[index++] = (uint8_t)(value >> 8); // 储存读取的寄存器长度
    pdata[index++] = (uint8_t)(value & 0xFF);

    Inf_ModBus_SendHandle(slave_addr, MODBUS_WRITE_SINGLE_REGISTER,
pdata, index);
    return M_SUCCESS;
}

/// @brief 0F 写多个线圈
/// @param slave_addr 从机地址
/// @param start_register 起始线圈地址
/// @param number 线圈数量
/// @param data 控制模式
M_STATUS Inf_Modbus_Write_Multiple_Coil(uint8_t slave_addr, uint16_t
start_register, uint16_t number, uint8_t *data)
{
    uint8_t pdata[256] = {0};
    uint8_t index      = 2;
    uint8_t write_len  = (uint8_t)((number + 7) / 8); // 确保向上取整

    if (number == 0 || number > 1968) { // 线圈数量范围是 1-1968
        return M_FAIL;
    }
    if (data == NULL) { // 数据指针不能为空
        return M_FAIL;
    }
    if (slave_addr == 0 || slave_addr > 247) { // 从机地址范围是 1-247
        return M_FAIL;
    }
    if (start_register > 0xFFFF) { // 寄存器地址范围是 0x0000-0xFFFF
        return M_FAIL;
    }

    pdata[index++] = (uint8_t)(start_register >> 8);
```

```
    pdata[index++] = (uint8_t)(start_register & 0xFF);
    pdata[index++] = (uint8_t)(number >> 8);
    pdata[index++] = (uint8_t)(number & 0xFF);
    pdata[index++] = write_len;

    for (int8_t i = 0; i < write_len; i++) {
        pdata[index++] = data[i];
    }

    Inf_ModBus_SendHandle(slave_addr, MODBUS_WRITE_MULTIPLE_COILS,
pdata, index);
    return M_SUCCESS;
}

/// @brief 10 写多个寄存器
/// @param slave_addr 从机地址
/// @param start_register 起始寄存器地址
/// @param number 寄存器数量
/// @param write_len 写入几个
/// @param control_mode 控制模式
M_STATUS Inf_Modbus_Write_Multiple_Register(uint8_t slave_addr,
uint16_t start_register, uint16_t number, uint16_t *data)
{
    uint8_t pdata[256] = {0};
    uint8_t index      = 2;

    if (number == 0 || number > 123) { // 线圈数量范围是 1-1968
        return M_FAIL;
    }
    if (data == NULL) { // 数据指针不能为空
        return M_FAIL;
    }
    if (slave_addr == 0 || slave_addr > 247) { // 从机地址范围是 1-247
        return M_FAIL;
    }
    if (start_register > 0xFFFF) { // 寄存器地址范围是 0x0000-0xFFFF
        return M_FAIL;
    }

    pdata[index++] = (uint8_t)(start_register >> 8);
    pdata[index++] = (uint8_t)(start_register & 0xFF);
    pdata[index++] = (uint8_t)(number >> 8);
    pdata[index++] = (uint8_t)(number & 0xFF);
```

```
    pdata[index++] = number * 2;

    for (uint8_t i = 0; i < number; i++) {
        pdata[index++] = (uint8_t)(data[i] >> 8);
        pdata[index++] = (uint8_t)(data[i] & 0xFF);
    }
    Inf_ModBus_SendHandle(slave_addr, MODBUS_WRITE_MULTIPLE_REGISTERS,
pdata, index);
    return M_SUCCESS;
}

/// @brief 对要发送的数据进行发送
/// @param slave_addr 从机地址
/// @param function_code 功能码
/// @param data 数据
/// @param data_len 数据的长度
/// @param crc crc 计算
/// @return
static M_STATUS Inf_ModBus_SendHandle(uint8_t slave_addr,
MODBUS_FUNCTION_CODE function_code, uint8_t *data, uint8_t data_len)
{

    data[0] = slave_addr;              // 赋值从设备地址
    data[1] = (uint8_t)function_code; // 赋值功能码
    // 璁＄畻 crc
    uint16_t crc = Inf_Modbus_CRC(data, data_len);

    data[data_len++] = crc & 0xff;
    data[data_len++] = crc >> 8 & 0xff;
    Inf_Modbus_Send(data, data_len);
    return M_SUCCESS;
}

void Inf_Modbus_RegisterCallback(MODBUS_FUNCTION_CODE code, void
(*callback)(uint8_t *data, uint16_t datalen))
{
    switch (code) {
        case MODBUS_READ_COILS:
            /* 处理读取线圈状态 */
            callbacks[0] = callback;
            break;
        case MODBUS_READ_DISCRETE_INPUTS:
            /* 处理读取离散输入状态? */
            callbacks[1] = callback;
```

```
            break;

        case MODBUS_READ_HOLDING_REGISTERS:
            /* 处理读取保持寄存器? */
            callbacks[2] = callback;
            break;

        case MODBUS_READ_INPUT_REGISTERS:
            /* 处理读取输入寄存器? */
            callbacks[3] = callback;
            break;

        case MODBUS_WRITE_SINGLE_COIL:
            /* 处理写单个线圈 */
            callbacks[4] = callback;
            break;

        case MODBUS_WRITE_SINGLE_REGISTER:
            /* 处理写单个保持寄存器 */
            callbacks[5] = callback;
            break;

        case MODBUS_WRITE_MULTIPLE_COILS:
            /* 处理写多个线圈 */
            callbacks[6] = callback;
            break;

        case MODBUS_WRITE_MULTIPLE_REGISTERS:
            /* 处理写多个保持寄存器 */
            callbacks[7] = callback;
            break;

        default:
             /* 处理未定义的功能码 */
            break;
    }
}

/// @brief 对串口收到的数据进行处理
/// @param data 串口收到的完整数据
/// @param data_len 数据的长度
void Inf_ModBus_ReadHandle(uint8_t *data, uint8_t data_len)
{
    printf("Inf_ModBus_ReadHandle receive data:");
```

```
    for (uint8_t i = 0; i < data_len; i++)
    {
        printf("%#x ", data[i]);
    }
    printf("\n");


    // 计算 crc,判断数据是否正确
    uint16_t crc = Inf_Modbus_CRC(data, data_len - 2);
    if (memcmp(&crc, data + data_len - 2, 2) != 0) {
        Error_Printf("modbus crc error");
        return;
    }
    // 错误响应帧
    if (data[1] & 0x80) {
        uint8_t error_code = data[2];  // 错误码位于数据帧的第 3 字节

        switch (error_code) {
            case MODBUS_ERROR_ILLEGAL_FUNCTION:
                Error_Printf("modbus error: illegal function (0x01)");
                break;
            case MODBUS_ERROR_ILLEGAL_DATA_ADDRESS:
                Error_Printf("modbus error: illegal data address
(0x02)");
                break;
            case MODBUS_ERROR_ILLEGAL_DATA_VALUE:
                Error_Printf("modbus error: illegal data value
(0x03)");
                break;
            case MODBUS_ERROR_SLAVE_DEVICE_FAILURE:
                Error_Printf("modbus error: slave device failure
(0x04)");
                break;
            case MODBUS_ERROR_ACKNOWLEDGE:
                Error_Printf("modbus error: acknowledge (0x05)");
                break;
            case MODBUS_ERROR_SLAVE_DEVICE_BUSY:
                Error_Printf("modbus error: slave device busy (0x06)");
                break;
            case MODBUS_ERROR_MEMORY_PARITY_ERROR:
                Error_Printf("modbus error: memory parity error
(0x07)");
                break;
            case MODBUS_ERROR_GATEWAY_PATH_UNAVAILABLE:
```

```
                Error_Printf("modbus error: gateway path unavailable
(0x08)");
                break;
            case MODBUS_ERROR_GATEWAY_TARGET_FAILED:
                Error_Printf("modbus error: gateway target failed
(0x09)");
                break;
            case MODBUS_ERROR_TIMEOUT:
                Error_Printf("modbus error: timeout (0x0A)");
                break;
            default:
                Error_Printf("modbus error: unknown error code
(0x%02X)", error_code);
                break;
        }
        return;
    }

    // 数据正确
    MODBUS_FUNCTION_CODE function_code = (MODBUS_FUNCTION_CODE)data[1];
    switch (function_code) // 根据不同的功能码来调用不同的函数
    {
        case MODBUS_READ_COILS:
            /* 处理读取线圈状态 */
            if (callbacks[0])
            {
                callbacks[0](data, data_len);
            }
            break;
        case MODBUS_READ_DISCRETE_INPUTS:
            /* 处理读取离散输入状态 */
            if (callbacks[1]) {
                callbacks[1](data, data_len);
            }
            break;

        case MODBUS_READ_HOLDING_REGISTERS:
            /* 处理读取保持寄存器 */
            if (callbacks[2]) {
                callbacks[2](data, data_len);
            }
            break;

        case MODBUS_READ_INPUT_REGISTERS:
```

```
            /* 处理读取输入寄存器 */
            if (callbacks[3]) {
                callbacks[3](data, data_len);
            }
            break;

        case MODBUS_WRITE_SINGLE_COIL:
            /* 处理写单个线圈 */
            if (callbacks[4]) {
                callbacks[4](data, data_len);
            }
            break;

        case MODBUS_WRITE_SINGLE_REGISTER:
            /* 处理写单个保持寄存器 */
            if (callbacks[5]) {
                callbacks[5](data, data_len);
            }

            break;

        case MODBUS_WRITE_MULTIPLE_COILS:
            /* 处理写多个线圈 */
            if (callbacks[6]) {
                callbacks[6](data, data_len);
            }
            break;

        case MODBUS_WRITE_MULTIPLE_REGISTERS:
            /* 处理写多个保持寄存器 */
            if (callbacks[7]) {
                callbacks[7](data, data_len);
            }
            break;

        default:
            /* 处理未定义的功能码 */
            break;
    }
}

/// @brief 485 计算 crc 函数
/// @param data 数据
/// @param length 数据长度
```

```
/// @return 0xDDFF, DD 为 crc 的最后一个，FF 为 crc 的前面一个
// temp[7] = Inf_Modbus_CRC(temp, 6) >> 8 & 0xff;
// temp[6] = Inf_Modbus_CRC(temp, 6) & 0xff;

static uint16_t Inf_Modbus_CRC(const uint8_t *data, uint16_t length)
{
    static const uint16_t crc_table[256] = {
        0x0000, 0xC0C1, 0xC181, 0x0140, 0xC301, 0x03C0, 0x0280, 0xC241,
        0xC601, 0x06C0, 0x0780, 0xC741, 0x0500, 0xC5C1, 0xC481, 0x0440,
        0xCC01, 0x0CC0, 0x0D80, 0xCD41, 0x0F00, 0xCFC1, 0xCE81, 0x0E40,
        0x0A00, 0xCAC1, 0xCB81, 0x0B40, 0xC901, 0x09C0, 0x0880, 0xC841,
        0xD801, 0x18C0, 0x1980, 0xD941, 0x1B00, 0xDBC1, 0xDA81, 0x1A40,
        0x1E00, 0xDEC1, 0xDF81, 0x1F40, 0xDD01, 0x1DC0, 0x1C80, 0xDC41,
        0x1400, 0xD4C1, 0xD581, 0x1540, 0xD701, 0x17C0, 0x1680, 0xD641,
        0xD201, 0x12C0, 0x1380, 0xD341, 0x1100, 0xD1C1, 0xD081, 0x1040,
        0xF001, 0x30C0, 0x3180, 0xF141, 0x3300, 0xF3C1, 0xF281, 0x3240,
        0x3600, 0xF6C1, 0xF781, 0x3740, 0xF501, 0x35C0, 0x3480, 0xF441,
        0x3C00, 0xFCC1, 0xFD81, 0x3D40, 0xFF01, 0x3FC0, 0x3E80, 0xFE41,
        0xFA01, 0x3AC0, 0x3B80, 0xFB41, 0x3900, 0xF9C1, 0xF881, 0x3840,
        0x2800, 0xE8C1, 0xE981, 0x2940, 0xEB01, 0x2BC0, 0x2A80, 0xEA41,
        0xEE01, 0x2EC0, 0x2F80, 0xEF41, 0x2D00, 0xEDC1, 0xEC81, 0x2C40,
        0xE401, 0x24C0, 0x2580, 0xE541, 0x2700, 0xE7C1, 0xE681, 0x2640,
        0x2200, 0xE2C1, 0xE381, 0x2340, 0xE101, 0x21C0, 0x2080, 0xE041,
        0xA001, 0x60C0, 0x6180, 0xA141, 0x6300, 0xA3C1, 0xA281, 0x6240,
        0x6600, 0xA6C1, 0xA781, 0x6740, 0xA501, 0x65C0, 0x6480, 0xA441,
        0x6C00, 0xACC1, 0xAD81, 0x6D40, 0xAF01, 0x6FC0, 0x6E80, 0xAE41,
        0xAA01, 0x6AC0, 0x6B80, 0xAB41, 0x6900, 0xA9C1, 0xA881, 0x6840,
        0x7800, 0xB8C1, 0xB981, 0x7940, 0xBB01, 0x7BC0, 0x7A80, 0xBA41,
        0xBE01, 0x7EC0, 0x7F80, 0xBF41, 0x7D00, 0xBDC1, 0xBC81, 0x7C40,
        0xB401, 0x74C0, 0x7580, 0xB541, 0x7700, 0xB7C1, 0xB681, 0x7640,
        0x7200, 0xB2C1, 0xB381, 0x7340, 0xB101, 0x71C0, 0x7080, 0xB041,
        0x5000, 0x90C1, 0x9181, 0x5140, 0x9301, 0x53C0, 0x5280, 0x9241,
        0x9601, 0x56C0, 0x5780, 0x9741, 0x5500, 0x95C1, 0x9481, 0x5440,
        0x9C01, 0x5CC0, 0x5D80, 0x9D41, 0x5F00, 0x9FC1, 0x9E81, 0x5E40,
        0x5A00, 0x9AC1, 0x9B81, 0x5B40, 0x9901, 0x59C0, 0x5880, 0x9841,
        0x8801, 0x48C0, 0x4980, 0x8941, 0x4B00, 0x8BC1, 0x8A81, 0x4A40,
        0x4E00, 0x8EC1, 0x8F81, 0x4F40, 0x8D01, 0x4DC0, 0x4C80, 0x8C41,
        0x4400, 0x84C1, 0x8581, 0x4540, 0x8701, 0x47C0, 0x4680, 0x8641,
        0x8201, 0x42C0, 0x4380, 0x8341, 0x4100, 0x81C1, 0x8081,
0x4040};

    uint16_t crc = 0xFFFF;
    for (uint16_t pos = 0; pos < length; pos++) {
        uint8_t index = (crc ^ data[pos]) & 0xFF;
```

```
        crc           = (crc >> 8) ^ crc_table[index];
    }
    return crc;
}
```

8.4.2 Inf_Modbus.h

```
#ifndef __INF_MODBUS_H__
#define __INF_MODBUS_H__

#include <stdlib.h>
#include <stdio.h>
#include "main.h"


// 打印宏定义

// 发送数据宏定义
typedef enum {
    M_FAIL    = -1,
    M_SUCCESS = 0,
} M_STATUS;

typedef enum {
    MODBUS_READ_COILS               = 0x01, /** 读取线圈状态 */
    MODBUS_READ_DISCRETE_INPUTS     = 0x02, /** 读取离散输入状态 */
    MODBUS_READ_HOLDING_REGISTERS   = 0x03, /** 读取保持寄存器 */
    MODBUS_READ_INPUT_REGISTERS     = 0x04, /** 读取输入寄存器 */
    MODBUS_WRITE_SINGLE_COIL        = 0x05, /** 写单个线圈 */
    MODBUS_WRITE_SINGLE_REGISTER    = 0x06, /** 写单个保持寄存器 */
    MODBUS_WRITE_MULTIPLE_COILS     = 0x0F, /** 写多个线圈 */
    MODBUS_WRITE_MULTIPLE_REGISTERS = 0x10, /** 写多个保持寄存器 */
} MODBUS_FUNCTION_CODE;

typedef enum {
    MODBUS_ERROR_ILLEGAL_FUNCTION         = 0x01, /** 错误：非法功能 */
    MODBUS_ERROR_ILLEGAL_DATA_ADDRESS     = 0x02, /** 错误：非法数据地址
*/
    MODBUS_ERROR_ILLEGAL_DATA_VALUE       = 0x03, /** 错误：非法数据值 */
    MODBUS_ERROR_SLAVE_DEVICE_FAILURE     = 0x04, /** 错误：从机设备故障
*/
```

```
    MODBUS_ERROR_ACKNOWLEDGE                = 0x05, /** 错误：确认（请求已
接收，正在处理） */
    MODBUS_ERROR_SLAVE_DEVICE_BUSY          = 0x06, /** 错误：从机设备忙 */
    MODBUS_ERROR_MEMORY_PARITY_ERROR        = 0x07, /** 错误：存储奇偶校验
错误 */
    MODBUS_ERROR_GATEWAY_PATH_UNAVAILABLE = 0x08, /** 错误：不可用网关路
径 */
    MODBUS_ERROR_GATEWAY_TARGET_FAILED      = 0x09, /** 错误：网关目标设备
响应失败 */
    MODBUS_ERROR_TIMEOUT                    = 0x0A, /** 错误：超时 */
} MODBUS_ERROR_CODE;

M_STATUS Inf_Modbus_Read_Coils(uint8_t slave_addr, uint16_t
start_register, uint16_t read_length);
M_STATUS Inf_Modbus_Read_Discrete_Inputs(uint8_t slave_addr, uint16_t
start_register, uint16_t read_length);
M_STATUS Inf_Modbus_Read_Holding_Registers(uint8_t slave_addr, uint16_t
start_register, uint16_t read_length);
M_STATUS Inf_Modbus_Read_Input_Registers(uint8_t slave_addr, uint16_t
start_register, uint16_t read_length);
M_STATUS Inf_Modbus_Write_Single_Coil(uint8_t slave_addr, uint16_t
register, uint16_t value);
M_STATUS Inf_Modbus_Write_Single_Register(uint8_t slave_addr, uint16_t
register, uint16_t value);
M_STATUS Inf_Modbus_Write_Multiple_Coil(uint8_t slave_addr, uint16_t
start_register, uint16_t number, uint8_t *data);
M_STATUS Inf_Modbus_Write_Multiple_Register(uint8_t slave_addr,
uint16_t start_register, uint16_t number, uint16_t *data);

void Inf_Modbus_RegisterCallback(MODBUS_FUNCTION_CODE code, void
(*callback)(uint8_t *data, uint16_t datalen));
void Inf_ModBus_ReadHandle(uint8_t *data, uint8_t dataLen);

#endif
```

8.4.3 Inf_Modbus_Port.c

```
#include "Inf_modbus_Port.h"
#include "FreeRTOSConfig.h"
#include "usart.h"
#include "FreeRTOS.h"
#include "task.h"
```

```
#include "semphr.h"
#include <string.h>
#include "Inf_LoRa.h"

#define RECV_BUFFER_SIZE     256
#define RECV_BUFFER_COUNT    4

#define MODBUS_TASK_NAME     "Modbus_Task"
#define MODBUS_TASK_STACK    256
#define MODBUS_TASK_PRIORITY 2

#define LORA_TASK_NAME       "Lora_Task"
#define LORA_TASK_STACK      256
#define LORA_TASK_PRIORITY   2

static uint8_t recv_buf[RECV_BUFFER_SIZE * RECV_BUFFER_COUNT] = {0};
static uint16_t recv_buf_size[RECV_BUFFER_COUNT]              = {0};
static volatile uint8_t current_read_index                    = 0;
static volatile uint8_t current_write_index                   = 0;
static void App_Lora_Task(void *pvParameters);

static SemaphoreHandle_t modbus_msg_semaphore = {0};

void Inf_Modbus_Task(void *arg)
{

    while (1) {
        if (xSemaphoreTake(modbus_msg_semaphore, portMAX_DELAY) ==
pdTRUE) {

            // 调用回调函数处理数据
            extern void Inf_ModBus_ReadHandle(uint8_t *data, uint16_t
dataLen);
            Inf_ModBus_ReadHandle(recv_buf + current_read_index *
RECV_BUFFER_SIZE, recv_buf_size[current_read_index]);

            // 清楚缓冲区
            memset(recv_buf + current_read_index * RECV_BUFFER_SIZE, 0,
RECV_BUFFER_SIZE);
            recv_buf_size[current_read_index] = 0;
            current_read_index++;
            if (current_read_index >= RECV_BUFFER_COUNT) {
                current_read_index = 0;
            }
```

```
        }
    }
}

void Inf_Modbus_Init()
{
    Inf_LoRa_Init();
    modbus_msg_semaphore = xSemaphoreCreateCounting(RECV_BUFFER_COUNT,
0);
    // 对串口的初始化


    xTaskCreate(Inf_Modbus_Task,
                MODBUS_TASK_NAME,
                MODBUS_TASK_STACK,
                NULL,
                MODBUS_TASK_PRIORITY,
                NULL);

    xTaskCreate(App_Lora_Task,
                LORA_TASK_NAME,
                LORA_TASK_STACK,
                NULL,
                LORA_TASK_PRIORITY,
                NULL);
}

void App_Lora_Task(void *pvParameters)
{
    while (1) {
        Inf_LoRa_Receive_Data(recv_buf + current_write_index *
RECV_BUFFER_SIZE, &recv_buf_size[current_write_index]);
        if (recv_buf_size[current_write_index] > 0) {
            current_write_index++;
            if (current_write_index >= RECV_BUFFER_COUNT) {

                current_write_index = 0;
            }
            xSemaphoreGiveFromISR(modbus_msg_semaphore, NULL);
        }
        vTaskDelay(50);
    }
}
```

```
// modbus 发送的代码
void Inf_Modbus_Send(uint8_t *data, uint16_t len)
{
    log_info("send:");
    for (uint8_t i = 0; i < len; i++) {
        printf("%#x ", data[i]);
    }
    printf("\n");
    // lora 发送的信息
    Inf_LoRa_Send_Data(data, len);
    HAL_Delay(600); // 为了防止发送太快,导致电机板接收不到数据
}
```

8.4.4 Inf_Modbus_Port.h

```
#ifndef __INF_MODBUS_PORT_H__
#define __INF_MODBUS_PORT_H__

#include "log.h"
#include "main.h"

// 日志输出
#define Debug_Printf log_debug
#define Error_Printf log_error

void Inf_Modbus_Init(void);
void Inf_Modbus_Send(uint8_t *data, uint16_t len);

#endif
```

8.5 以太网和 MQTT 消息接收

8.5.1 ETH

ETH.zip

8.5.2 Inf_mqtt.c

```
#include "Inf_mqtt.h"
```

```
#include "log.h"
#include <usart.h>
#include <stdlib.h>
#include <string.h>

#include "MQTTClient.h"
#include "mqtt_interface.h"
#include "socket.h"
#include "cJSON.h"
#include "FreeRTOS.h"
#include "task.h"
#include "semphr.h"
#include "w5500.h"

#define TCP_SOCKET  0
#define BUFFER_SIZE 1024
#define TARGET_IP   {44, 232, 241, 40}
#define TARGET_PORT 1883 // mqtt server port

#define PULL_TOPIC  "pull-atguigu"
#define PUSH_TOPIC  "push-atguigu"
#define MAC_ADDR    {110, 120, 130, 142, 150, 162}
#define IP_ADDR     {172, 17, 0, 55}
#define IP_MASK     {255, 255, 255, 0}
#define IP_GATEWAY  {172, 17, 0, 1}

static unsigned char recvBuf[BUFFER_SIZE] = {};
static unsigned char sendBuf[BUFFER_SIZE] = {};

static MQTTClient c;
Network n;

static void (*message_callback)(void *data, size_t data_len);

void messageArrived(MessageData *md)
{

    MQTTMessage *message = md->message;
    // 选择是否打印
    log_debug("%.*s", (int)message->payloadlen, (char
*)message->payload);
    message_callback(message->payload, message->payloadlen);
}
```

```
static void ETH_Reset(void)
{

    HAL_GPIO_WritePin(W5500_RST_GPIO_Port, W5500_RST_Pin,
GPIO_PIN_RESET);
    HAL_Delay(800);

    HAL_GPIO_WritePin(W5500_RST_GPIO_Port, W5500_RST_Pin,
GPIO_PIN_SET);
    log_info("W5500 init success\r\n");
}

static void Inf_ETH_Init(void)
{
    uint8_t mac[6]     = MAC_ADDR;   // myself MAC
    uint8_t ip[4]      = IP_ADDR;    // myself IP
    uint8_t submask[4] = IP_MASK;    // myself submask
    uint8_t gateway[4] = IP_GATEWAY; // myself gateway

    user_register_function();

    ETH_Reset();

    setSHAR(mac);

    setSIPR(ip);

    setSUBR(submask);

    setGAR(gateway);
}

// mqtt 初始化




void Inf_Mqtt_Init(void)
{
    int rc = 0;
    MQTTPacket_connectData data = MQTTPacket_connectData_initializer;
    uint8_t targetIP[4]         = TARGET_IP; // mqtt server IP
```

```
    Inf_ETH_Init();                                 // 初始化网络接口并配置 IP 地
址
    HAL_Delay(8000);                                // 等待网络稳定

    printf("W5500 init success\r\n");
    // 选择对应的 socket
    NewNetwork(&n, 5);
    rc = ConnectNetwork(&n, targetIP, TARGET_PORT);
    if (rc != SOCK_OK) {
        printf("Error connecting network\n");
    }
    printf("success connected network\n");

    MQTTClientInit(&c, &n, 1000, sendBuf, 1024, recvBuf, 1024);

    data.willFlag         = 0;
    data.MQTTVersion      = 3;
    data.clientID.cstring = (char *)"stdout-subscriber-zc";
    data.username.cstring = NULL;
    data.password.cstring = NULL;

    data.keepAliveInterval = 60;
    data.cleansession      = 1;

    rc = MQTTConnect(&c, &data);
    if (rc != SUCCESSS) {
        printf("Error connecting MQTT.\n");
    }
    printf("Connected %d\r\n", rc);

    printf("Subscribing to %s\r\n", PULL_TOPIC);
    // 接收来自这个的信息 PULL_TOPIC
    rc = MQTTSubscribe(&c, PULL_TOPIC, QOS0, messageArrived);
    printf("Subscribed %d\r\n", rc);
}

// 循环调用的函数，用来处理接收回调
void Inf_Mqtt_Yield()
{
    MQTTYield(&c, 100);
}

// 通过 mqtt 发送数据到服务器
void Inf_Mqtt_SendData(const char *data)
```

```
{

    MQTTMessage message;
    message.dup        = 0;
    message.qos        = QOS0;
    message.retained   = 0;
    message.payload    = (void *)data;
    message.payloadlen = strlen(data);
    MQTTPublish(&c, PUSH_TOPIC, &message);
}

void Inf_Mqtt_RegisterRecvCallback(void (*callback)(void *data, size_t
data_len))
{
    message_callback = callback;
}
```

8.5.3 Inf_mqtt.h

```
/*
 * @Author: wushengran
 * @Date: 2024-11-20 15:37:35
 * @Description:
 *
 * Copyright (c) 2024 by atguigu, All Rights Reserved.
 */
#ifndef __ETH_H
#define __ETH_H

#include <stdio.h>

void Inf_Mqtt_Init(void);
void Inf_Mqtt_Yield(void);

void Inf_Mqtt_SendData(const char *data);
void Inf_Mqtt_RegisterRecvCallback(void (*callback)(void *data, size_t
data_len));
#endif
```

8.6 应用层实现

8.6.1 App_Task.c

```
#include "App_Task.h"
#include <string.h>
#include "FreeRTOS.h"
#include "task.h"
#include "semphr.h"
#include "usart.h"
#include "cJSON.h"
#include "Inf_mqtt.h"
#include "queue.h"
#include "Inf_Modbus_Port.h"
#include "Inf_Modbus.h"
#include "log.h"



#define START_TASK_NAME         "Start_Task"
#define START_TASK_STACK        128
#define START_TASK_PRIORITY     5

#define MQTTYIELD_TASK_NAME     "Yield_Task"
#define MQTTYIELD_TASK_STACK    1024
#define MQTTYIELD_TASK_PRIORITY 1



static void App_MQTTYield_Task(void *pvParameters);
static void App_MessageCallback(void *data, size_t datalen);
static void App_Task_ModbusCallback(uint8_t *data, uint16_t datalen);

static void App_Start_Task(void *pvParameters)
{
    // 进入临界区
    Inf_Mqtt_RegisterRecvCallback(App_MessageCallback);
    Inf_Mqtt_Init();
    Inf_Modbus_RegisterCallback(MODBUS_READ_COILS,
App_Task_ModbusCallback);
    Inf_Modbus_RegisterCallback(MODBUS_READ_DISCRETE_INPUTS,
App_Task_ModbusCallback);
    Inf_Modbus_RegisterCallback(MODBUS_READ_HOLDING_REGISTERS,
App_Task_ModbusCallback);
    Inf_Modbus_RegisterCallback(MODBUS_READ_INPUT_REGISTERS,
App_Task_ModbusCallback);
    Inf_Modbus_Init();
```

```
    taskENTER_CRITICAL();

    // 创建任务
    xTaskCreate(App_MQTTYield_Task,
                MQTTYIELD_TASK_NAME,
                MQTTYIELD_TASK_STACK,
                NULL,
                MQTTYIELD_TASK_PRIORITY,
                NULL);



    // 销毁自身
    log_info("App_Start_over");

    vTaskDelete(NULL);
    taskEXIT_CRITICAL();
}

// 启动任务
void App_MQTTYield_Task(void *pvParameters)
{
    while (1) {
        Inf_Mqtt_Yield();
        HAL_Delay(200);
    }
}


static void App_MessageCallback(void *data, size_t datalen)
{
    // 解析 JSON 数据
    cJSON *root = cJSON_ParseWithLength((char *)data, datalen);
    if (root == NULL) {
        log_error("Error parsing JSON string.");
        return;
    }

    // 提取必填字段
    cJSON *connection_type = cJSON_GetObjectItem(root,
"connection_type");
    cJSON *device_id       = cJSON_GetObjectItem(root, "device_id");

    // 检查必填字段是否存在
```

```
    if (!cJSON_IsString(connection_type) || !cJSON_IsNumber(device_id))
{
        log_error("Invalid JSON format: missing required fields.");
        cJSON_Delete(root);
        return;
    }

    // 检查连接类型是否为 lora
    if (strcmp(connection_type->valuestring, "lora") != 0) {
        log_error("Unsupported connection type: %s",
connection_type->valuestring);
        cJSON_Delete(root);
        return;
    }
    log_info("Connection type: %s", connection_type->valuestring);

    // 获取设备地址
    uint8_t slave_addr = (uint8_t)device_id->valueint;

    // 检查是否为查询请求
    cJSON *motor_status     = cJSON_GetObjectItem(root,
"motor_status");
    cJSON *target_direction = cJSON_GetObjectItem(root,
"target_direction");
    cJSON *target_speed     = cJSON_GetObjectItem(root,
"target_speed");

    M_STATUS status;

    // 写入线圈状态（电机启停）
    if (cJSON_IsBool(motor_status)) {
        uint16_t value = motor_status->valueint ? 1 : 0; // MODBUS 绾垮湪
鐩舵€佸€?
        status         = Inf_Modbus_Write_Single_Coil(slave_addr, 2,
value);
        if (status != M_SUCCESS) {
            log_error("Failed to write coil status.");
        }
    }

    // 写入线圈状态（目标方向）
    if (cJSON_IsBool(target_direction)) {
        uint16_t value = target_direction->valueint ? 1 : 0; // MODBUS
绾垮湪鐩舵€佸€?
```

```
        status          = Inf_Modbus_Write_Single_Coil(slave_addr, 3,
value);
        if (status != M_SUCCESS) {
            log_error("Failed to write coil status for target
direction.");
        }
    }

    // 写入保持寄存器（目标转速）
    if (cJSON_IsNumber(target_speed)) {
        uint16_t value = (uint16_t)target_speed->valueint;
        status          = Inf_Modbus_Write_Single_Register(slave_addr,
2, value);
        if (status != M_SUCCESS) {
            log_error("Failed to write holding register for target
speed.");
        }
    }

    // 设置完成后，查询所有寄存器状态 ，总共 5 个
    // 查询电机启停
    status = Inf_Modbus_Read_Coils(slave_addr, 2, 1);
    if (status != M_SUCCESS) {
        log_error("Failed to read coil status.");
    }

    // 查询目标方向
    status = Inf_Modbus_Read_Coils(slave_addr, 3, 1);
    if (status != M_SUCCESS) {
        log_error("Failed to read coil status.");
    }

    // 查询当前方向
    status = Inf_Modbus_Read_Discrete_Inputs(slave_addr, 3, 1);
    if (status != M_SUCCESS) {
        log_error("Failed to read discrete input status.");
    }

    // 查询当前转速
    status = Inf_Modbus_Read_Input_Registers(slave_addr, 2, 1);
    if (status != M_SUCCESS) {
        log_error("Failed to read input register.");
    }
```

```
    // 查询目标转速
    status = Inf_Modbus_Read_Holding_Registers(slave_addr, 2, 1);
    if (status != M_SUCCESS) {
        log_error("Failed to read holding register.");
    }

    // 释放 JSON 对象
    cJSON_Delete(root);
}

static void App_Task_ModbusCallback(uint8_t *data, uint16_t datalen)
{

    static cJSON *obj = NULL;
    if (obj == NULL) {
        obj = cJSON_CreateObject();
        if (obj == NULL) {
            log_error("Failed to create JSON object.");
            return;
        }
        log_info("receive device_id: %d", data[0]);
        // 添加固定字段
        cJSON_AddStringToObject(obj, "connection_type", "lora");
        cJSON_AddNumberToObject(obj, "device_id", data[0]); // 设备地址
为 MODBUS 响应帧的第 1 字节
    }

    // 解析 MODBUS 响应帧的功能码
    uint8_t function_code = data[1];

    // 根据功能码处理数据
    switch (function_code) {
        case MODBUS_READ_COILS: {
            // 处理读取线圈状态的响应
            // 数据格式: [Slave Addr][Func Code][Byte Count][Coil Data]
            if (datalen >= 4) {
                uint8_t coil_data   =
data[3];                                    // 第 3 字节为线圈状态
                cJSON *motor_status = cJSON_GetObjectItem(obj,
"motor_status"); // 获取电机启停状态有没有
                // 如果电机启停状态有，代表进来了一次，这个是第二次进来，那就
是获取目标方向
                if (cJSON_IsBool(motor_status)) {
                    log_info("get target_direction");
```

```
                    int8_t target_direction = (coil_data & 0x01) != 0;
// 目标方向（00003）
                    cJSON_AddBoolToObject(obj, "target_direction",
target_direction);
                } else // 如果没有代表第一次进来，那就是获取电机启停
                {
                    log_info("get motor_status");
                    int8_t motor_status = (coil_data & 0x01) != 0; // 电
机启停状态（00002）
                    cJSON_AddBoolToObject(obj, "motor_status",
motor_status);
                }
            }
            break;
        }

        case MODBUS_READ_DISCRETE_INPUTS: {
            // 处理读取离散输入的响应
            // 数据格式: [Slave Addr][Func Code][Byte Count][Input Data]
            if (datalen >= 4) {
                log_info("get current_direction");
                uint8_t input_data       = data[3];                  //
第 3 字节为离散输入状态
                int8_t current_direction = (input_data & 0x01) != 0; //
当前方向（10003）
                cJSON_AddBoolToObject(obj, "current_direction",
current_direction);
            }
            break;
        }

        case MODBUS_READ_INPUT_REGISTERS: {
            // 处理读取输入寄存器的响应
            // 数据格式: [Slave Addr][Func Code][Byte Count][Register
Data High][Register Data Low]
            if (datalen >= 5) {
                log_info("get current_speed");
                uint16_t current_speed = (data[3] << 8) | data[4]; // 当
前转速（40002）
                cJSON_AddNumberToObject(obj, "current_speed",
current_speed);
            }
            break;
        }
```

```
        case MODBUS_READ_HOLDING_REGISTERS: {
            // 处理读取保持寄存器的响应
            // 数据格式：[Slave Addr][Func Code][Byte Count][Register
Data High][Register Data Low]
            if (datalen >= 5) {
                log_info("get target_speed");
                uint16_t target_speed = (data[3] << 8) | data[4]; // 目
标转速（30002）
                cJSON_AddNumberToObject(obj, "target_speed",
target_speed);
            }
            break;
        }

        default:
            log_warn("Unsupported function code: %02X", function_code);
            break;
    }

    // 检查是否所有字段都已填充
    if (cJSON_HasObjectItem(obj, "motor_status") &&
        cJSON_HasObjectItem(obj, "target_direction") &&
        cJSON_HasObjectItem(obj, "current_direction") &&
        cJSON_HasObjectItem(obj, "current_speed") &&
        cJSON_HasObjectItem(obj, "target_speed")) {
        log_info("All fields filled, sending data.");
        // 将 JSON 对象转换为字符串
        char *json_str = cJSON_Print(obj);
        cJSON_DeleteItemFromObject(obj, "motor_status");
        cJSON_DeleteItemFromObject(obj, "target_direction");
        cJSON_DeleteItemFromObject(obj, "current_direction");
        cJSON_DeleteItemFromObject(obj, "current_speed");
        cJSON_DeleteItemFromObject(obj, "target_speed");
        if (json_str != NULL) {
            log_info("json_str: %s", json_str);
            // 通过 MQTT 发送数据
            Inf_Mqtt_SendData(json_str);
            // 释放 JSON 字符串内存
            cJSON_free(json_str);
        } else {
            log_error("Failed to convert JSON to string.");
        }
    }
```

```
}

void App_FreeRTOS_Start()
{
    log_info("FreeRTOS Start");

    // mqtt 的初始化
    xTaskCreate(App_Start_Task,
                START_TASK_NAME,
                START_TASK_STACK,
                NULL,
                START_TASK_PRIORITY,
                NULL);

    vTaskStartScheduler();
    // 启动任务调度器
    log_info("Task Scheduler Started");
}
```

8.6.2 App_Task.h

```
#ifndef __APP_TASK_H__
#define __APP_TASK_H__

void App_FreeRTOS_Start(void);
#endif
```

8.6.3 main.c

```
/* USER CODE BEGIN Header */
/**
 ******************************************************************************
 * @file           : main.c
 * @brief          : Main program body
 ******************************************************************************
 * @attention
 *
 * Copyright (c) 2024 STMicroelectronics.
 * All rights reserved.
 *
```

```
 * This software is licensed under terms that can be found in the
LICENSE file
 * in the root directory of this software component.
 * If no LICENSE file comes with this software, it is provided AS-IS.
 *
 ****************************************************************
********
 */
/* USER CODE END Header */
/* Includes ------------------------------------------------------
-------*/
#include "main.h"
#include "dma.h"
#include "i2c.h"
#include "spi.h"
#include "usart.h"
#include "gpio.h"

/* Private includes ----------------------------------------------
-------*/
/* USER CODE BEGIN Includes */
#include "App_Task.h"
#include "FreeRTOS.h"
#include "task.h"
/* USER CODE END Includes */

/* Private typedef -----------------------------------------------
-------*/
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define ------------------------------------------------
-------*/
/* USER CODE BEGIN PD */

/* USER CODE END PD */

/* Private macro -------------------------------------------------
-------*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */
```

```
/* Private variables ---------------------------------------------------
-------*/

/* USER CODE BEGIN PV */

/* USER CODE END PV */

/* Private function prototypes -----------------------------------------
-------*/
void SystemClock_Config(void);
/* USER CODE BEGIN PFP */

/* USER CODE END PFP */

/* Private user code ---------------------------------------------------
-------*/
/* USER CODE BEGIN 0 */

/* USER CODE END 0 */

/**
 * @brief  The application entry point.
 * @retval int
 */
int main(void)
{
    /* USER CODE BEGIN 1 */

    /* USER CODE END 1 */

    /* MCU Configuration--------------------------------------------------
---------*/

    /* Reset of all peripherals, Initializes the Flash interface and
the Systick. */
    HAL_Init();

    /* USER CODE BEGIN Init */

    /* USER CODE END Init */

    /* Configure the system clock */
    SystemClock_Config();
```

```
    /* USER CODE BEGIN SysInit */

    /* USER CODE END SysInit */

    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    MX_DMA_Init();
    MX_USART1_UART_Init();
    MX_SPI1_Init();
    MX_USART2_UART_Init();
    MX_SPI2_Init();
    MX_I2C1_Init();
    MX_USART3_UART_Init();
    /* USER CODE BEGIN 2 */

    App_FreeRTOS_Start();
    /* USER CODE END 2 */

    /* Infinite loop */
    /* USER CODE BEGIN WHILE */

    while (1) {
        /* USER CODE END WHILE */

        /* USER CODE BEGIN 3 */
    }
    /* USER CODE END 3 */
}

/**
 * @brief System Clock Configuration
 * @retval None
 */
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

    /** Initializes the RCC Oscillators according to the specified
parameters
     * in the RCC_OscInitTypeDef structure.
     */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
    RCC_OscInitStruct.HSEState       = RCC_HSE_ON;
```

```
    RCC_OscInitStruct.HSEPredivValue = RCC_HSE_PREDIV_DIV1;
    RCC_OscInitStruct.HSIState       = RCC_HSI_ON;
    RCC_OscInitStruct.PLL.PLLState   = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource  = RCC_PLLSOURCE_HSE;
    RCC_OscInitStruct.PLL.PLLMUL     = RCC_PLL_MUL9;
    if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK) {
        Error_Handler();
    }

    /** Initializes the CPU, AHB and APB buses clocks
     */
    RCC_ClkInitStruct.ClockType      = RCC_CLOCKTYPE_HCLK |
RCC_CLOCKTYPE_SYSCLK | RCC_CLOCKTYPE_PCLK1 | RCC_CLOCKTYPE_PCLK2;
    RCC_ClkInitStruct.SYSCLKSource   = RCC_SYSCLKSOURCE_PLLCLK;
    RCC_ClkInitStruct.AHBCLKDivider  = RCC_SYSCLK_DIV1;
    RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
    RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

    if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) !=
HAL_OK) {
        Error_Handler();
    }
}

/* USER CODE BEGIN 4 */
void HAL_Delay(uint32_t Delay)
{
    vTaskDelay(Delay / portTICK_PERIOD_MS);
}
/* USER CODE END 4 */

/**
 * @brief  This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void)
{
    /* USER CODE BEGIN Error_Handler_Debug */
    /* User can add his own implementation to report the HAL error
return state */
    __disable_irq();
    while (1) {
    }
    /* USER CODE END Error_Handler_Debug */
```

```
}

#ifdef USE_FULL_ASSERT
/**
 * @brief  Reports the name of the source file and the source line
number
 *         where the assert_param error has occurred.
 * @param  file: pointer to the source file name
 * @param  line: assert_param error line source number
 * @retval None
 */
void assert_failed(uint8_t *file, uint32_t line)
{
    /* USER CODE BEGIN 6 */
    /* User can add his own implementation to report the file name and
line number,
       ex: printf("Wrong parameters value: file %s on line %d\r\n",
file, line) */
    /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */
```

第 9 章 CAN 网关实现

9.1 FreeRTOS 移植

9.1.1 FreeRTOS 库

FreeRTOS.zip

9.1.2 FreeRTOSConfig.h

```
    /*
 * FreeRTOS V202212.01
 * Copyright (C) 2020 Amazon.com, Inc. or its affiliates.  All Rights
Reserved.
 *
 * Permission is hereby granted, free of charge, to any person
obtaining a copy of
 * this software and associated documentation files (the "Software"),
to deal in
```

```
 * the Software without restriction, including without limitation the
rights to
 * use, copy, modify, merge, publish, distribute, sublicense, and/or
sell copies of
 * the Software, and to permit persons to whom the Software is
furnished to do so,
 * subject to the following conditions:
 *
 * The above copyright notice and this permission notice shall be
included in all
 * copies or substantial portions of the Software.
 *
 * THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
EXPRESS OR
 * IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
MERCHANTABILITY, FITNESS
 * FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR
 * COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER
 * IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR
IN
 * CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
 *
 * https://www.FreeRTOS.org
 * https://github.com/FreeRTOS
 *
 */

#ifndef FREERTOS_CONFIG_H
#define FREERTOS_CONFIG_H

/*-----------------------------------------------------------
 * Application specific definitions.
 *
 * These definitions should be adjusted for your particular hardware
and
 * application requirements.
 *
 * THESE PARAMETERS ARE DESCRIBED WITHIN THE 'CONFIGURATION' SECTION OF
THE
 * FreeRTOS API DOCUMENTATION AVAILABLE ON THE FreeRTOS.org WEB SITE.
 *
```

```
 * See http://www.freertos.org/a00110.html
 *----------------------------------------------------------*/

// #define configUSE_PREEMPTION 1
#define configUSE_IDLE_HOOK 0
#define configUSE_TICK_HOOK 0
#define configCPU_CLOCK_HZ ((unsigned long)72000000)
#define configTICK_RATE_HZ ((TickType_t)1000) /**/
#define configMAX_PRIORITIES (32)
#define configMINIMAL_STACK_SIZE ((unsigned short)128)
#define configTOTAL_HEAP_SIZE ((size_t)(10 * 1024))
#define configMAX_TASK_NAME_LEN (16)
// #define configUSE_TRACE_FACILITY 0
#define configUSE_16_BIT_TICKS 0 /* 时间组最多有 24 个事件 */
#define configIDLE_SHOULD_YIELD 1

/* Set the following definitions to 1 to include the API function, or
zero
to exclude the API function. */

#define INCLUDE_vTaskPrioritySet 1
#define INCLUDE_uxTaskPriorityGet 1
#define INCLUDE_vTaskDelete 1 /* 删除任务 */
#define INCLUDE_vTaskCleanUpResources 0
#define INCLUDE_vTaskSuspend 1 /* 挂起任务(暂停) */
#define INCLUDE_vTaskDelayUntil 1
#define INCLUDE_vTaskDelay 1 /* 延时 */

/* This is the raw value as per the Cortex-M3 NVIC.  Values can be 255
(lowest) to 0 (1?) (highest). */
// #define configKERNEL_INTERRUPT_PRIORITY 255
/* !!!! configMAX_SYSCALL_INTERRUPT_PRIORITY must not be set to
zero !!!!
See http://www.FreeRTOS.org/RTOS-Cortex-M3-M4.html. */
// #define configMAX_SYSCALL_INTERRUPT_PRIORITY 191 /* equivalent to
0xb0, or priority 11. */

/* This is the value being used as per the ST library which permits 16
priority values, 0 to 15.  This must correspond to the
configKERNEL_INTERRUPT_PRIORITY setting.  Here 15 corresponds to the
lowest
NVIC value of 255. */
#define configLIBRARY_KERNEL_INTERRUPT_PRIORITY 15
```

```
/* 1. 基础的必须的配置 */
#define xPortPendSVHandler PendSV_Handler /* 用来处理 PendSV 中断:系统的任务的切换 */
#define vPortSVCHandler SVC_Handler       /* svc: 实现操作系统的系统调用 */

#define INCLUDE_xTaskGetSchedulerState 1 /* 用来获取调度状态  */

/* 2. 动态创建任务 */
/* 动态内存分配 */
#define configSUPPORT_DYNAMIC_ALLOCATION 1
#define configUSE_COUNTING_SEMAPHORES 1
#define configFRTOS_MEMORY_SCHEME 4  // 或者选择其他方案，根据你的需求

/* 3. 显示任务的状态 */
#define configUSE_TRACE_FACILITY 1              /* 使用追功能 */
#define configUSE_STATS_FORMATTING_FUNCTIONS 1 /* 字符格式化 */

/* 4. 中断相关 */
/* rtos 内核的中断优先级: 配置为最低优先级, 主要是为了不影响其他中断 */
#define configKERNEL_INTERRUPT_PRIORITY (15 << 4)
/* rtos 管理的中断的阈值 */
#define configMAX_SYSCALL_INTERRUPT_PRIORITY (5 << 4)
/* 是新版本,与上面的等价 */
#define configMAX_API_CALL_INTERRUPT_PRIORITY
configMAX_SYSCALL_INTERRUPT_PRIORITY

/* 5. 时间片轮转 */
/* 抢占式调度 */
#define configUSE_PREEMPTION 1
/* 时间片轮转 */
#define configUSE_TIME_SLICING 1

// /* 6. 任务相关 api */
// #define INCLUDE_xTaskGetHandle 1
// #define INCLUDE_uxTaskGetStackHighWaterMark 1

// /* 7. 运行时间统计 */
// /* 开启时间统计 */

// #define configGENERATE_RUN_TIME_STATS 1
// /* 更改精度的计数器青 0 */
// #define portCONFIGURE_TIMER_FOR_RUN_TIME_STATS() (highCount = 0)
// /* 返回当前计数器的值 */
```

```
// #define portGET_RUN_TIME_COUNTER_VALUE() (highCount)

// /* 8. 互斥信号量 */
// #define configUSE_MUTEXES 1

// /* 9. 队列集 */
// #define configUSE_QUEUE_SETS 1

// /* 10 任务通知 */
// #define configUSE_TASK_NOTIFICATIONS 1
// /* 一个任务最多可以同时缓冲 2 个通知 */
// #define configTASK_NOTIFICATION_ARRAY_ENTRIES 2

// 这个是中断要添加的
//  #include "FreeRTOS.h"
//  #include "task.h"
//  extern void xPortSysTickHandler(void);

// if (xTaskGetSchedulerState() != taskSCHEDULER_NOT_STARTED)
// {
//     xPortSysTickHandler();
// }

#endif /* FREERTOS_CONFIG_H */
```

9.1.3 stm32f1xx_it.c

```
/* USER CODE BEGIN Header */
/**
 ******************************************************************************
 * @file    stm32f1xx_it.c
 * @brief   Interrupt Service Routines.
 ******************************************************************************
 * @attention
 *
 * Copyright (c) 2024 STMicroelectronics.
 * All rights reserved.
 *
 * This software is licensed under terms that can be found in the LICENSE file
 * in the root directory of this software component.
 * If no LICENSE file comes with this software, it is provided AS-IS.
```

```
 *
 ******************************************************************************
 */
/* USER CODE END Header */

/* Includes ------------------------------------------------------------------*/
#include "main.h"
#include "stm32f1xx_it.h"
/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */

#include "FreeRTOS.h"
#include "task.h"
#include "usart.h"
/* USER CODE END Includes */

/* Private typedef -----------------------------------------------------------*/
/* USER CODE BEGIN TD */
/* USER CODE END TD */

/* Private define ------------------------------------------------------------*/
/* USER CODE BEGIN PD */

/* USER CODE END PD */

/* Private macro -------------------------------------------------------------*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables ---------------------------------------------------------*/
/* USER CODE BEGIN PV */

/* USER CODE END PV */

/* Private function prototypes -----------------------------------------------*/
```

```
/* USER CODE BEGIN PFP */

/* USER CODE END PFP */

/* Private user code ---------------------------------------------------
-------*/
/* USER CODE BEGIN 0 */

extern void xPortSysTickHandler(void);
extern void MilliTimer_Handler(void);
/* USER CODE END 0 */

/* External variables ---------------------------------------------------
-------*/
extern CAN_HandleTypeDef hcan;
extern UART_HandleTypeDef huart1;
/* USER CODE BEGIN EV */

/* USER CODE END EV */

/******************************************************************************
********/
/*           Cortex-M3 Processor Interruption and Exception
Handlers          */
/******************************************************************************
********/
/**
  * @brief This function handles Non maskable interrupt.
  */
void NMI_Handler(void)
{
  /* USER CODE BEGIN NonMaskableInt_IRQn 0 */

  /* USER CODE END NonMaskableInt_IRQn 0 */
  /* USER CODE BEGIN NonMaskableInt_IRQn 1 */
  while (1)
  {
  }
  /* USER CODE END NonMaskableInt_IRQn 1 */
}

/**
  * @brief This function handles Hard fault interrupt.
  */
```

```
void HardFault_Handler(void)
{
  /* USER CODE BEGIN HardFault_IRQn 0 */

  /* USER CODE END HardFault_IRQn 0 */
  while (1)
  {
    /* USER CODE BEGIN W1_HardFault_IRQn 0 */
    /* USER CODE END W1_HardFault_IRQn 0 */
  }
}

/**
  * @brief This function handles Memory management fault.
  */
void MemManage_Handler(void)
{
  /* USER CODE BEGIN MemoryManagement_IRQn 0 */

  /* USER CODE END MemoryManagement_IRQn 0 */
  while (1)
  {
    /* USER CODE BEGIN W1_MemoryManagement_IRQn 0 */
    /* USER CODE END W1_MemoryManagement_IRQn 0 */
  }
}

/**
  * @brief This function handles Prefetch fault, memory access fault.
  */
void BusFault_Handler(void)
{
  /* USER CODE BEGIN BusFault_IRQn 0 */

  /* USER CODE END BusFault_IRQn 0 */
  while (1)
  {
    /* USER CODE BEGIN W1_BusFault_IRQn 0 */
    /* USER CODE END W1_BusFault_IRQn 0 */
  }
}

/**
```

```
  * @brief This function handles Undefined instruction or illegal
state.
  */
void UsageFault_Handler(void)
{
  /* USER CODE BEGIN UsageFault_IRQn 0 */

  /* USER CODE END UsageFault_IRQn 0 */
  while (1)
  {
    /* USER CODE BEGIN W1_UsageFault_IRQn 0 */

    /* USER CODE END W1_UsageFault_IRQn 0 */
  }
}

/**
  * @brief This function handles System service call via SWI
instruction.
  */
// void SVC_Handler(void)
// {
//   /* USER CODE BEGIN SVCall_IRQn 0 */

//   /* USER CODE END SVCall_IRQn 0 */
//   /* USER CODE BEGIN SVCall_IRQn 1 */

//   /* USER CODE END SVCall_IRQn 1 */
// }

/**
  * @brief This function handles Debug monitor.
  */
void DebugMon_Handler(void)
{
  /* USER CODE BEGIN DebugMonitor_IRQn 0 */

  /* USER CODE END DebugMonitor_IRQn 0 */
  /* USER CODE BEGIN DebugMonitor_IRQn 1 */

  /* USER CODE END DebugMonitor_IRQn 1 */
}

/**
```

```
  * @brief This function handles Pendable request for system service.
  */
// void PendSV_Handler(void)
// {
//   /* USER CODE BEGIN PendSV_IRQn 0 */

//   /* USER CODE END PendSV_IRQn 0 */
//   /* USER CODE BEGIN PendSV_IRQn 1 */

//   /* USER CODE END PendSV_IRQn 1 */
// }

/**
  * @brief This function handles System tick timer.
  */
void SysTick_Handler(void)
{
  /* USER CODE BEGIN SysTick_IRQn 0 */

  /* USER CODE END SysTick_IRQn 0 */
  HAL_IncTick();
  /* USER CODE BEGIN SysTick_IRQn 1 */

  if (xTaskGetSchedulerState() != taskSCHEDULER_NOT_STARTED)
  {
    xPortSysTickHandler();
  }
  MilliTimer_Handler();
  /* USER CODE END SysTick_IRQn 1 */
}

/******************************************************************************/
/* STM32F1xx Peripheral Interrupt Handlers                                    */
/* Add here the Interrupt Handlers for the used peripherals.                  */
/* For the available peripheral interrupt handler names,                      */
/* please refer to the startup file (startup_stm32f1xx.s).                    */
/******************************************************************************/
```

```
/**
  * @brief This function handles USB low priority or CAN RX0
interrupts.
  */
void USB_LP_CAN1_RX0_IRQHandler(void)
{
  /* USER CODE BEGIN USB_LP_CAN1_RX0_IRQn 0 */

  /* USER CODE END USB_LP_CAN1_RX0_IRQn 0 */
  HAL_CAN_IRQHandler(&hcan);
  /* USER CODE BEGIN USB_LP_CAN1_RX0_IRQn 1 */

  /* USER CODE END USB_LP_CAN1_RX0_IRQn 1 */
}

/**
  * @brief This function handles USART1 global interrupt.
  */
void USART1_IRQHandler(void)
{
  /* USER CODE BEGIN USART1_IRQn 0 */

  /* USER CODE END USART1_IRQn 0 */
  HAL_UART_IRQHandler(&huart1);
  /* USER CODE BEGIN USART1_IRQn 1 */

  /* USER CODE END USART1_IRQn 1 */
}

/* USER CODE BEGIN 1 */

/* USER CODE END 1 */
```

9.2 工具类

9.2.1 CJSON

cJSON.zip

9.3 CAN 协议实现

9.3.1 Inf_Can_Port.c

```
#include "Inf_Can_Port.h"
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"
#include "log.h"


#define CAN_TASK_NAME "Can_Task"
#define CAN_TASK_STACK 512
#define CAN_TASK_PRIORITY 2

static void Inf_Can_Task(void *pvParameters);
static void CAN_FilterConfig(void);

// 接收缓冲队列
#define QUEUE_LENGTH 4
QueueHandle_t xRxMsgQueue = {0};
// 发送信息，指定邮箱 0，传入标准 ID、数据、数据长度
void Inf_CAN_Init(void)
{
  // 配置过滤器
  CAN_FilterConfig();
  // 启动 can
  HAL_CAN_Start(&hcan);
  // 开启 can 中断
  HAL_CAN_ActivateNotification(&hcan, CAN_IT_RX_FIFO0_MSG_PENDING);
  // 初始化接收缓冲队列
  xRxMsgQueue = xQueueCreate(QUEUE_LENGTH, sizeof(RxMsg));

  xTaskCreate(Inf_Can_Task,
              CAN_TASK_NAME,
              CAN_TASK_STACK,
              NULL,
              CAN_TASK_PRIORITY,
              NULL);
}

void Inf_Can_Task(void *pvParameters)
{
  RxMsg msgReceived;
  while (1)
  {
    if (xQueueReceive(xRxMsgQueue, &msgReceived, portMAX_DELAY) ==
pdPASS)
```

```
    {
      extern void Inf_Can_ReadHandler(RxMsg msgReceived);
      Inf_Can_ReadHandler(msgReceived);

    }
  }
}

/// @brief 配置过滤器
/// @param
void CAN_FilterConfig(void)
{
  CAN_FilterTypeDef sFilterConfig;

  // 1. 过滤器组：0
  sFilterConfig.FilterBank = 0;

  // 2. 过滤器模式：掩码模式
  sFilterConfig.FilterMode = CAN_FILTERMODE_IDMASK;

  // 3. 过滤器位宽：32 位
  sFilterConfig.FilterScale = CAN_FILTERSCALE_32BIT;

  // 4. 关联 FIFO0
  sFilterConfig.FilterFIFOAssignment = CAN_RX_FIFO0;

  // 5. 设置 FR1 - ID，FR2 - MASK
  sFilterConfig.FilterIdHigh = 0x0000;
  sFilterConfig.FilterIdLow = 0x0000;

  sFilterConfig.FilterMaskIdHigh = 0x0000;
  sFilterConfig.FilterMaskIdLow = 0x0000;

  // 6. 激活过滤器组
  sFilterConfig.FilterActivation = ENABLE;

  HAL_CAN_ConfigFilter(&hcan, &sFilterConfig);
}

// 发送信息，指定邮箱 0，传入标准 ID、数据、数据长度
C_STATUS Inf_CAN_SendMsg(uint16_t stdID, uint8_t *data, uint8_t len)
{
  // 1. 检测是否有可用的发送邮箱
  uint32_t timeout = 0xFFFFFF;
```

```
  while (HAL_CAN_GetTxMailboxesFreeLevel(&hcan) == 0 && timeout--)
  {
    // log_info("Can Send Timeout\n");
  }
  if (timeout == 0)
  {
    return C_FAIL;
  }
  // 2. 包装数据帧头
  CAN_TxHeaderTypeDef txHeader;

  txHeader.StdId = stdID;
  txHeader.IDE = CAN_ID_STD;
  txHeader.RTR = CAN_RTR_DATA;
  txHeader.DLC = len;

  uint32_t txMailBox;
  //-----------------------------------------------
  printf("Can Send SendMsg:");
  for (uint8_t i = 0; i < len; i++)
  {
      printf("%#x ", data[i]);
  }
  printf("\n");
  log_info("Can Send sid: %#x\n",stdID);
  //-----------------------------------------------
  HAL_CAN_AddTxMessage(&hcan, &txHeader, data, &txMailBox);
  return C_SUCCESS;
}

/// @brief can 收到消息后会执行的函数
/// @param msg
void Inf_Can_IRQ_Callback(RxMsg msg)
{
  // 放入队列
  xQueueSendFromISR(xRxMsgQueue, &msg, NULL);
}
```

9.3.2 Inf_Can_Port.h

```
#ifndef __COM_CAN_PORT_H__
#define __COM_CAN_PORT_H__
```

```
#include "can.h"



// 定义接收数据信息的结构体类型
typedef struct
{
    uint16_t stdID;
    uint8_t data[8];
    uint8_t len;
} RxMsg;

typedef enum {
    C_FAIL    = -1,
    C_SUCCESS = 0,
} C_STATUS;


typedef enum {
    CAN_READ_1 = 1,
    CAN_READ_2 = 2,
    CAN_READ_3 = 3,
    CAN_READ_4 = 4,
    CAN_WRITE_5 = 5,
    CAN_WRITE_6 = 6
} C_FUNCTION_CODE;


void Inf_CAN_Init(void);
C_STATUS Inf_CAN_SendMsg(uint16_t stdID, uint8_t *data, uint8_t len);



#endif
```

9.3.3 Inf_Can.c

```
#include "Inf_Can.h"
#include "log.h"

static void (*callback)(uint8_t *, uint16_t);

/// @brief 异或校验检查
```

```
/// @param data 要校验的数据
/// @param length 数据的长度
/// @return
uint8_t CalculateCheckXor(uint8_t *data, uint8_t length)
{
    uint8_t checksum = 0;
    for (uint8_t i = 0; i < length; i++)
    {
        checksum ^= data[i];
    }
    return checksum;
}

/// @brief can 发送读数据请求
/// @param addr 地址
/// @param functionCode 功能码
/// @param startbit 起始位
/// @param length 长度
C_STATUS Inf_Can_SendReadData(uint8_t addr, C_FUNCTION_CODE
functionCode, uint8_t startbit, uint8_t length)
{
    uint8_t tempData[6];
    uint8_t index = 0;

    if (addr == 0 || addr > 247) { // 从机地址范围是 1-247
        return C_FAIL;
    }



    uint16_t sid = (uint16_t)((addr << 8) | functionCode);
    tempData[index++] = addr;
    tempData[index++] = functionCode;
    tempData[index++] = startbit;
    tempData[index++] = length;

    tempData[index++] = CalculateCheckXor(tempData, index);

    // 发送数据
    return Inf_CAN_SendMsg(sid, tempData + 2, index - 2);
}


/// @brief can 写多个数据
/// @param addr 地址
```

```
/// @param functionCode 功能码
/// @param regbit 寄存器位
/// @param length 长度
/// @param data 数据
C_STATUS Inf_Can_SendWriteData(uint8_t addr, C_FUNCTION_CODE
functionCode, uint8_t regbit, uint8_t length, uint8_t *data)
{
    uint8_t tempData[11];
    uint8_t index = 0;

    if (addr == 0 || addr > 247) { // 从机地址范围是 1-247
        return C_FAIL;
    }

    uint16_t sid = (uint16_t)((addr << 8) | functionCode);
    tempData[index++] = addr;
    tempData[index++] = (uint8_t)functionCode;
    tempData[index++] = regbit;
    tempData[index++] = length;
    for (uint8_t i = 0; i < length; i++)
    {
        tempData[index++] = data[i];
    }

    tempData[index++] = CalculateCheckXor(tempData, index);

    // 发送数据
    return Inf_CAN_SendMsg(sid, tempData + 2, index - 2);
}

/// @brief can 收到消息后会执行的函数
/// @param msgReceived
void Inf_Can_ReadHandler(RxMsg msgReceived)
{

    uint8_t rev[12];
    uint8_t index = 0;
    rev[index++] = msgReceived.stdID >> 8;
    rev[index++] = msgReceived.stdID & 0xFF;
    for (uint8_t i = 0; i < msgReceived.len; i++)
    {
        rev[index++] = msgReceived.data[i];
    }
    printf("received:");
```

```
    for (uint8_t i = 0; i < index; i++)
    {
      printf("%#x ", rev[i]);
    }
    log_info("\r\n");
    //判断校验有没有问题
    uint8_t crc = CalculateCheckXor(rev, index - 1);
    if (crc != rev[index - 1])
    {
        log_info("crc error\n");
        return;
    }
    log_info("crc ok\n");
    //判断错误帧

    //执行回调代码
    if(callback)
    {
        callback(rev, index);
    }
}

void Inf_Can_RegisterCallback(void (*cb)(uint8_t *, uint16_t))
{
    callback = cb;
}
```

9.3.4 Inf_Can.h

```
#ifndef __COM_CAN_H__
#define __COM_CAN_H__

#include "Inf_Can_Port.h"




C_STATUS Inf_Can_SendReadData(uint8_t addr, C_FUNCTION_CODE
functionCode, uint8_t startbit, uint8_t length);


C_STATUS Inf_Can_SendWriteData(uint8_t addr, C_FUNCTION_CODE
functionCode, uint8_t regbit, uint8_t length, uint8_t *data);
```

```
void Inf_Can_RegisterCallback(void (*cb)(uint8_t *, uint16_t));

#endif
```

9.4 以太网和 MQTT 消息接收

9.4.1 ETH

ETH.zip

9.4.2 Inf_mqtt.c

```
#include "Inf_mqtt.h"
#include "log.h"
#include <usart.h>
#include <stdlib.h>
#include <string.h>

#include "MQTTClient.h"
#include "mqtt_interface.h"
#include "socket.h"
#include "cJSON.h"
#include "FreeRTOS.h"
#include "task.h"
#include "semphr.h"
#include "w5500.h"

#define TCP_SOCKET  0
#define BUFFER_SIZE 1024
#define TARGET_IP   {44, 232, 241, 40}
#define TARGET_PORT 1883 // mqtt server port

#define PULL_TOPIC  "pull-atguigu"
#define PUSH_TOPIC  "push-atguigu"
#define MAC_ADDR    {110, 120, 130, 142, 150, 162}
#define IP_ADDR     {172, 17, 0, 55}
#define IP_MASK     {255, 255, 255, 0}
#define IP_GATEWAY  {172, 17, 0, 1}

static unsigned char recvBuf[BUFFER_SIZE] = {};
static unsigned char sendBuf[BUFFER_SIZE] = {};
```

```
static MQTTClient c;
Network n;

static void (*message_callback)(void *data, size_t data_len);

void messageArrived(MessageData *md)
{

    MQTTMessage *message = md->message;
    // 选择是否打印
    log_debug("%.*s", (int)message->payloadlen, (char
*)message->payload);
    message_callback(message->payload, message->payloadlen);
}

static void ETH_Reset(void)
{

    HAL_GPIO_WritePin(W5500_RST_GPIO_Port, W5500_RST_Pin,
GPIO_PIN_RESET);
    HAL_Delay(800);

    HAL_GPIO_WritePin(W5500_RST_GPIO_Port, W5500_RST_Pin,
GPIO_PIN_SET);
    log_info("W5500 init success\r\n");
}

static void Inf_ETH_Init(void)
{
    uint8_t mac[6]     = MAC_ADDR;   // myself MAC
    uint8_t ip[4]      = IP_ADDR;    // myself IP
    uint8_t submask[4] = IP_MASK;    // myself submask
    uint8_t gateway[4] = IP_GATEWAY; // myself gateway

    user_register_function();

    ETH_Reset();

    setSHAR(mac);

    setSIPR(ip);

    setSUBR(submask);
```

```
    setGAR(gateway);
}

// mqtt 初始化




void Inf_Mqtt_Init(void)
{
    int rc = 0;
    MQTTPacket_connectData data = MQTTPacket_connectData_initializer;
    uint8_t targetIP[4]         = TARGET_IP; // mqtt server IP
    Inf_ETH_Init();                          // 初始化网络接口并配置 IP 地
址
    HAL_Delay(8000);                         // 等待网络稳定

    printf("W5500 init success\r\n");
    // 选择对应的 socket
    NewNetwork(&n, 5);
    rc = ConnectNetwork(&n, targetIP, TARGET_PORT);
    if (rc != SOCK_OK) {
        printf("Error connecting network\n");
    }
    printf("success connected network\n");

    MQTTClientInit(&c, &n, 1000, sendBuf, 1024, recvBuf, 1024);

    data.willFlag         = 0;
    data.MQTTVersion      = 3;
    data.clientID.cstring = (char *)"stdout-subscriber-zc";
    data.username.cstring = NULL;
    data.password.cstring = NULL;

    data.keepAliveInterval = 60;
    data.cleansession      = 1;

    rc = MQTTConnect(&c, &data);
    if (rc != SUCCESSS) {
        printf("Error connecting MQTT.\n");
    }
    printf("Connected %d\r\n", rc);
```

```
    printf("Subscribing to %s\r\n", PULL_TOPIC);
    // 接收来自这个的信息 PULL_TOPIC
    rc = MQTTSubscribe(&c, PULL_TOPIC, QOS0, messageArrived);
    printf("Subscribed %d\r\n", rc);
}

// 循环调用的函数，用来处理接收回调
void Inf_Mqtt_Yield()
{
    MQTTYield(&c, 100);
}

// 通过 mqtt 发送数据到服务器
void Inf_Mqtt_SendData(const char *data)
{

    MQTTMessage message;
    message.dup        = 0;
    message.qos        = QOS0;
    message.retained   = 0;
    message.payload    = (void *)data;
    message.payloadlen = strlen(data);
    MQTTPublish(&c, PUSH_TOPIC, &message);
}

void Inf_Mqtt_RegisterRecvCallback(void (*callback)(void *data, size_t
data_len))
{
    message_callback = callback;
}
```

9.4.3 Inf_mqtt.h

```
/*
 * @Author: wushengran
 * @Date: 2024-11-20 15:37:35
 * @Description:
 *
 * Copyright (c) 2024 by atguigu, All Rights Reserved.
 */
```

```
#ifndef __ETH_H
#define __ETH_H

#include <stdio.h>

void Inf_Mqtt_Init(void);
void Inf_Mqtt_Yield(void);

void Inf_Mqtt_SendData(const char *data);
void Inf_Mqtt_RegisterRecvCallback(void (*callback)(void *data, size_t
data_len));
#endif
```

9.5 应用层实现

9.5.1 App_Task.c

```
#include "App_Task.h"
#include <string.h>
#include "FreeRTOS.h"
#include "task.h"
#include "semphr.h"
#include "usart.h"
#include "cJSON.h"
#include "Inf_mqtt.h"
#include "queue.h"
#include "Inf_Can_Port.h"
#include "Inf_Can.h"
#include "log.h"

#define START_TASK_NAME         "Start_Task"
#define START_TASK_STACK        256
#define START_TASK_PRIORITY     5

#define MQTTYIELD_TASK_NAME     "Yield_Task"
#define MQTTYIELD_TASK_STACK    1024
#define MQTTYIELD_TASK_PRIORITY 1

static void App_MQTTYield_Task(void *pvParameters);
static void App_MessageCallback(void *data, size_t datalen);
static void App_Task_CanCallback(uint8_t *data, uint16_t datalen);

static void App_Start_Task(void *pvParameters)
```

```
{
    // 进入临界区
    Inf_Mqtt_RegisterRecvCallback(App_MessageCallback);
    Inf_Can_RegisterCallback(App_Task_CanCallback);
    Inf_Mqtt_Init();
    Inf_CAN_Init();
    taskENTER_CRITICAL();

    // 创建任务
    xTaskCreate(App_MQTTYield_Task,
                MQTTYIELD_TASK_NAME,
                MQTTYIELD_TASK_STACK,
                NULL,
                MQTTYIELD_TASK_PRIORITY,
                NULL);

    // 销毁自身
    log_info("App_Start_over");

    vTaskDelete(NULL);
    taskEXIT_CRITICAL();
}

// 启动任务
void App_MQTTYield_Task(void *pvParameters)
{
    while (1) {
        Inf_Mqtt_Yield();
        HAL_Delay(200);
    }
}

static void App_MessageCallback(void *data, size_t datalen)
{
    // 解析 JSON 数据
    cJSON *root = cJSON_ParseWithLength((char *)data, datalen);
    if (root == NULL) {
        log_error("Error parsing JSON string.");
        return;
    }

    // 提取必填字段
    cJSON *connection_type = cJSON_GetObjectItem(root,
"connection_type");
```

```
    cJSON *device_id       = cJSON_GetObjectItem(root, "device_id");

    // 检查必填字段是否存在
    if (!cJSON_IsString(connection_type) || !cJSON_IsNumber(device_id))
{
        log_error("Invalid JSON format: missing required fields.");
        cJSON_Delete(root);
        return;
    }

    // 检查连接类型是否为 can
    if (strcmp(connection_type->valuestring, "can") != 0) {
        log_error("Unsupported connection type: %s",
connection_type->valuestring);
        cJSON_Delete(root);
        return;
    }
    log_info("Connection type: %s", connection_type->valuestring);

    // 获取设备地址
    uint8_t slave_addr = (uint8_t)device_id->valueint;

    // 检查是否为查询请求
    cJSON *motor_status     = cJSON_GetObjectItem(root,
"motor_status");
    cJSON *target_direction = cJSON_GetObjectItem(root,
"target_direction");
    cJSON *target_speed     = cJSON_GetObjectItem(root,
"target_speed");

    C_STATUS status;

    // 写入线圈状态（电机启停）
    if (cJSON_IsBool(motor_status)) {
        uint8_t value = motor_status->valueint ? 1 : 0; // MODBUS 绾垮湇
鐩舵€佸€?
        status         = Inf_Can_SendWriteData(slave_addr, CAN_WRITE_5,
2, 1, &value);
        if (status != C_SUCCESS) {
            log_error("Failed to write coil status.");
        }
    }

    // 写入线圈状态（目标方向）
```

```
    if (cJSON_IsBool(target_direction)) {
        uint8_t value = target_direction->valueint ? 1 : 0; // MODBUS 绾垮湪鐩舵€佸€?
        status         = Inf_Can_SendWriteData(slave_addr, CAN_WRITE_5, 3, 1, &value);
        if (status != C_SUCCESS) {
            log_error("Failed to write coil status for target direction.");
        }
    }

    // 写入保持寄存器（目标转速）
    if (cJSON_IsNumber(target_speed)) {
        uint8_t value[2];
        value[0] = (uint16_t)target_speed->valueint >> 8;
        value[1] = (uint16_t)target_speed->valueint & 0xFF;
        status         = Inf_Can_SendWriteData(slave_addr, CAN_WRITE_6, 2, 2, value);
        if (status != C_SUCCESS) {
            log_error("Failed to write holding register for target speed.");
        }
    }

    // 设置完成后，查询所有寄存器状态 ，总共 5 个
    // 查询电机启停
    status = Inf_Can_SendReadData(slave_addr, CAN_READ_1, 2, 1);
    if (status != C_SUCCESS) {
        log_error("Failed to read coil status.");
    }

    // 查询目标方向
    status = Inf_Can_SendReadData(slave_addr, CAN_READ_1, 3, 1);
    if (status != C_SUCCESS) {
        log_error("Failed to read coil status.");
    }

    // 查询当前方向
    status = Inf_Can_SendReadData(slave_addr, CAN_READ_2, 3, 1);
    if (status != C_SUCCESS) {
        log_error("Failed to read discrete input status.");
    }

    // 查询当前转速
```

```
    status = Inf_Can_SendReadData(slave_addr, CAN_READ_3, 2, 1);
    if (status != C_SUCCESS) {
        log_error("Failed to read input register.");
    }

    // 查询目标转速
    status = Inf_Can_SendReadData(slave_addr, CAN_READ_4, 2, 1);
    if (status != C_SUCCESS) {
        log_error("Failed to read holding register.");
    }

    // 释放 JSON 对象
    cJSON_Delete(root);
}

static void App_Task_CanCallback(uint8_t *data, uint16_t datalen)
{
    // log_info("App_Task_ModbusCallback");
    // // 创建 JSON 对象
    // for (uint8_t i = 0; i < datalen; i++)
    // {
    //     printf("%#x ",data[i]);
    // }
    // printf("\n");

    static cJSON *obj = NULL;
    if (obj == NULL) {
        obj = cJSON_CreateObject();
        if (obj == NULL) {
            log_error("Failed to create JSON object.");
            return;
        }
        log_info("receive device_id: %d", data[0]);
        // 添加固定字段
        cJSON_AddStringToObject(obj, "connection_type", "can");
        cJSON_AddNumberToObject(obj, "device_id", data[0]); // 设备地址
为 MODBUS 响应帧的第 1 字节
    }

    // 解析 MODBUS 响应帧的功能码
    uint8_t function_code = data[1];

    // 根据功能码处理数据
    switch (function_code) {
```

```
        case CAN_READ_1: {
            // 处理读取线圈状态的响应
            // 数据格式：[Slave Addr][Func Code][Byte Count][Coil Data]
            if (datalen >= 4) {
                uint8_t coil_data   =
data[3];                                    // 第 3 字节为线圈状态
                cJSON *motor_status = cJSON_GetObjectItem(obj,
"motor_status"); // 获取电机启停状态有没有
                // 如果电机启停状态有，代表进来了一次，这个是第二次进来，那就
是获取目标方向
                if (cJSON_IsBool(motor_status)) {
                    log_info("get target_direction");
                    int8_t target_direction = (coil_data & 0x01) != 0;
// 目标方向（00003）
                    cJSON_AddBoolToObject(obj, "target_direction",
target_direction);
                } else // 如果没有代表第一次进来，那就是获取电机启停
                {
                    log_info("get motor_status");
                    int8_t motor_status = (coil_data & 0x01) != 0; // 电
机启停状态（00002）
                    cJSON_AddBoolToObject(obj, "motor_status",
motor_status);
                }
            }
            break;
        }

        case CAN_READ_2: {
            // 处理读取离散输入的响应
            // 数据格式：[Slave Addr][Func Code][Byte Count][Input Data]
            if (datalen >= 4) {
                log_info("get current_direction");
                uint8_t input_data       = data[3];                  //
第 3 字节为离散输入状态
                int8_t current_direction = (input_data & 0x01) != 0; //
当前方向（10003）
                cJSON_AddBoolToObject(obj, "current_direction",
current_direction);
            }
            break;
        }

        case CAN_READ_3: {
```

```
            // 处理读取输入寄存器的响应
            // 数据格式：[Slave Addr][Func Code][Byte Count][Register
Data High][Register Data Low]
            if (datalen >= 5) {
                log_info("get current_speed");
                uint16_t current_speed = (data[3] << 8) | data[4]; // 当
前转速（40002）
                cJSON_AddNumberToObject(obj, "current_speed",
current_speed);
            }
            break;
        }

        case CAN_READ_4: {
            // 处理读取保持寄存器的响应
            // 数据格式：[Slave Addr][Func Code][Byte Count][Register
Data High][Register Data Low]
            if (datalen >= 5) {
                log_info("get target_speed");
                uint16_t target_speed = (data[3] << 8) | data[4]; // 目
标转速（30002）
                cJSON_AddNumberToObject(obj, "target_speed",
target_speed);
            }
            break;
        }

        default:
            log_warn("Unsupported function code: %02X", function_code);
            break;
    }

    // 检查是否所有字段都已填充
    if (cJSON_HasObjectItem(obj, "motor_status") &&
        cJSON_HasObjectItem(obj, "target_direction") &&
        cJSON_HasObjectItem(obj, "current_direction") &&
        cJSON_HasObjectItem(obj, "current_speed") &&
        cJSON_HasObjectItem(obj, "target_speed")) {
        log_info("All fields filled, sending data.");
        // 将 JSON 对象转换为字符串
        char *json_str = cJSON_Print(obj);
        cJSON_DeleteItemFromObject(obj, "motor_status");
        cJSON_DeleteItemFromObject(obj, "target_direction");
        cJSON_DeleteItemFromObject(obj, "current_direction");
```

```
        cJSON_DeleteItemFromObject(obj, "current_speed");
        cJSON_DeleteItemFromObject(obj, "target_speed");
        if (json_str != NULL) {
            log_info("json_str: %s", json_str);
            // 通过 MQTT 发送数据
            Inf_Mqtt_SendData(json_str);
            // 释放 JSON 字符串内存
            cJSON_free(json_str);
        } else {
            log_error("Failed to convert JSON to string.");
        }
    }
    // 测试 mqtt 发送
    // Inf_Mqtt_SendData("{\"connection_type\":
\"rs485\",\"device_id\": 5}");
    // Inf_Mqtt_SendData("aaa");
    // log_info("json_print: %s", cJSON_PrintUnformatted(obj));
    // char *json_str = cJSON_Print(obj);
    // if (json_str != NULL) {
    //     log_info("json_str: %s", json_str);
    //     // 通过 MQTT 发送数据
    //     Inf_Mqtt_SendData(json_str);
    //     // 释放 JSON 字符串内存
    //     cJSON_free(json_str);
    // } else {
    //     log_error("Failed to convert JSON to string.");
    // }
}

void App_FreeRTOS_Start()
{
    log_info("FreeRTOS Start");

    // mqtt 的初始化
    xTaskCreate(App_Start_Task,
                START_TASK_NAME,
                START_TASK_STACK,
                NULL,
                START_TASK_PRIORITY,
                NULL);

    vTaskStartScheduler();
    // 启动任务调度器
    log_info("Task Scheduler Started");
```

```
}
```

9.5.2 App_Task.h

```
#ifndef __APP_TASK_H__
#define __APP_TASK_H__

void App_FreeRTOS_Start(void);
#endif
```

9.5.3 Main.c

```
/* USER CODE BEGIN Header */
/**
 ******************************************************************************
 * @file           : main.c
 * @brief          : Main program body
 ******************************************************************************
 * @attention
 *
 * Copyright (c) 2024 STMicroelectronics.
 * All rights reserved.
 *
 * This software is licensed under terms that can be found in the LICENSE file
 * in the root directory of this software component.
 * If no LICENSE file comes with this software, it is provided AS-IS.
 *
 ******************************************************************************
 */
/* USER CODE END Header */
/* Includes ------------------------------------------------------------------*/
#include "main.h"
#include "spi.h"
#include "usart.h"
#include "gpio.h"
```

```
/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */
#include "App_Task.h"
#include "FreeRTOS.h"
#include "task.h"
#include "Inf_mqtt.h"
#include "cJSON.h"
#include "log.h"
#include <string.h>
/* USER CODE END Includes */

/* Private typedef -----------------------------------------------------------*/
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define ------------------------------------------------------------*/
/* USER CODE BEGIN PD */

/* USER CODE END PD */

/* Private macro -------------------------------------------------------------*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables ---------------------------------------------------------*/

/* USER CODE BEGIN PV */

/* USER CODE END PV */

/* Private function prototypes -----------------------------------------------*/
void SystemClock_Config(void);
/* USER CODE BEGIN PFP */

/* USER CODE END PFP */
```

```
/* Private user code ---------------------------------------------------
-------*/
/* USER CODE BEGIN 0 */

/* USER CODE END 0 */

/**
  * @brief  The application entry point.
  * @retval int
  */
int main(void)
{
  /* USER CODE BEGIN 1 */

  /* USER CODE END 1 */

  /* MCU Configuration---------------------------------------------------
-------*/

  /* Reset of all peripherals, Initializes the Flash interface and the
Systick. */
  HAL_Init();

  /* USER CODE BEGIN Init */

  /* USER CODE END Init */

  /* Configure the system clock */
  SystemClock_Config();

  /* USER CODE BEGIN SysInit */

  /* USER CODE END SysInit */

  /* Initialize all configured peripherals */
  MX_GPIO_Init();
  MX_USART1_UART_Init();
  MX_SPI1_Init();
  MX_SPI2_Init();
  /* USER CODE BEGIN 2 */

    App_FreeRTOS_Start();
  /* USER CODE END 2 */
```

```
  /* Infinite loop */
  /* USER CODE BEGIN WHILE */

    while (1) {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
    }
  /* USER CODE END 3 */
}

/**
  * @brief System Clock Configuration
  * @retval None
  */
void SystemClock_Config(void)
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

  /** Initializes the RCC Oscillators according to the specified
parameters
  * in the RCC_OscInitTypeDef structure.
  */
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
  RCC_OscInitStruct.HSEState = RCC_HSE_ON;
  RCC_OscInitStruct.HSEPredivValue = RCC_HSE_PREDIV_DIV1;
  RCC_OscInitStruct.HSIState = RCC_HSI_ON;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
  RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL9;
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
  {
    Error_Handler();
  }

  /** Initializes the CPU, AHB and APB buses clocks
  */
  RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                              |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
```

```
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) !=
HAL_OK)
  {
    Error_Handler();
  }
}

/* USER CODE BEGIN 4 */
void HAL_Delay(uint32_t Delay)
{
    vTaskDelay(Delay / portTICK_PERIOD_MS);
}
/* USER CODE END 4 */

/**
  * @brief  This function is executed in case of error occurrence.
  * @retval None
  */
void Error_Handler(void)
{
  /* USER CODE BEGIN Error_Handler_Debug */
    /* User can add his own implementation to report the HAL error
return state */
    __disable_irq();
    while (1) {
    }
  /* USER CODE END Error_Handler_Debug */
}

#ifdef  USE_FULL_ASSERT
/**
  * @brief  Reports the name of the source file and the source line
number
  *         where the assert_param error has occurred.
  * @param  file: pointer to the source file name
  * @param  line: assert_param error line source number
  * @retval None
  */
void assert_failed(uint8_t *file, uint32_t line)
{
  /* USER CODE BEGIN 6 */
```

```
    /* User can add his own implementation to report the file name and
line number,
       ex: printf("Wrong parameters value: file %s on line %d\r\n",
file, line) */
  /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */
```